Mini project report on

CrewGuard: Regulatory Compliance Validation and Violation Explanation for Airline Crew Rosters

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Samyak Sanklecha	**PES2UG24CS436**
Shriram Chandrasekar	**PES2UG24CS495**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

CrewGuard: Regulatory Compliance Validation and Violation Explanation for Airline Crew Rosters

is a bonafide work carried out by

Samyak Sanklecha	**PES2UG24CS436**
Shriram Chandrasekar	**PES2UG24CS495**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **CrewGuard: Regulatory Compliance Validation and Violation Explanation for Airline Crew Rosters** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Samyak Sanklecha	**PES2UG24CS436**	**<Signature>**
Shriram Chandrasekar	**PES2UG24CS495**	**<Signature>**

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our families and friends.

ABSTRACT

An airline crew roster can be fully staffed and still be illegal. The Directorate General of Civil Aviation limits how long crew may work and how much rest they must get, and most of those limits depend on time windows that span many duties, so a breach cannot be seen by looking at any single flight.

CrewGuard is a database-centric web application that stores an airline roster in PostgreSQL, prevents impossible data with integrity constraints and a trigger, and detects regulatory breaches with a stored procedure. Four rules are checked: minimum rest between duties (RST), maximum length of one duty (FDP), maximum duty hours in any rolling seven days (WDH) and a valid aircraft type rating (CER). Every detected violation is linked to the exact duties that caused it through a many-to-many table, which is the explanation feature of the system.

The schema has eleven relational tables in third normal form, with primary, foreign, unique, check and exclusion constraints, four functions, three views, four triggers, a validation procedure and full-text search. MongoDB stores a per-violation evidence document and an activity log, linked to PostgreSQL by the document's ObjectId, and powers the analytics dashboard through aggregation pipelines. A Next.js application with two roles, Scheduler and Auditor, provides the user interface. On the sample two-week roster the validator finds exactly the sixteen violations that were planted, and 46 automated tests pass.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**INTRODUCTION**	**1**
2	**PROBLEM DEFINITION**	**3**
3	**ER MODEL**	**5**
4	**ER TO RELATIONAL MAPPING**	**7**
5	**DDL STATEMENTS**	**12**
6	**DML STATEMENTS**	**25**
7	**QUERIES**	**37**
8	**STORED PROCEDURES, FUNCTIONS AND TRIGGERS**	**46**
9	**FRONT END DEVELOPMENT**	**55**
10	**CONCLUSION AND FUTURE ENHANCEMENTS**	**81**
	REFERENCES	**82**
	APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS	**83**

LIST OF TABLES

Table No.	Title	Page No.
1.1	Technology stack	**2**
2.1	Rules handled by CrewGuard	**3**
2.2	Roles and their permissions	**3**
3.1	Entities in the ER model	**6**
3.2	Relationships and cardinalities	**6**
4.1	Data dictionary (all tables and columns)	**11**
8.1	Triggers	**51**
9.1	Choice of database for each kind of data	**57**
10.1	Individual contributions	**81**
A.1	Definitions, acronyms and abbreviations	**83**

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	ER diagram of CrewGuard	**5**
4.1	Relational schema with every foreign key	**8**
5.1	All eleven tables created (\dt)	**12**
5.2	Structure of airport (\d airport)	**13**
5.3	Structure of aircraft_type (\d aircraft_type)	**14**
5.4	Structure of app_user (\d app_user)	**15**
5.5	Structure of crew_member (\d crew_member)	**16**
5.6	Structure of certification (\d certification)	**17**
5.7	Structure of flight (\d flight)	**18**
5.8	Structure of duty_assignment (\d duty_assignment)	**19**
5.9	Structure of rule_definition (\d rule_definition)	**20**
5.10	Structure of validation_run (\d validation_run)	**21**
5.11	Structure of violation (\d violation)	**22**
5.12	Structure of violation_assignment (\d violation_assignment)	**23**
5.13	All indexes (\di)	**24**
6.1	Rows in airport	**26**
6.2	Rows in aircraft_type	**27**
6.3	Rows in app_user (password hashes shortened)	**28**
6.4	Rows in crew_member	**29**
6.5	Rows in certification	**30**
6.6	Rows in flight	**31**
6.7	Rows in duty_assignment (first 24 of 62)	**32**
6.8	Rows in rule_definition	**33**
6.9	Rows in validation_run after one run	**34**
6.10	Rows in violation: the 16 planted breaches	**34**
6.11	Rows in violation_assignment: the duties behind each violation	**35**

6.12	A valid INSERT inside a transaction (rolled back afterwards)	**36**
6.13	Invalid INSERTs rejected by UNIQUE and CHECK constraints	**36**
7.1	GROUP BY with aggregates, HAVING and FILTER	**38**
7.2	UPDATE statements, including one rejected by a CHECK constraint	**39**
7.3	DELETE with ON DELETE CASCADE, and a DELETE blocked by RESTRICT	**40**
7.4	Correlated sub-queries	**41**
7.5	Nested queries with NOT IN and NOT EXISTS	**42**
7.6	Querying the v_crew_summary and v_violation_report views	**43**
7.7	LAG and running SUM over one crew member's duties	**44**
7.8	Full-text search and EXPLAIN ANALYZE using the composite index	**45**
8.1	The four functions and their results	**47**
8.2	CALL sp_run_validation for the first week	**50**
8.3	The explanation: the duties behind a violation add up to the observed value	**50**
8.4	The triggers installed in the database	**52**
8.5	trg_validate_duty rejecting an inactive crew member, a wrong role and a duty that does not cover its flight	**52**
8.6	Compliance triggers maintaining the count and the review time, and the resolution-note CHECK	**54**
9.1	System architecture	**55**
9.2	MongoDB collections, document counts and indexes	**57**
9.3	A WDH evidence document (violation 11, Farhan Ali)	**58**
9.4	Evidence documents for an RST and a CER violation: different shapes per rule	**59**
9.5	Aggregation pipelines behind the dashboard	**59**
9.6	The activity log in audit_log	**60**
9.7	Sign-in page with the two demo accounts	**61**
9.8	Overview (scheduler): roster statistics, duties per day, busiest crew and ratings needing attention	**62**
9.9	Crew list with search and filters, read from v_crew_summary	**63**
9.10	Crew detail: weekly duty hours against the 60-hour limit, type ratings and roster	**64**

9.11	Flight detail with the assign-crew form and warnings computed with EXISTS and range overlap	**65**
9.12	Duty roster read from v_duty_roster	**66**
9.13	Database rejecting a flight with the same departure and arrival airport (CHECK)	**67**
9.14	Database rejecting a duplicate employee code (UNIQUE)	**67**
9.15	Assigning two crew in one transaction: the overlap is rejected by EXCLUDE and neither duty is saved	**68**
9.16	Validation runs (auditor) with the run form	**69**
9.17	Result of one run: violations per rule	**70**
9.18	Violations list from v_violation_report with filters	**71**
9.19	Explanation page for an RST violation: rule, observed value, causing duties, MongoDB evidence and review	**72**
9.20	Explanation page for an FDP violation (resolved with a note)	**73**
9.21	Explanation page for a WDH violation: six duties adding up to 66 hours	**74**
9.22	Explanation page for a CER violation: no rating for the aircraft type	**75**
9.23	Explanation page for a CER violation: rating expired	**76**
9.24	Rule catalogue with full-text search	**77**
9.25	Dashboard: PostgreSQL GROUP BY next to MongoDB aggregation pipelines	**78**
9.26	Activity log read from the MongoDB audit_log collection	**79**
9.27	Output of npm test: 46 tests, 46 passed	**80**

1. INTRODUCTION

Airlines plan who works which flight in advance. This plan is the crew roster: every row says that one crew member works one flight, from a report time before departure to a release time after arrival. To keep fatigue in check, India's Directorate General of Civil Aviation (DGCA) limits duty and rest through its Civil Aviation Requirements (CAR), for example a minimum rest between duties, a maximum length for one duty and a maximum number of duty hours in any seven consecutive days. Pilots must also hold a valid type rating for the aircraft they fly.

These limits are hard to check by hand because most of them are about time windows across many duties. A captain who flies six 11-hour duties in six days has nothing wrong on any single flight, yet the week adds up to 66 hours, above the 60-hour limit. A roster can be fully staffed and still be illegal.

CrewGuard is a database-centric application that stores a roster, refuses impossible data, detects regulatory breaches and explains each breach by pointing at the duties that caused it. It is a detector and an explainer, not a roster generator: a scheduler builds the roster, and an auditor runs the validator over a date range and reviews what it finds.

1.1 Objectives

- Store airports, aircraft, crew, type ratings, flights and duties in PostgreSQL, with the integrity rules enforced by the database itself.
- Prevent impossible data, such as one person on two overlapping duties, with constraints and a trigger.
- Detect breaches of four DGCA-style rules with a stored procedure, reading every limit from a rule table rather than from code.
- Explain every breach by linking it to the duties that caused it through a many-to-many table.
- Give auditors a review workflow (OPEN, ACKNOWLEDGED, RESOLVED) with an accountable trail of who did what.
- Keep per-violation evidence and an activity log in MongoDB, and use aggregation pipelines for analytics.
- Provide a web interface with authentication and two roles, and an automated test suite.

1.2 Technology used

Layer	Technology	Purpose
Relational database	PostgreSQL 16	Tables, constraints, views, functions, triggers, procedure, full-text search
NoSQL database	MongoDB 7	Violation evidence documents, activity log, aggregation pipelines
Backend	Next.js 16 route handlers (Node.js), node-postgres (`pg`), MongoDB driver	API, parameterised SQL, transactions
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4	Pages, forms, tables, dashboard
Security	bcrypt password hashes, signed JWT session cookie, role checks	Authentication and authorisation
Testing	Node.js test runner	46 tests against the real databases

Table 1.1: *Technology stack*

2. PROBLEM DEFINITION

Given a crew roster for a period, the system must store it with full integrity, and on request find every crew member who breaks a duty-time or certification rule in a chosen date range, record each breach, and show exactly which duties caused it. The rules checked are listed below. The thresholds are stored in the rule_definition table, so a limit can change without changing any code.

Code	Rule	Limit	Severity	How it is enforced
RST	Minimum rest between two consecutive duties	at least 10 h	HIGH	Detected by the validator (LAG window function)
FDP	Maximum length of a single flight duty period	at most 13 h	CRITICAL	Detected by the validator
WDH	Maximum duty hours in any rolling 7 days	at most 60 h	MEDIUM	Detected by the validator (rolling SUM over a 7-day RANGE)
CER	Pilot holds a valid type rating for the aircraft flown (cabin crew exempt)	valid on the duty date	CRITICAL	Detected by the validator (NOT EXISTS)
–	One crew member cannot be on two overlapping duties	no overlap	–	Prevented by an EXCLUDE constraint

Table 2.1: *Rules handled by CrewGuard*

Two kinds of protection are used on purpose. Impossible data (overlapping duties, an inactive crew member on a flight, a duty that does not cover its flight) is prevented: the database rejects the row. Data that is possible but illegal (too little rest, too many hours, an expired rating) is accepted, because a scheduler may need to save a draft, and is then detected and reported by the validator.

2.1 Users and functional requirements

Role	Can do
Both roles	Sign in and out; view the overview, crew, flights, duty roster, airports and fleet, validation runs, violations, rules, dashboard and activity log; search and filter every list.
SCHEDULER (sched_sam)	Add, edit and delete crew members; add and remove type ratings; schedule and delete flights; assign several crew to a flight in one transaction; remove a duty.
AUDITOR (audit_shriram)	Run a validation over a date range; review a violation by moving it from OPEN to ACKNOWLEDGED to RESOLVED with a note.

Table 2.2: *Roles and their permissions*

Non-functional requirements: passwords are stored only as bcrypt hashes, database credentials live in an environment file that is never committed, every write is checked by the database, and every role check is repeated on the server so that hiding a button is never the only protection.

2.2 Sample data

The seed data is a two-week roster from 2 to 15 November 2026 (all times in UTC) with 8 airports, 4 aircraft types, 13 crew, 13 type ratings, 24 flights and 62 duties. Breaches were planted on purpose so the expected output is known: a run over the whole period must find 16 violations (RST 7, FDP 2, WDH 3, CER 4).

3. ER MODEL

The ER model has eleven entity types. The two central relationships are many-to-many: a crew member works many flights and a flight has many crew (resolved by DUTY_ASSIGNMENT, which also carries the duty times and role), and a violation is caused by many duties while one duty can contribute to several violations (resolved by VIOLATION_ASSIGNMENT, which carries the hours each duty contributed). This second relationship is what lets the system explain a violation.

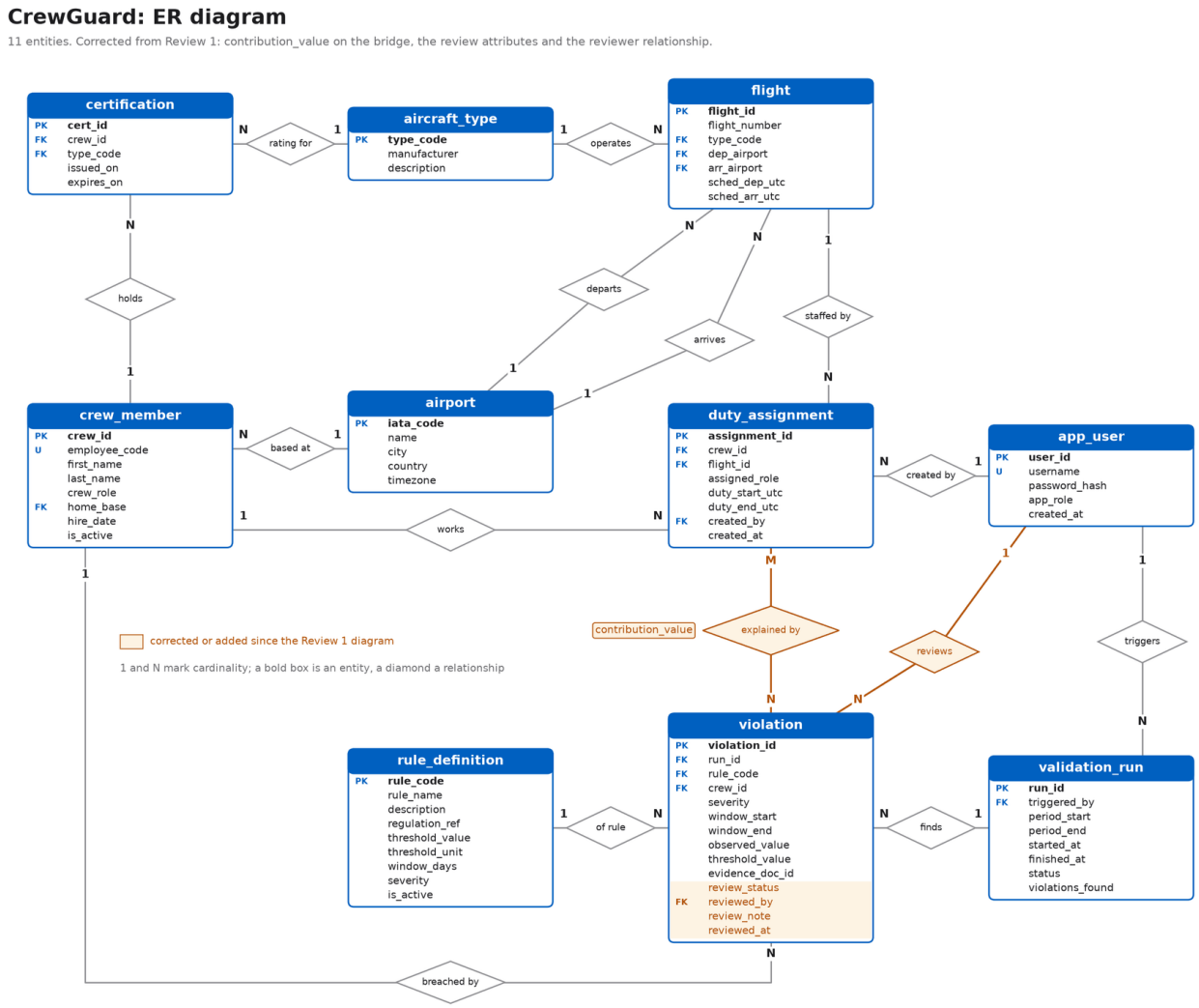


Figure 3.1: *ER diagram of CrewGuard*

Entity	Key	Description
AIRPORT	iata_code	An airport, identified by its three-letter IATA code
AIRCRAFT_TYPE	type_code	An aircraft model that crew can be rated on (A320, B788, ...)

Entity	Key	Description
APP_USER	user_id	A login account with role SCHEDULER or AUDITOR
CREW_MEMBER	crew_id	A captain, first officer or cabin crew member, with a home base
CERTIFICATION	cert_id	A type rating: crew member X may operate type Y between two dates
FLIGHT	flight_id	One scheduled flight between two airports on one aircraft type
DUTY_ASSIGNMENT	assignment_id	One crew member working one flight (the roster)
RULE_DEFINITION	rule_code	A regulation with its threshold, window and severity
VALIDATION_RUN	run_id	One execution of the validator over a date range
VIOLATION	violation_id	One breach of one rule by one crew member in one run
VIOLATION_ASSIGNMENT	(violation_id, assignment_id)	Which duties caused which violation

Table 3.1: *Entities in the ER model*

Relationship	Cardinality	Participation
AIRPORT is home base of CREW_MEMBER	1 : N	every crew member has one base
AIRPORT is departure / arrival of FLIGHT	1 : N (two relationships)	every flight has both
AIRCRAFT_TYPE operates FLIGHT	1 : N	every flight has one type
CREW_MEMBER holds CERTIFICATION for AIRCRAFT_TYPE	1 : N and N : 1	a crew member may hold none
CREW_MEMBER works FLIGHT (via DUTY_ASSIGNMENT)	M : N	partial on both sides
APP_USER created DUTY_ASSIGNMENT	1 : N	optional (SET NULL)
APP_USER triggered VALIDATION_RUN / reviewed VIOLATION	1 : N	optional (SET NULL)
VALIDATION_RUN finds VIOLATION	1 : N	every violation belongs to a run
RULE_DEFINITION is broken in VIOLATION	1 : N	every violation has one rule
CREW_MEMBER commits VIOLATION	1 : N	every violation has one crew member
VIOLATION caused by DUTY_ASSIGNMENT (via VIOLATION_ASSIGNMENT)	M : N	every violation has at least one duty

Table 3.2: *Relationships and cardinalities*

4. ER TO RELATIONAL MAPPING

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

The standard seven-step ER-to-relational mapping algorithm was applied as follows.

- **Step 1, strong entity types:** every strong entity becomes a table with its simple attributes and its key as the primary key: airport (iata_code), aircraft_type (type_code), app_user (user_id), crew_member (crew_id), flight (flight_id), rule_definition (rule_code), validation_run (run_id). Surrogate keys use SERIAL; natural codes such as IATA codes and rule codes are used directly.
- **Step 2, weak entity types:** there are none. A type rating could be seen as weak (it has no meaning without its crew member), but renewals create new rows, so certification gets its own key cert_id and the natural key (crew_id, type_code, issued_on) is kept as a UNIQUE constraint.
- **Step 3, binary 1:1 relationships:** there are none in this model.
- **Step 4, binary 1:N relationships:** the key of the 1-side becomes a foreign key in the N-side table: crew_member.home_base → airport, flight.dep_airport and flight.arr_airport → airport, flight.type_code → aircraft_type, certification.crew_id → crew_member, certification.type_code → aircraft_type, violation.run_id → validation_run, violation.rule_code → rule_definition, violation.crew_id → crew_member, and the optional user links duty_assignment.created_by, validation_run.triggered_by and violation.reviewed_by → app_user.
- **Step 5, binary M:N relationships:** each becomes its own table holding both foreign keys. CREW_MEMBER–FLIGHT becomes duty_assignment, which also stores the relationship attributes assigned_role, duty_start_utc and duty_end_utc; (crew_id, flight_id) is UNIQUE and a surrogate assignment_id is the primary key because other tables reference a duty. VIOLATION–DUTY becomes violation_assignment with the composite primary key (violation_id, assignment_id) and the attribute contribution_value.
- **Step 6, multivalued attributes:** there are none; a crew member's several ratings are already modelled as the CERTIFICATION entity.
- **Step 7, n-ary relationships:** there are none; every relationship is binary.
- **Referential actions:** child rows that cannot exist alone use ON DELETE CASCADE (a crew member's ratings and duties), master data uses ON DELETE RESTRICT (an airport in use cannot be deleted), and optional author links use ON DELETE SET NULL.

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

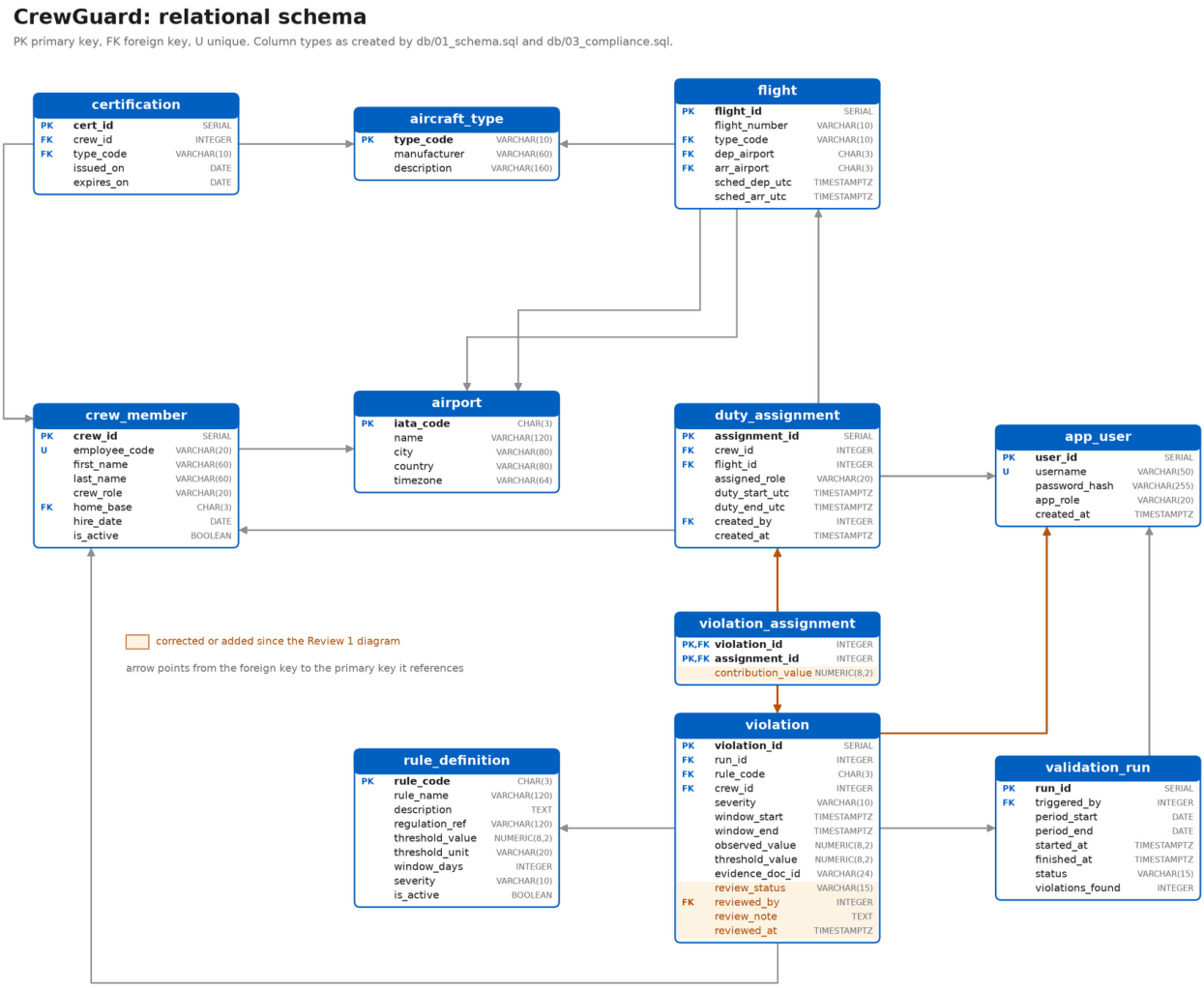


Figure 4.1: *Relational schema with every foreign key*

4.3 Normalisation

Each table is about one thing. Crew details are stored once in crew_member and duties hold only crew_id; airport and aircraft details live in their own tables; a renewed rating is a new certification row, so history is kept. Every non-key column depends on the key, the whole key and nothing but the key, so the schema is in third normal form and free of update, insert and delete anomalies. One copy is deliberate: violation stores severity and threshold_value from rule_definition as a snapshot, so an old violation keeps the limit it was judged against even if the rule changes later.

4.4 Data dictionary

The data dictionary below is generated from the live database catalogue (information_schema), so it matches the deployed schema exactly.

Table	Column	Type	Null	Key
airport	iata_code	char(3)	NOT NULL	PK
airport	name	varchar(120)	NOT NULL	
airport	city	varchar(80)	NOT NULL	
airport	country	varchar(80)	NOT NULL	
airport	timezone	varchar(64)	NOT NULL	
aircraft_type	type_code	varchar(10)	NOT NULL	PK
aircraft_type	manufacturer	varchar(60)	NOT NULL	
aircraft_type	description	varchar(160)		
app_user	user_id	integer	NOT NULL	PK
app_user	username	varchar(50)	NOT NULL	UNIQUE
app_user	password_hash	varchar(255)	NOT NULL	
app_user	app_role	varchar(20)	NOT NULL	
app_user	created_at	timestamptz	NOT NULL	
crew_member	crew_id	integer	NOT NULL	PK
crew_member	employee_code	varchar(20)	NOT NULL	UNIQUE
crew_member	first_name	varchar(60)	NOT NULL	
crew_member	last_name	varchar(60)	NOT NULL	
crew_member	crew_role	varchar(20)	NOT NULL	
crew_member	home_base	char(3)	NOT NULL	FK
crew_member	hire_date	date	NOT NULL	
crew_member	is_active	boolean	NOT NULL	
certification	cert_id	integer	NOT NULL	PK
certification	crew_id	integer	NOT NULL	FK, UNIQUE
certification	type_code	varchar(10)	NOT NULL	FK, UNIQUE
certification	issued_on	date	NOT NULL	UNIQUE
certification	expires_on	date	NOT NULL	
flight	flight_id	integer	NOT NULL	PK
flight	flight_number	varchar(10)	NOT NULL	UNIQUE
flight	type_code	varchar(10)	NOT NULL	FK
flight	dep_airport	char(3)	NOT NULL	FK

Table	Column	Type	Null	Key
flight	arr_airport	char(3)	NOT NULL	FK
flight	sched_dep_utc	timestamptz	NOT NULL	UNIQUE
flight	sched_arr_utc	timestamptz	NOT NULL	
duty_assignment	assignment_id	integer	NOT NULL	PK
duty_assignment	crew_id	integer	NOT NULL	FK, UNIQUE
duty_assignment	flight_id	integer	NOT NULL	FK, UNIQUE
duty_assignment	assigned_role	varchar(20)	NOT NULL	
duty_assignment	duty_start_utc	timestamptz	NOT NULL	
duty_assignment	duty_end_utc	timestamptz	NOT NULL	
duty_assignment	created_by	integer		FK
duty_assignment	created_at	timestamptz	NOT NULL	
rule_definition	rule_code	char(3)	NOT NULL	PK
rule_definition	rule_name	varchar(120)	NOT NULL	
rule_definition	description	text	NOT NULL	
rule_definition	regulation_ref	varchar(120)		
rule_definition	threshold_value	numeric(8,2)		
rule_definition	threshold_unit	varchar(20)		
rule_definition	window_days	integer		
rule_definition	severity	varchar(10)	NOT NULL	
rule_definition	is_active	boolean	NOT NULL	
validation_run	run_id	integer	NOT NULL	PK
validation_run	triggered_by	integer		FK
validation_run	period_start	date	NOT NULL	
validation_run	period_end	date	NOT NULL	
validation_run	started_at	timestamptz	NOT NULL	
validation_run	finished_at	timestamptz		
validation_run	status	varchar(15)	NOT NULL	
validation_run	violations_found	integer	NOT NULL	
violation	violation_id	integer	NOT NULL	PK
violation	run_id	integer	NOT NULL	FK
violation	rule_code	char(3)	NOT NULL	FK
violation	crew_id	integer	NOT NULL	FK
violation	severity	varchar(10)	NOT NULL	

Table	Column	Type	Null	Key
violation	window_start	timestamptz		
violation	window_end	timestamptz		
violation	observed_value	numeric(8,2)		
violation	threshold_value	numeric(8,2)		
violation	evidence_doc_id	varchar(24)		
violation	review_status	varchar(15)	NOT NULL	
violation	reviewed_by	integer		FK
violation	review_note	text		
violation	reviewed_at	timestamptz		
violation_assignment	violation_id	integer	NOT NULL	FK, PK
violation_assignment	assignment_id	integer	NOT NULL	FK, PK
violation_assignment	contribution_value	numeric(8,2)		

Table 4.1: *Data dictionary (all tables and columns)*

5. DDL STATEMENTS

The schema is created by db/01_schema.sql, and the compliance additions by db/03_compliance.sql. The script starts with DROP TABLE ... CASCADE so it can be re-run from zero, and creates parent tables before the tables whose foreign keys point at them. Each statement is followed by a screenshot of the created table (\d in psql), which lists its columns, indexes, constraints, references and triggers.

```
DROP TABLE IF EXISTS violation_assignment, violation, validation_run,
                     rule_definition, duty_assignment, flight, certification,
                     crew_member, app_user, aircraft_type, airport CASCADE;

-- btree_gist lets a GiST index compare plain values (crew_id WITH =)
-- alongside ranges; the no-overlap EXCLUDE constraint needs it.
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

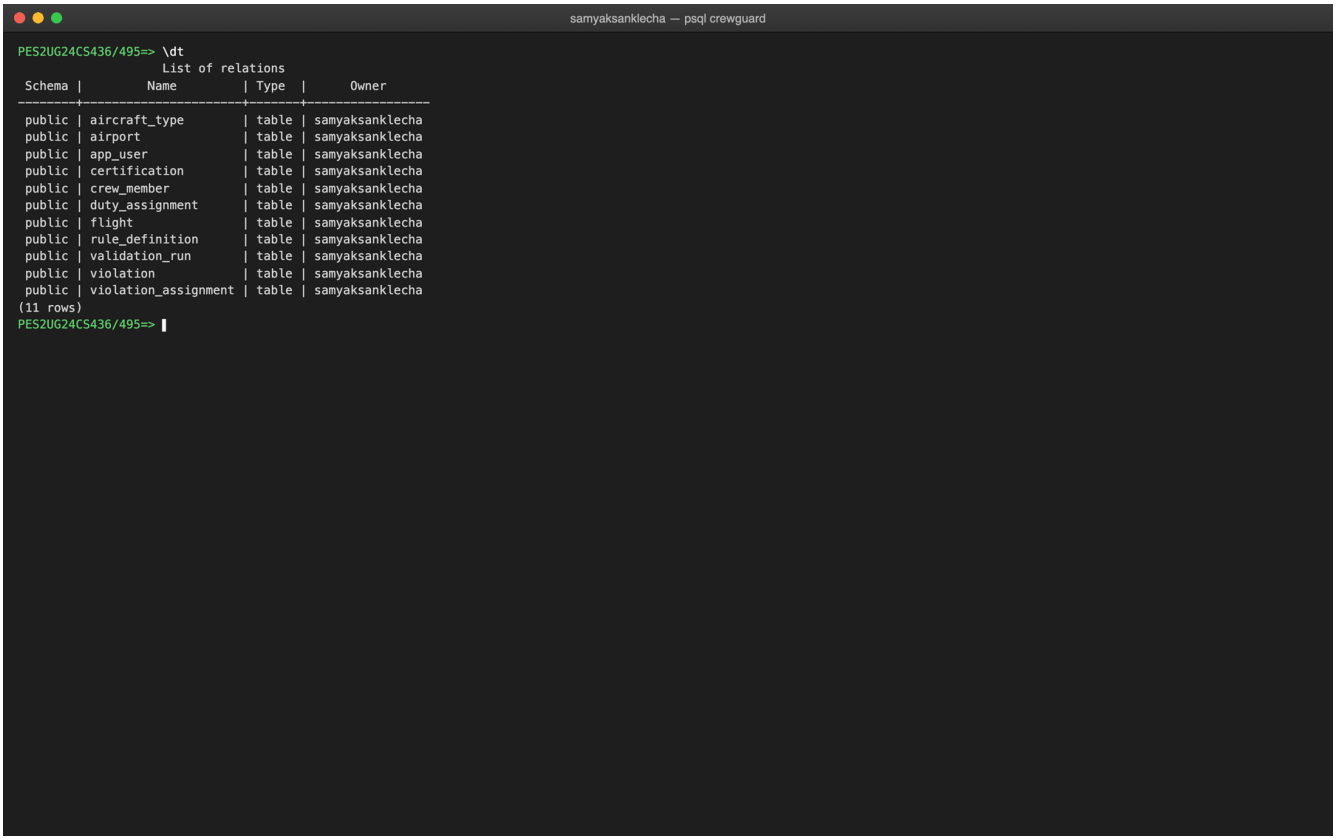


Figure 5.1: *All eleven tables created (\dt)*

Table airport

```
CREATE TABLE airport (
    iata_code       CHAR(3)      PRIMARY KEY,
    name            VARCHAR(120) NOT NULL,
    city            VARCHAR(80)  NOT NULL,
    country         VARCHAR(80)  NOT NULL,
    timezone        VARCHAR(64)  NOT NULL,
    -- regex check: exactly three capital letters, e.g. BLR
    CONSTRAINT chk_airport_iata_format CHECK (iata_code ~ '^[A-Z]{3}$')
);
```

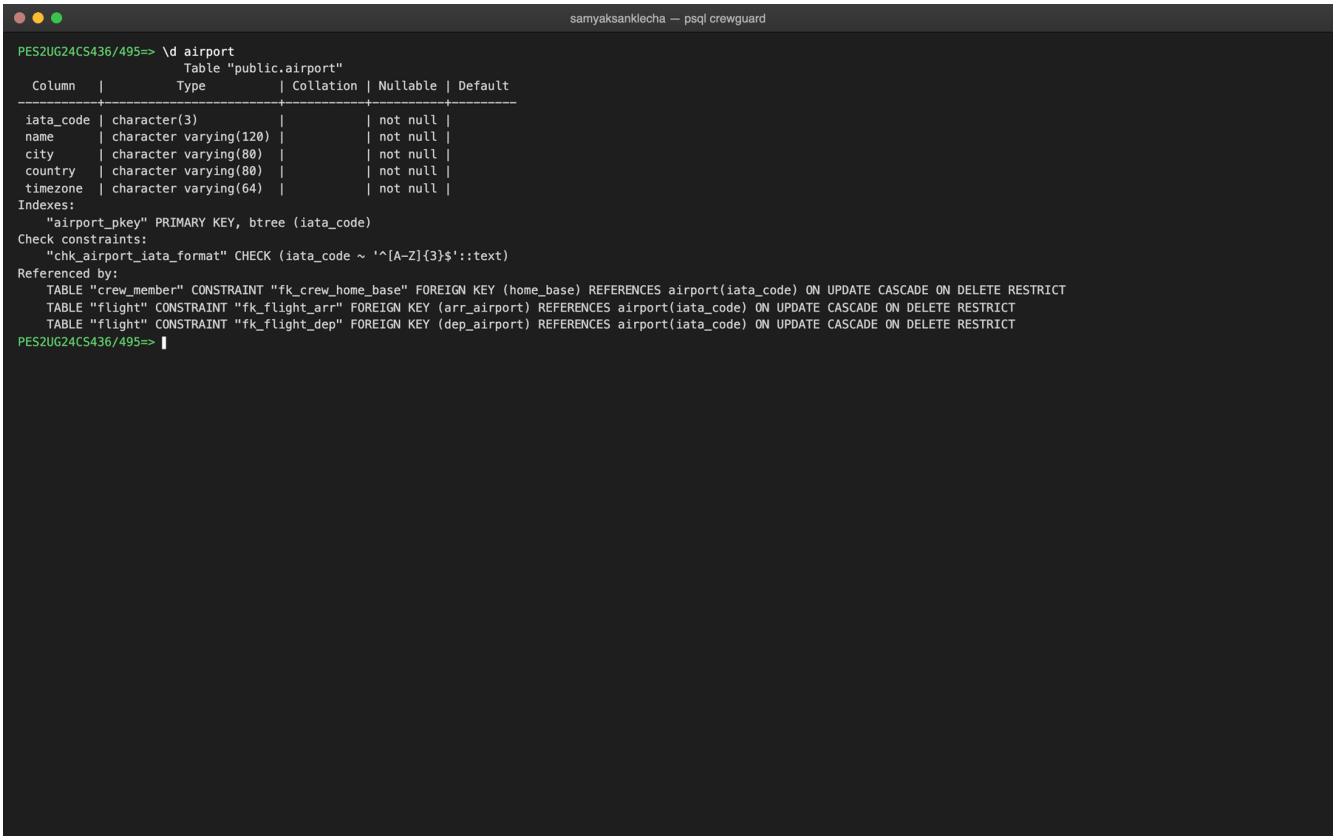


Figure 5.2: *Structure of airport (\d airport)*

Table aircraft_type

```
CREATE TABLE aircraft_type (
    type_code        VARCHAR(10)  PRIMARY KEY,
    manufacturer     VARCHAR(60)  NOT NULL,
    description      VARCHAR(160),
    CONSTRAINT chk_aircraft_type_upper CHECK (type_code = UPPER(type_code))
);
```

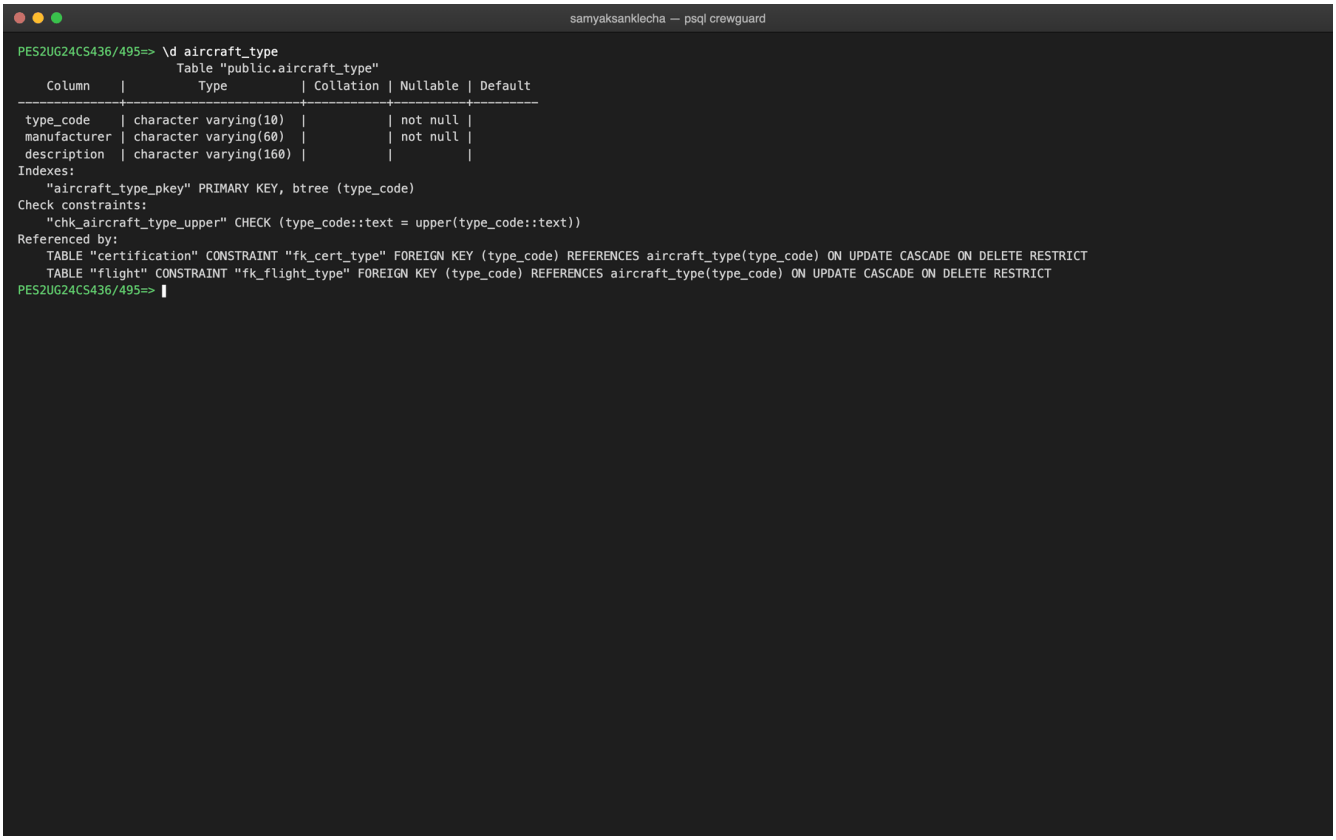


Figure 5.3: *Structure of aircraft_type (\d aircraft_type)*

Table app_user

```
CREATE TABLE app_user (
    user_id          SERIAL       PRIMARY KEY,
    username         VARCHAR(50)  NOT NULL UNIQUE,
    password_hash    VARCHAR(255) NOT NULL,
    app_role         VARCHAR(20)  NOT NULL,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_app_role CHECK (app_role IN ('SCHEDULER', 'AUDITOR'))
);
```

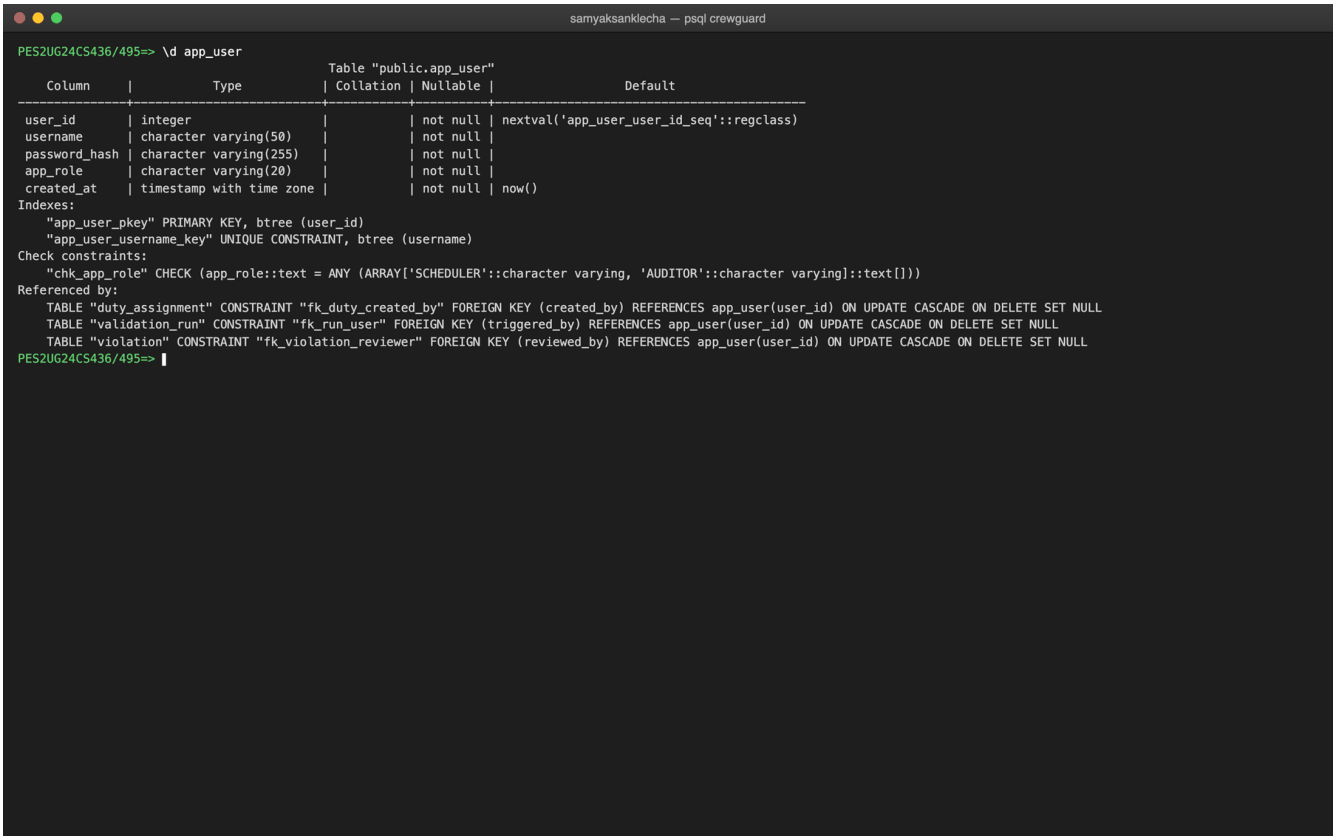


Figure 5.4: *Structure of app_user (\d app_user)*

Table crew_member

```
CREATE TABLE crew_member (
    crew_id          SERIAL        PRIMARY KEY,
    employee_code    VARCHAR(20)   NOT NULL UNIQUE,
    first_name       VARCHAR(60)   NOT NULL,
    last_name        VARCHAR(60)   NOT NULL,
    crew_role        VARCHAR(20)   NOT NULL,
    home_base        CHAR(3)       NOT NULL,
    hire_date        DATE          NOT NULL,
    is_active        BOOLEAN       NOT NULL DEFAULT TRUE,
    -- an airport that is someone's home base cannot be deleted (RESTRICT)
    CONSTRAINT fk_crew_home_base FOREIGN KEY (home_base) REFERENCES airport(iata_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_crew_role CHECK (crew_role IN ('CAPTAIN', 'FIRST_OFFICER', 'CABIN_CREW')),
    CONSTRAINT chk_crew_hire_date CHECK (hire_date <= CURRENT_DATE)
);
```

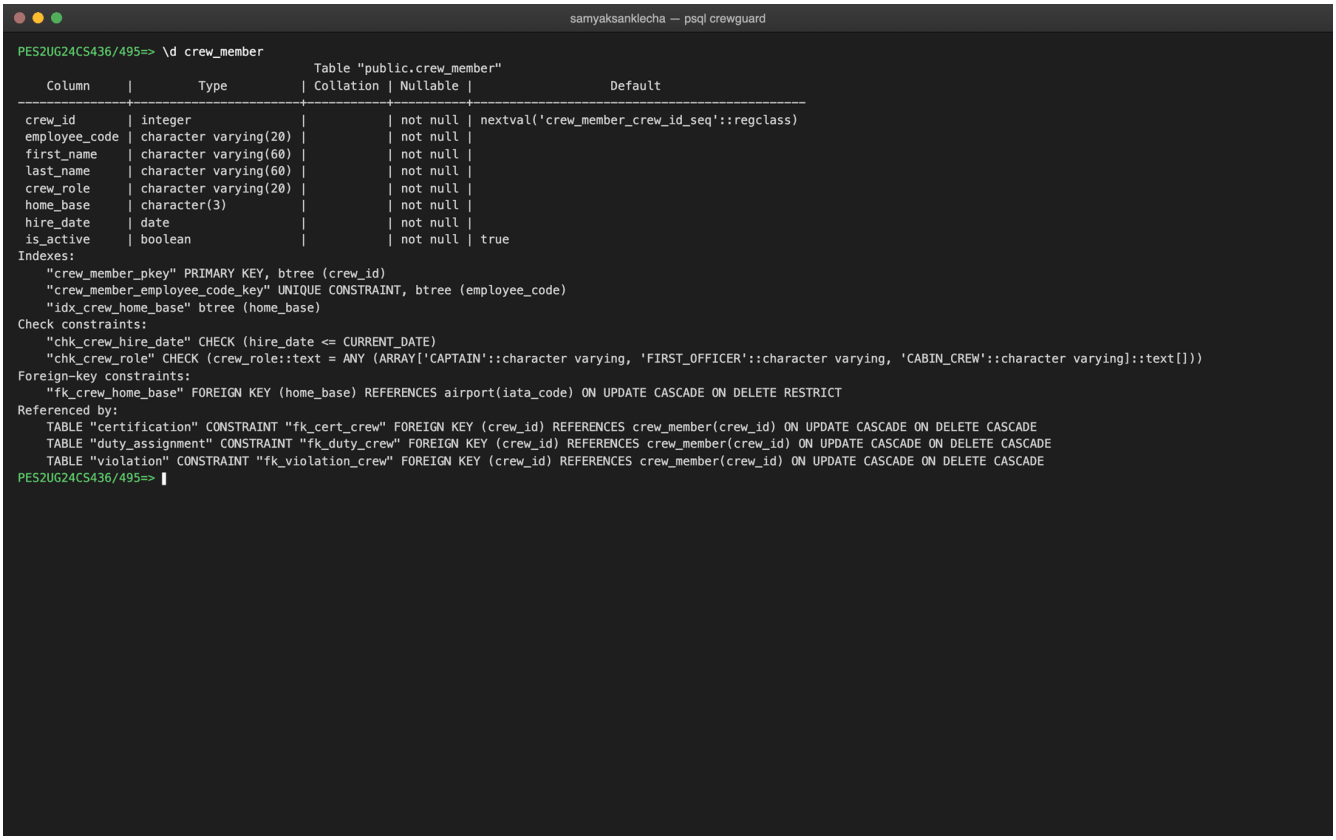


Figure 5.5: *Structure of crew_member (\d crew_member)*

Table certification

```
CREATE TABLE certification (
    cert_id          SERIAL       PRIMARY KEY,
    crew_id          INTEGER      NOT NULL,
    type_code        VARCHAR(10)  NOT NULL,
    issued_on        DATE         NOT NULL,
    expires_on       DATE         NOT NULL,
    -- deleting a crew member deletes their certificates (CASCADE)
    CONSTRAINT fk_cert_crew FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_cert_type FOREIGN KEY (type_code) REFERENCES aircraft_type(type_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    -- renewals are new rows, so (crew, type, issue date) is the natural key
    CONSTRAINT uk_cert_crew_type_issue UNIQUE (crew_id, type_code, issued_on),
    CONSTRAINT chk_cert_validity_order CHECK (expires_on > issued_on)
);
```

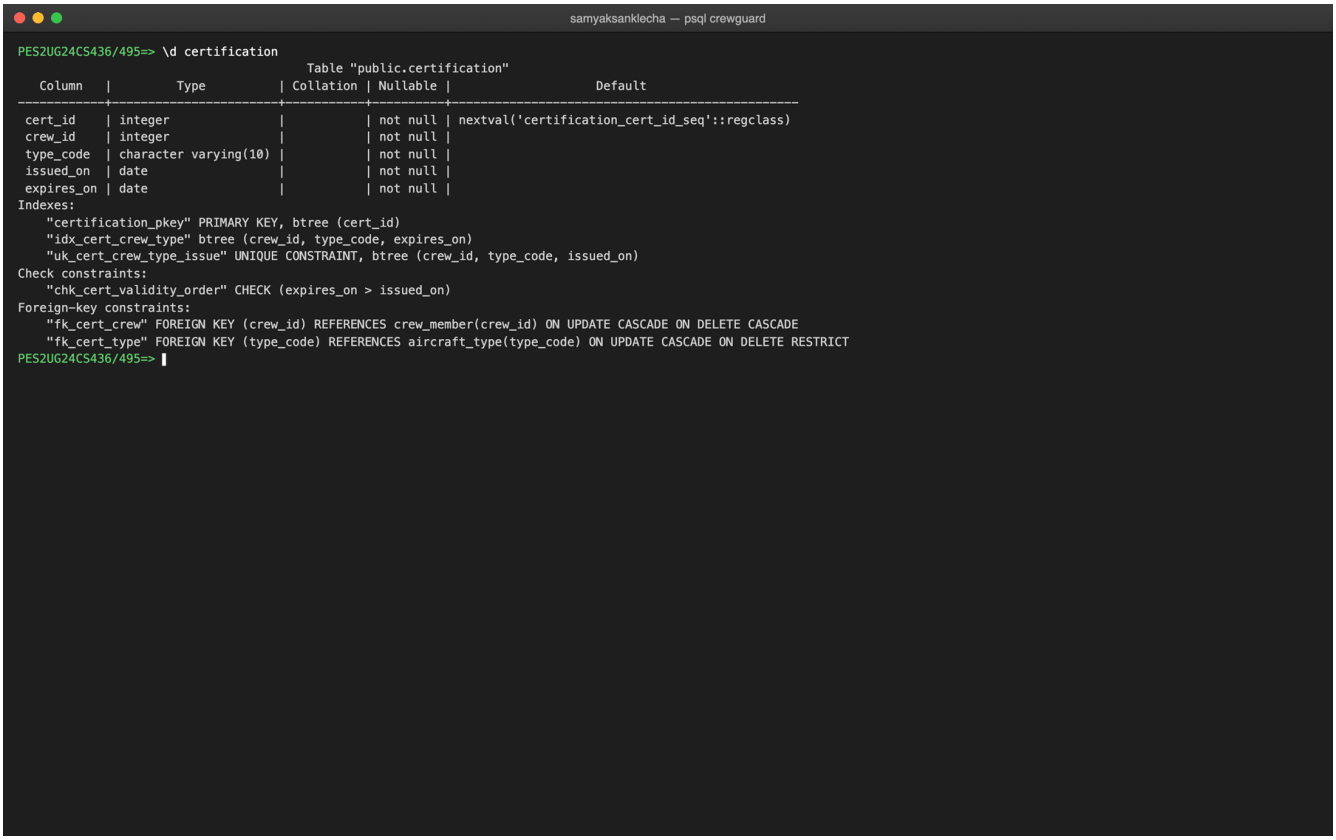


Figure 5.6: *Structure of certification (\d certification)*

Table flight

```
CREATE TABLE flight (
    flight_id       SERIAL       PRIMARY KEY,
    flight_number   VARCHAR(10)  NOT NULL,
    type_code       VARCHAR(10)  NOT NULL,
    dep_airport     CHAR(3)      NOT NULL,
    arr_airport     CHAR(3)      NOT NULL,
    sched_dep_utc   TIMESTAMPTZ  NOT NULL,
    sched_arr_utc   TIMESTAMPTZ  NOT NULL,
    CONSTRAINT fk_flight_type FOREIGN KEY (type_code) REFERENCES aircraft_type(type_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_flight_dep FOREIGN KEY (dep_airport) REFERENCES airport(iata_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_flight_arr FOREIGN KEY (arr_airport) REFERENCES airport(iata_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    -- the same flight number flies every day, so it is unique per departure
    CONSTRAINT uk_flight_number_dep UNIQUE (flight_number, sched_dep_utc),
    CONSTRAINT chk_flight_time_order CHECK (sched_arr_utc > sched_dep_utc),
    CONSTRAINT chk_flight_distinct_endpoints CHECK (dep_airport <> arr_airport)
);
```

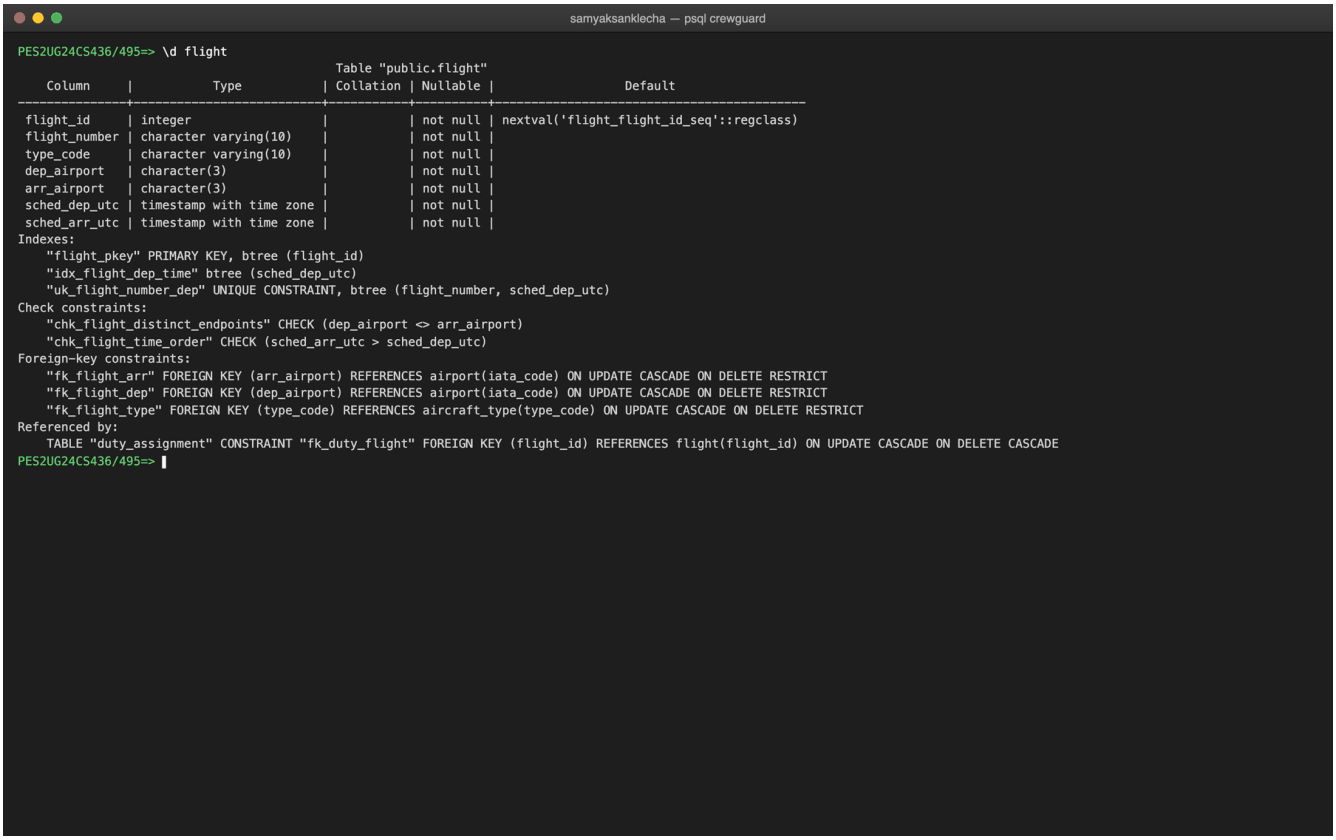


Figure 5.7: *Structure of flight (\d flight)*

Table duty_assignment

```
CREATE TABLE duty_assignment (
    assignment_id   SERIAL       PRIMARY KEY,
    crew_id         INTEGER      NOT NULL,
    flight_id       INTEGER      NOT NULL,
    assigned_role   VARCHAR(20)  NOT NULL,
    duty_start_utc  TIMESTAMPTZ  NOT NULL,
    duty_end_utc    TIMESTAMPTZ  NOT NULL,
    created_by      INTEGER,
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_duty_crew FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_duty_flight FOREIGN KEY (flight_id) REFERENCES flight(flight_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    -- if the scheduler's account is removed, keep the duty but forget the author
    CONSTRAINT fk_duty_created_by FOREIGN KEY (created_by) REFERENCES app_user(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT uk_duty_crew_flight UNIQUE (crew_id, flight_id),
    CONSTRAINT chk_duty_time_order CHECK (duty_end_utc > duty_start_utc),
    CONSTRAINT chk_duty_assigned_role CHECK (assigned_role IN ('CAPTAIN', 'FIRST_OFFICER',
'CABIN_CREW'))
);
```

ALTER TABLE adds the exclusion constraint. A UNIQUE constraint can only reject equal values; EXCLUDE USING GIST rejects any two rows with the same crew_id whose time ranges overlap (&&), which is what "no overlapping duties" means. The btree_gist extension lets one GiST index compare the integer and the range together.

```
ALTER TABLE duty_assignment
    ADD CONSTRAINT excl_duty_no_overlap
    EXCLUDE USING GIST (
        crew_id WITH =,
        tstzrange(duty_start_utc, duty_end_utc) WITH &&
    );
```

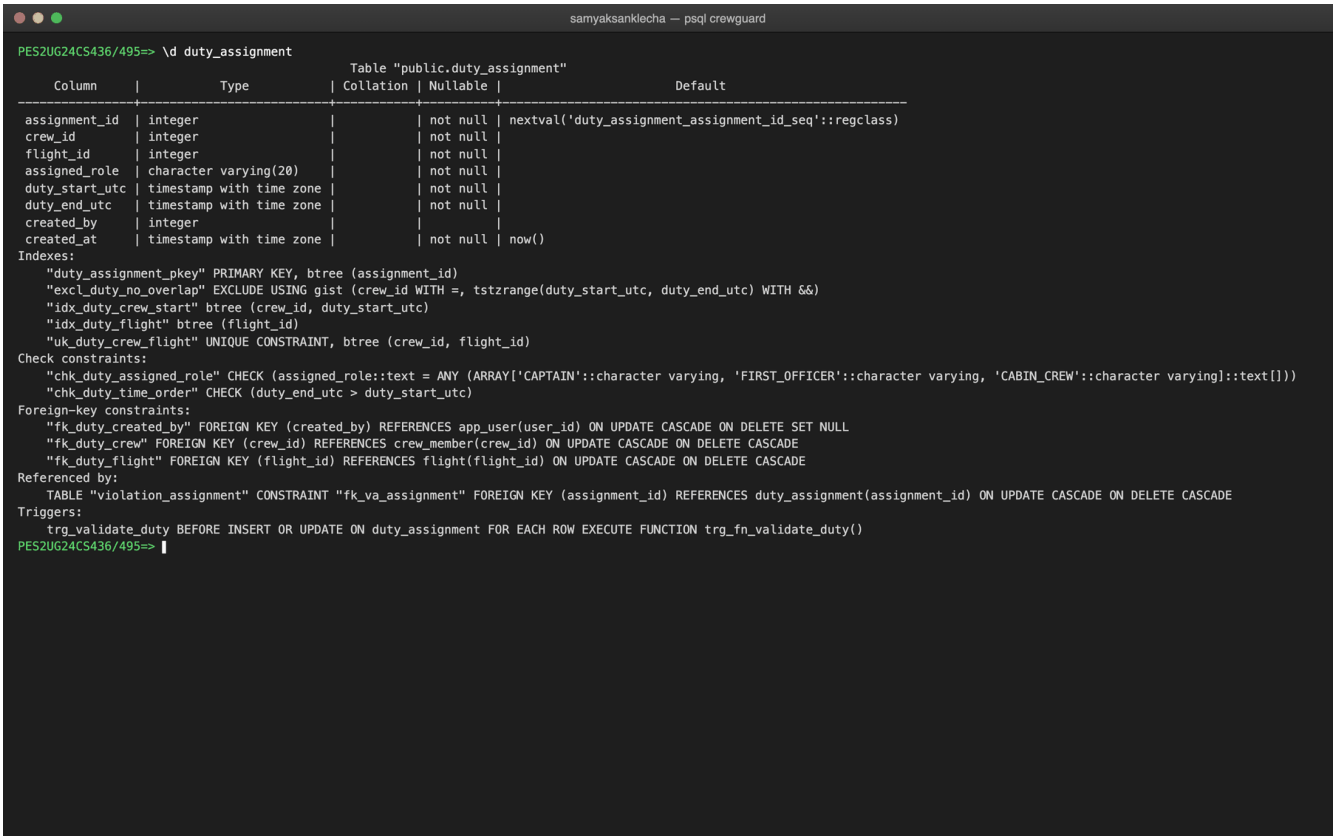


Figure 5.8: *Structure of duty_assignment (\d duty_assignment)*

Table rule_definition

```
CREATE TABLE rule_definition (
    rule_code       CHAR(3)      PRIMARY KEY,
    rule_name       VARCHAR(120) NOT NULL,
    description     TEXT         NOT NULL,
    regulation_ref  VARCHAR(120),
    threshold_value NUMERIC(8,2),
    threshold_unit  VARCHAR(20),
    window_days     INTEGER,
    severity        VARCHAR(10)  NOT NULL,
    is_active       BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT chk_rule_severity CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    CONSTRAINT chk_rule_window_days CHECK (window_days IS NULL OR window_days > 0)
);
```

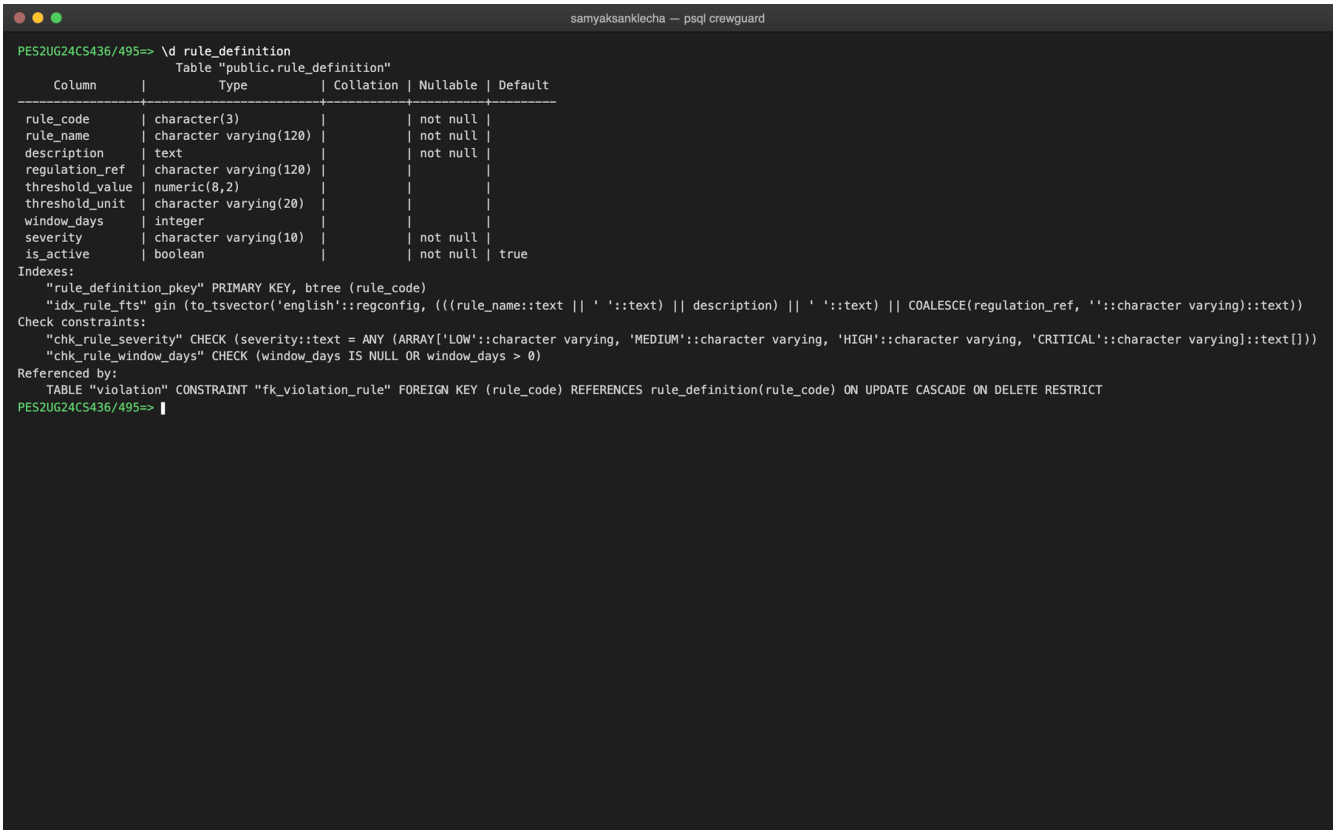


Figure 5.9: *Structure of rule_definition (\d rule_definition)*

Table validation_run

```
CREATE TABLE validation_run (
    run_id           SERIAL      PRIMARY KEY,
    triggered_by     INTEGER,
    period_start     DATE        NOT NULL,
    period_end       DATE        NOT NULL,
    started_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    finished_at      TIMESTAMPTZ,
    status           VARCHAR(15) NOT NULL DEFAULT 'RUNNING',
    violations_found INTEGER     NOT NULL DEFAULT 0,
    CONSTRAINT fk_run_user FOREIGN KEY (triggered_by) REFERENCES app_user(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_run_status CHECK (status IN ('RUNNING', 'COMPLETED', 'FAILED', 'STALE')),
    CONSTRAINT chk_run_period_order CHECK (period_end >= period_start),
    CONSTRAINT chk_run_violation_count CHECK (violations_found >= 0)
);
```

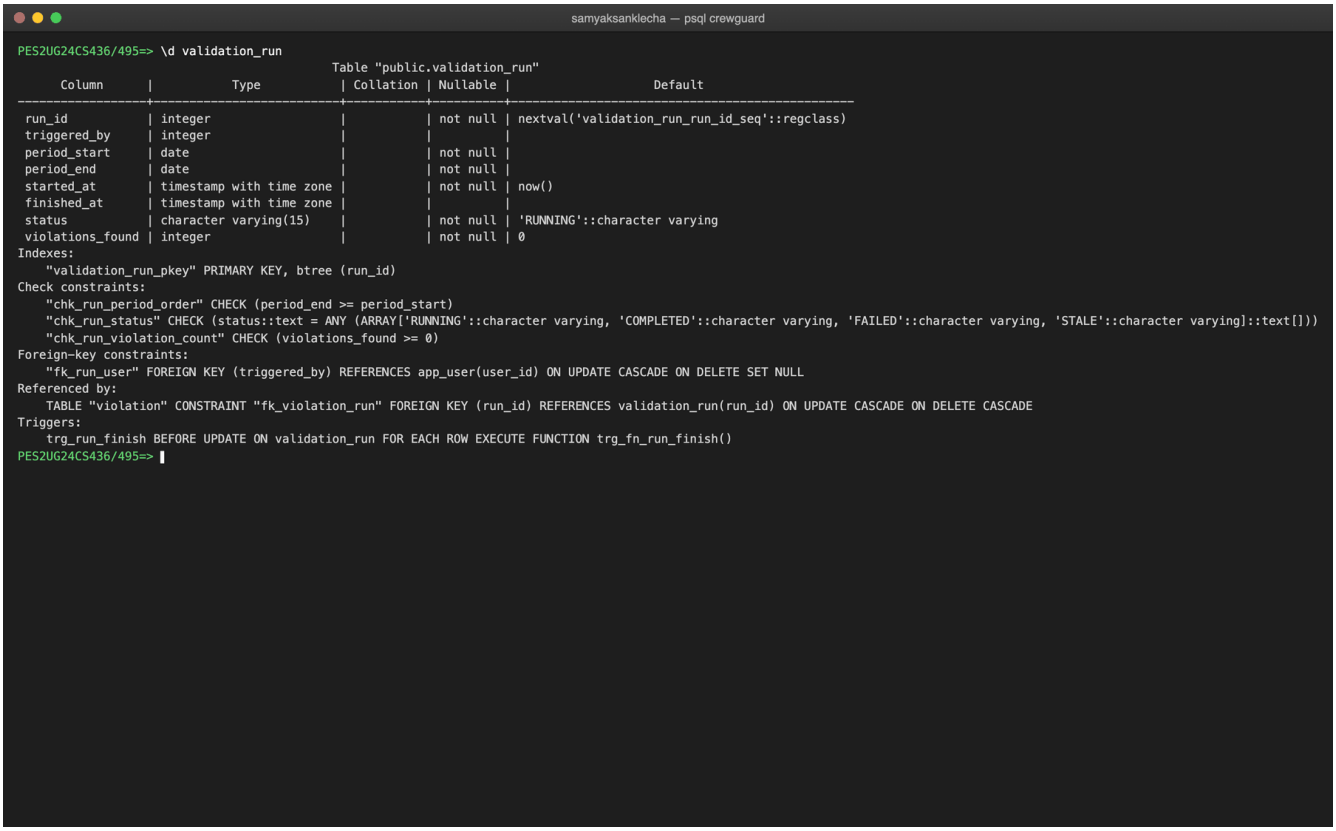


Figure 5.10: *Structure of validation_run (\d validation_run)*

Table violation

```
CREATE TABLE violation (
    violation_id    SERIAL       PRIMARY KEY,
    run_id          INTEGER      NOT NULL,
    rule_code       CHAR(3)      NOT NULL,
    crew_id         INTEGER      NOT NULL,
    severity        VARCHAR(10)  NOT NULL,
    window_start    TIMESTAMPTZ,
    window_end      TIMESTAMPTZ,
    observed_value  NUMERIC(8,2),
    threshold_value NUMERIC(8,2),
    evidence_doc_id VARCHAR(24),             -- MongoDB ObjectId of the evidence document
    CONSTRAINT fk_violation_run FOREIGN KEY (run_id) REFERENCES validation_run(run_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_violation_rule FOREIGN KEY (rule_code) REFERENCES rule_definition(rule_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_violation_crew FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT chk_violation_severity CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    CONSTRAINT chk_violation_window_order
        CHECK (window_end IS NULL OR window_start IS NULL OR window_end >= window_start)
);
```

The review workflow was added after Review 1 with ALTER TABLE in db/03_compliance.sql: four columns, a foreign key to the reviewer, and two CHECK constraints (a valid status, and a note required to resolve).

```
ALTER TABLE violation
    ADD COLUMN review_status VARCHAR(15) NOT NULL DEFAULT 'OPEN',
```

```
    ADD COLUMN reviewed_by   INTEGER,
    ADD COLUMN review_note   TEXT,
    ADD COLUMN reviewed_at   TIMESTAMPTZ,
    ADD CONSTRAINT chk_violation_review_status
        CHECK (review_status IN ('OPEN', 'ACKNOWLEDGED', 'RESOLVED')),
    -- if the auditor's account is removed, keep the review but forget who did it
    ADD CONSTRAINT fk_violation_reviewer FOREIGN KEY (reviewed_by) REFERENCES app_user(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    -- a violation cannot be closed without saying how it was resolved
    ADD CONSTRAINT chk_violation_resolution_note
        CHECK (review_status <> 'RESOLVED' OR review_note IS NOT NULL);
```

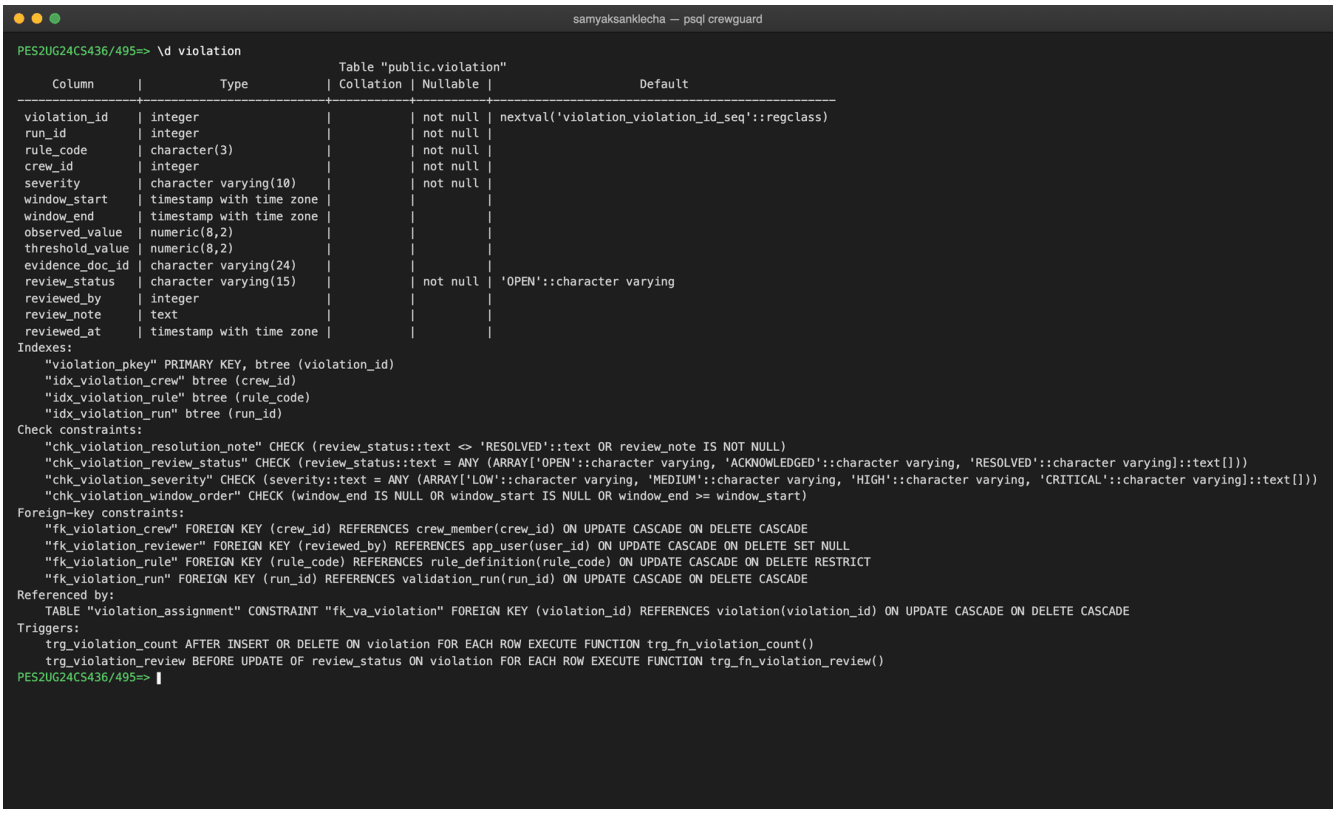


Figure 5.11: *Structure of violation (\d violation)*

Table violation_assignment

```
CREATE TABLE violation_assignment (
    violation_id       INTEGER   NOT NULL,
    assignment_id      INTEGER   NOT NULL,
    contribution_value NUMERIC(8,2),         -- hours this duty contributed
    CONSTRAINT pk_violation_assignment PRIMARY KEY (violation_id, assignment_id),
    CONSTRAINT fk_va_violation FOREIGN KEY (violation_id) REFERENCES violation(violation_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_va_assignment FOREIGN KEY (assignment_id) REFERENCES
duty_assignment(assignment_id)
        ON UPDATE CASCADE ON DELETE CASCADE
);
```

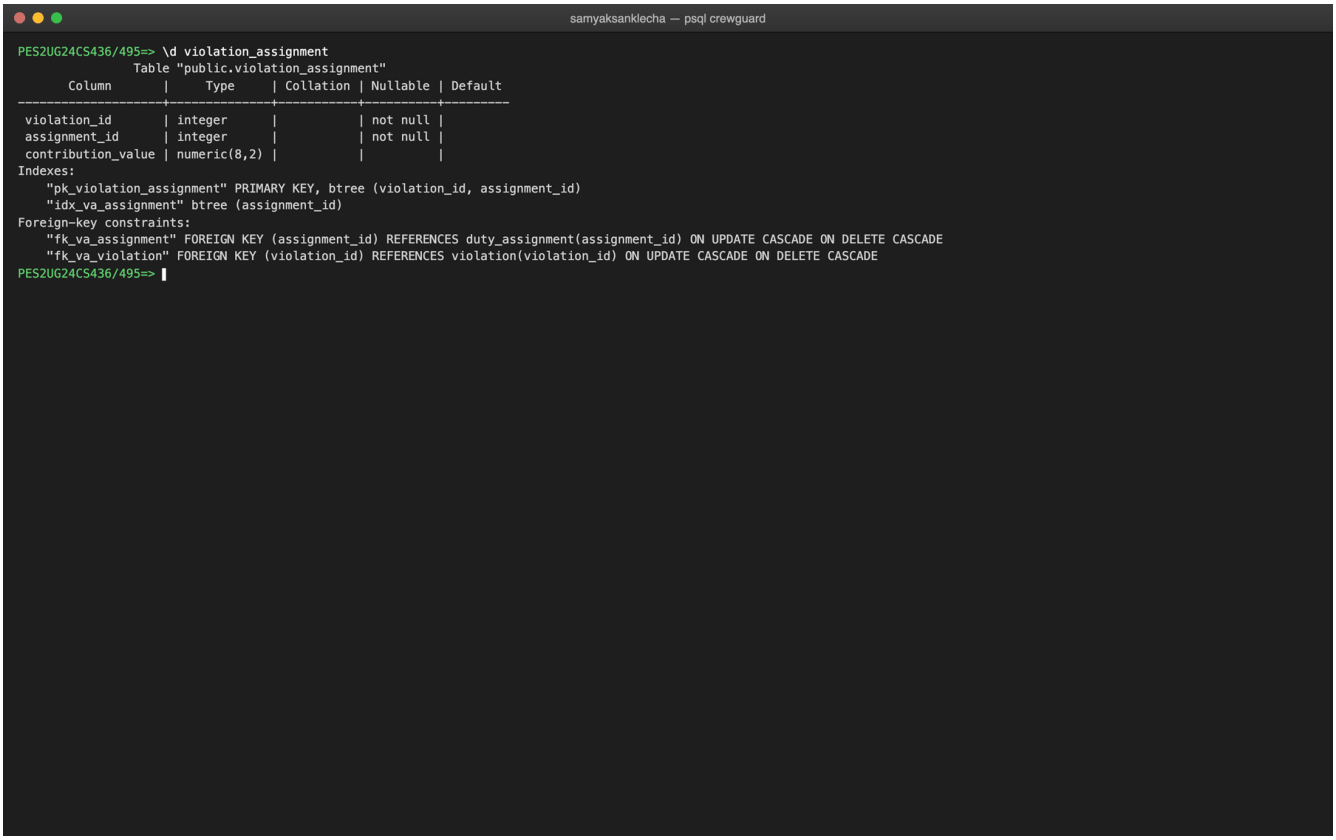


Figure 5.12: *Structure of violation_assignment (\d violation_assignment)*

Indexes

PostgreSQL indexes primary keys and UNIQUE columns automatically, but not foreign keys. The join and filter columns used by the application are therefore indexed explicitly. The composite index on (crew_id, duty_start_utc) serves "all duties of one crew member in time order", which the roster pages and the RST and WDH checks use. A GIN index supports full-text search on the rule catalogue.

```
CREATE INDEX idx_duty_crew_start   ON duty_assignment (crew_id, duty_start_utc);
CREATE INDEX idx_duty_flight       ON duty_assignment (flight_id);
CREATE INDEX idx_flight_dep_time   ON flight (sched_dep_utc);
CREATE INDEX idx_cert_crew_type    ON certification (crew_id, type_code, expires_on);
CREATE INDEX idx_crew_home_base    ON crew_member (home_base);
CREATE INDEX idx_violation_run     ON violation (run_id);
CREATE INDEX idx_violation_crew    ON violation (crew_id);
CREATE INDEX idx_va_assignment     ON violation_assignment (assignment_id);
CREATE INDEX idx_violation_rule ON violation (rule_code);
CREATE INDEX idx_rule_fts ON rule_definition
    USING GIN (to_tsvector('english', rule_name || ' ' || description || ' ' ||
COALESCE(regulation_ref, '')));
```

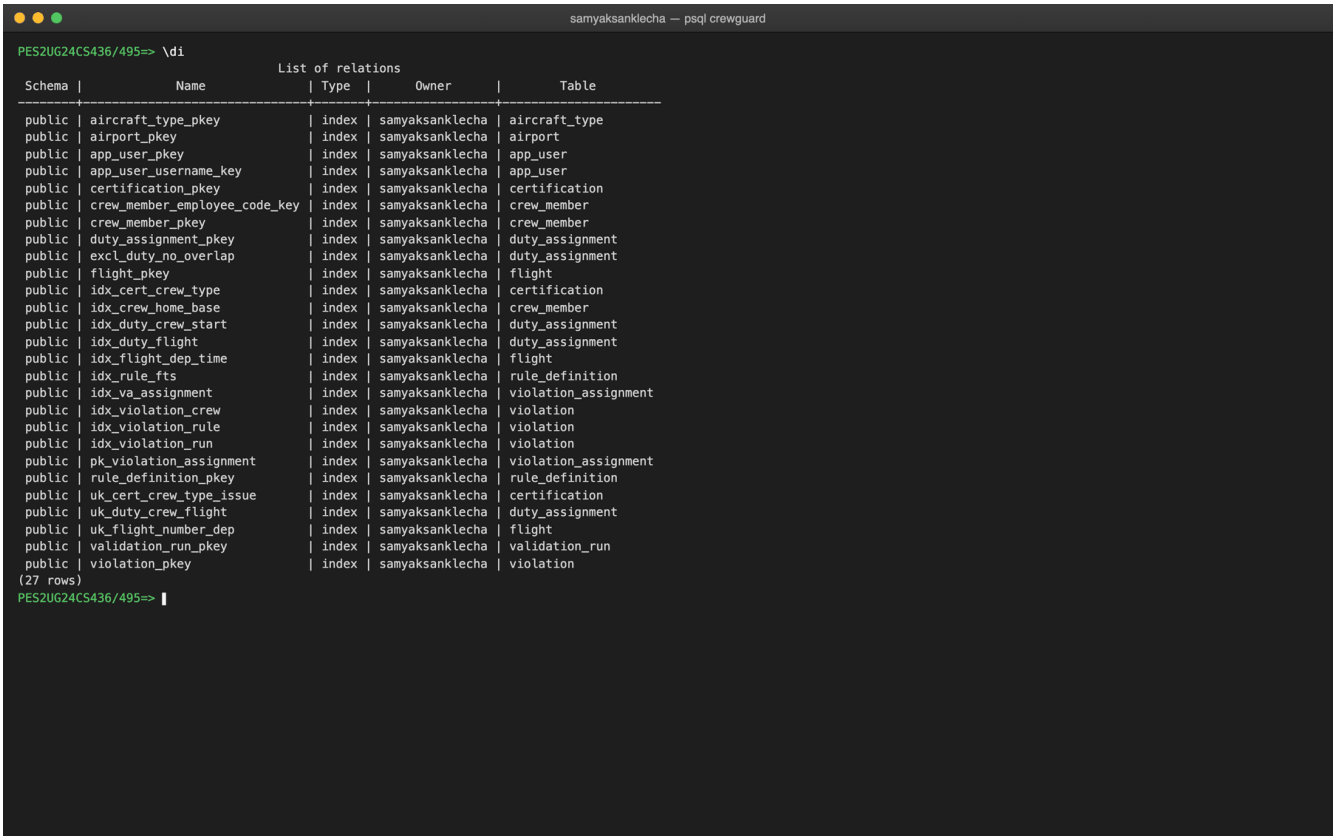


Figure 5.13: *All indexes (\di)*

6. DML STATEMENTS

The sample data is loaded by db/04_seed.sql. Most tables use multi-row INSERT ... VALUES. Duties use INSERT ... SELECT from a VALUES list joined to flight and crew_member, so each duty's times are computed from its flight (report one hour before departure, release 30 minutes after arrival, plus any delay) instead of being typed by hand. Explicit ids are used for readability, and the sequences are re-synchronised with setval at the end. Each statement is followed by a screenshot of the table with its inserted values.

Table airport

```
INSERT INTO airport (iata_code, name, city, country, timezone) VALUES
('BLR', 'Kempegowda International Airport',      'Bengaluru', 'India',               'Asia/
Kolkata'),
('DEL', 'Indira Gandhi International Airport',   'Delhi',     'India',               'Asia/
Kolkata'),
('BOM', 'Chhatrapati Shivaji Maharaj Intl',      'Mumbai',    'India',               'Asia/
Kolkata'),
('MAA', 'Chennai International Airport',         'Chennai',   'India',               'Asia/
Kolkata'),
('HYD', 'Rajiv Gandhi International Airport',    'Hyderabad', 'India',               'Asia/
Kolkata'),
('CCU', 'Netaji Subhas Chandra Bose Intl',       'Kolkata',   'India',               'Asia/
Kolkata'),
('DXB', 'Dubai International Airport',           'Dubai',     'United Arab Emirates', 'Asia/
Dubai'),
('LHR', 'London Heathrow Airport',               'London',    'United Kingdom',      'Europe/
London');
```

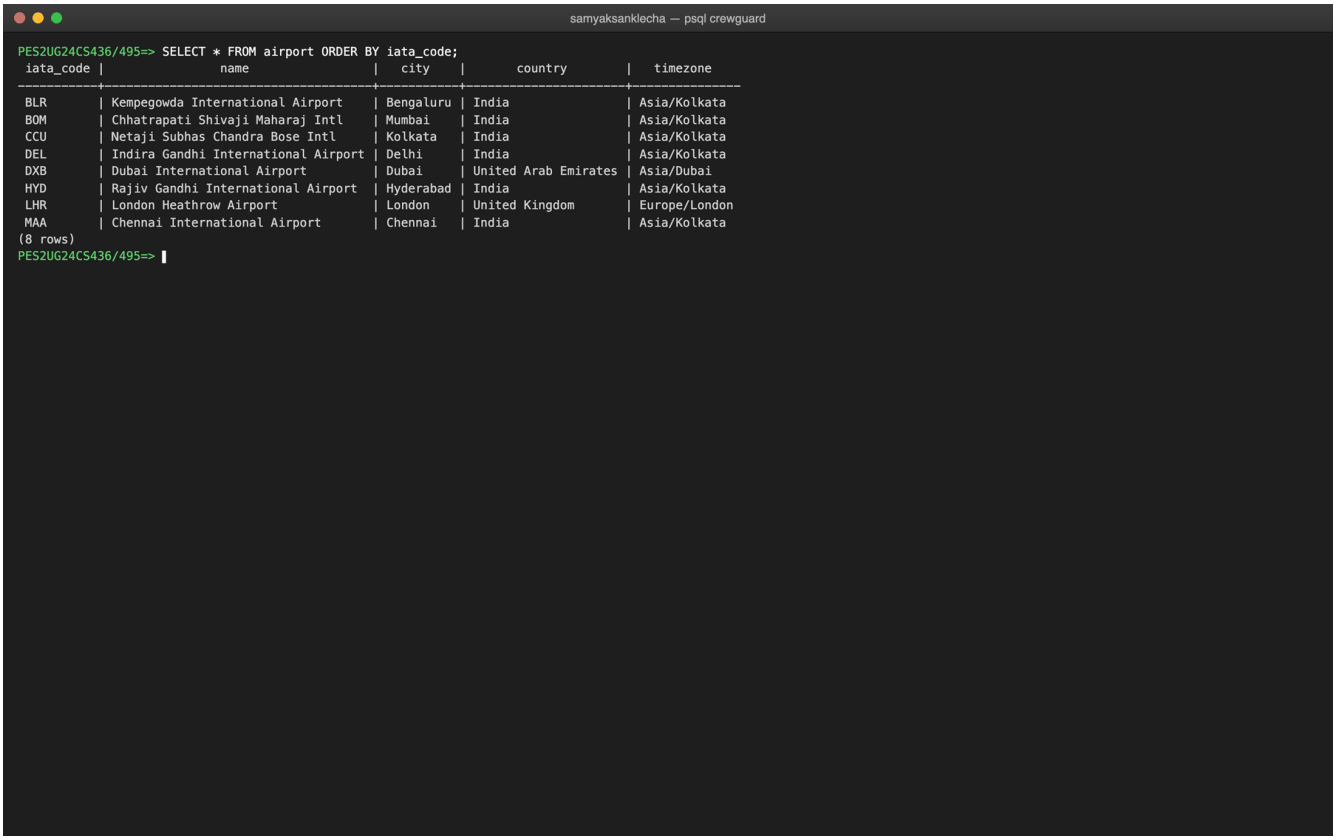


Figure 6.1: *Rows in airport*

Table aircraft_type

```
INSERT INTO aircraft_type (type_code, manufacturer, description) VALUES
('A320', 'Airbus', 'Airbus A320neo narrow-body'),
('A321', 'Airbus', 'Airbus A321neo narrow-body'),
('B738', 'Boeing', 'Boeing 737-800 narrow-body'),
('B788', 'Boeing', 'Boeing 787-8 Dreamliner wide-body');
```

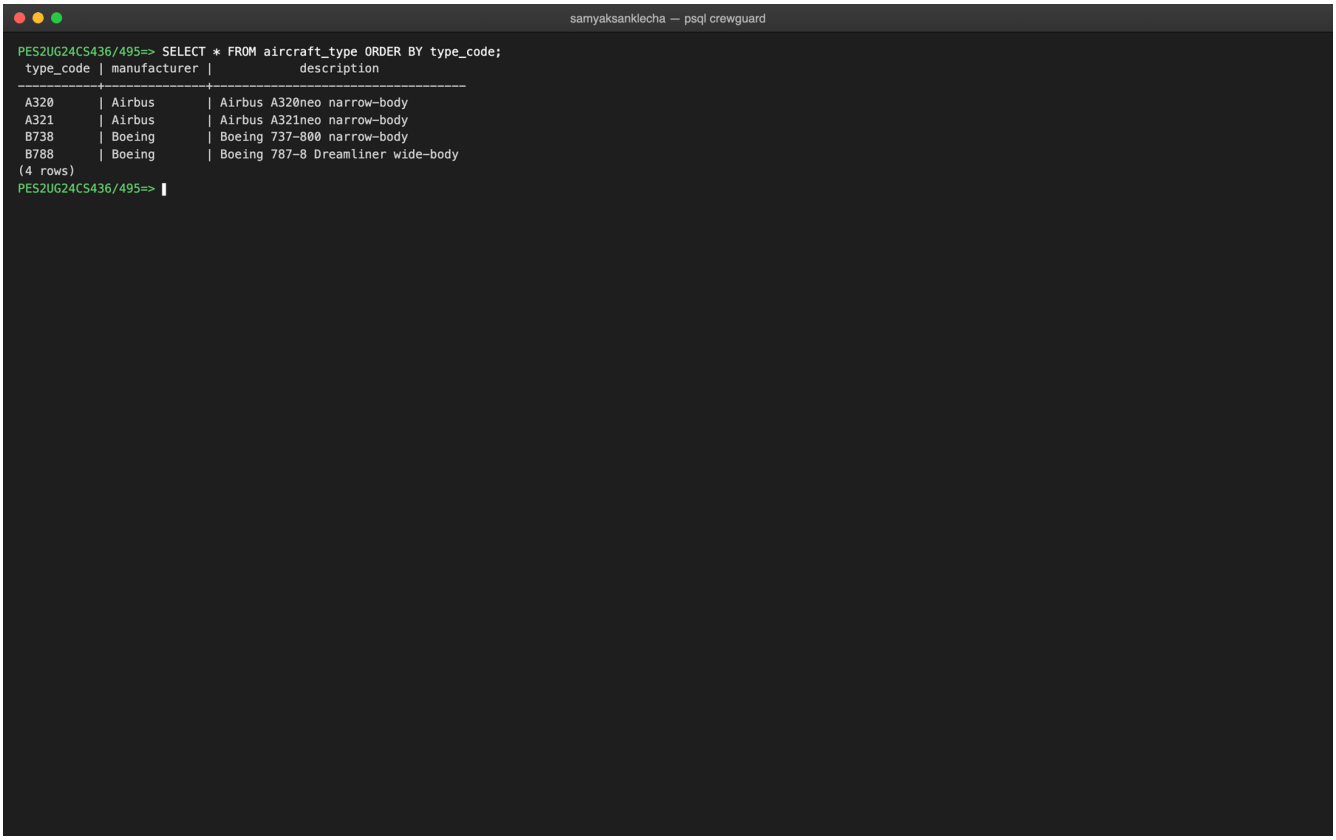


Figure 6.2: *Rows in aircraft_type*

Table app_user

```
INSERT INTO app_user (user_id, username, password_hash, app_role) VALUES
(1, 'sched_sam',     '$2b$10$cwu/pvgdqv2C95XdPtGVUuxjRJHuLDi5K0gYnnFfQnQki6lkuBAlC',
'SCHEDULER'),
(2, 'audit_shriram', '$2b$10$5bokBBUTwKn8PqLJFAEPN.vewwqJNS.MgS2S3b1pQ4p0QtPX2mZzm', 'AUDITOR');
```

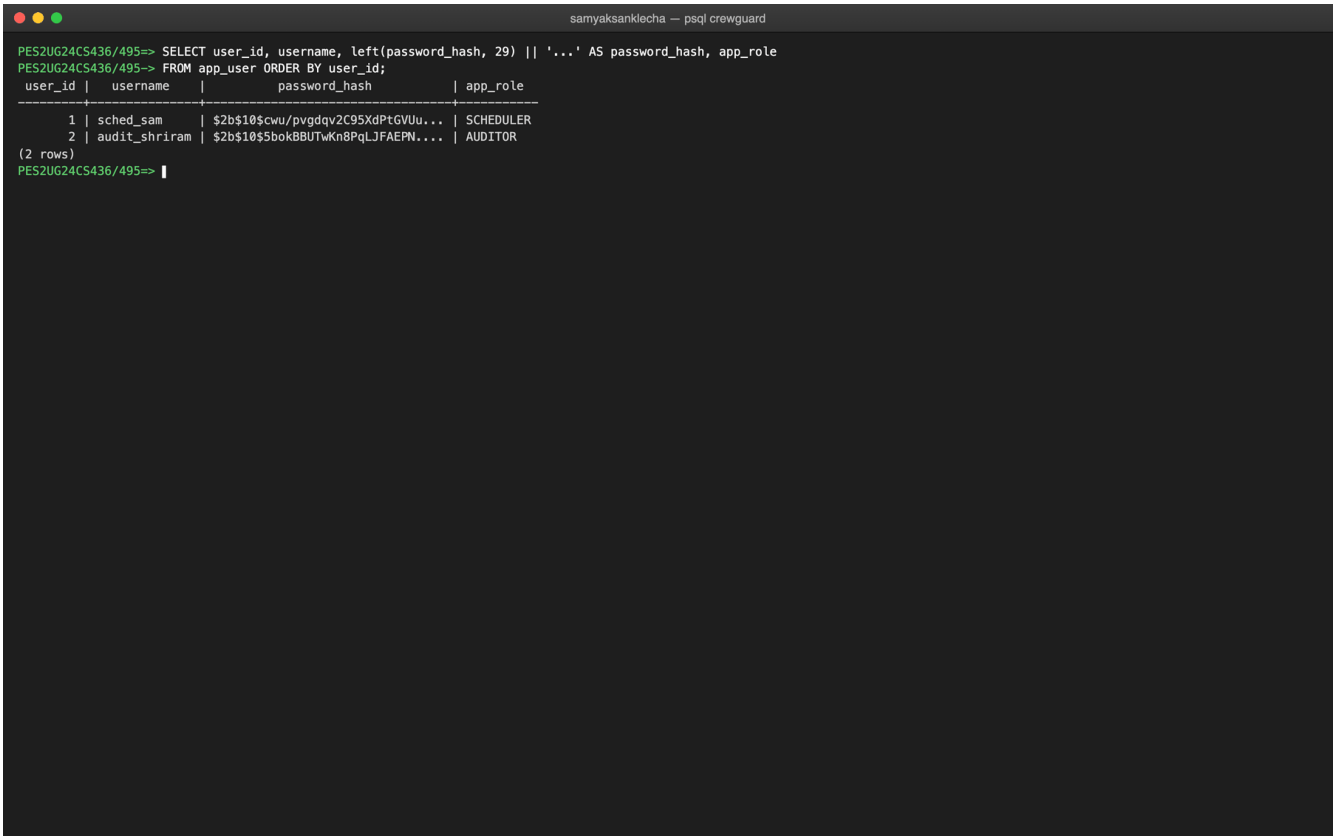


Figure 6.3: *Rows in app_user (password hashes shortened)*

Table crew_member

```
INSERT INTO crew_member (crew_id, employee_code, first_name, last_name, crew_role, home_base,
hire_date, is_active) VALUES
(1,  'EMP1001', 'Arjun',  'Menon',    'CAPTAIN',       'BLR', '2015-06-01', TRUE),
(2,  'EMP1002', 'Kavya',  'Sharma',   'CAPTAIN',       'BLR', '2017-03-15', TRUE),
(3,  'EMP1003', 'Rohan',  'Kumar',    'FIRST_OFFICER', 'BLR', '2019-07-10', TRUE),
(4,  'EMP1004', 'Sneha',  'Patil',    'FIRST_OFFICER', 'DEL', '2020-01-20', TRUE),
(5,  'EMP1005', 'Vikram', 'Joshi',    'CABIN_CREW',    'BLR', '2021-09-05', TRUE),
(6,  'EMP1006', 'Nisha',  'Shah',     'CABIN_CREW',    'DEL', '2022-02-11', TRUE),
    ...  -- remaining rows omitted, see db/04_seed.sql
```

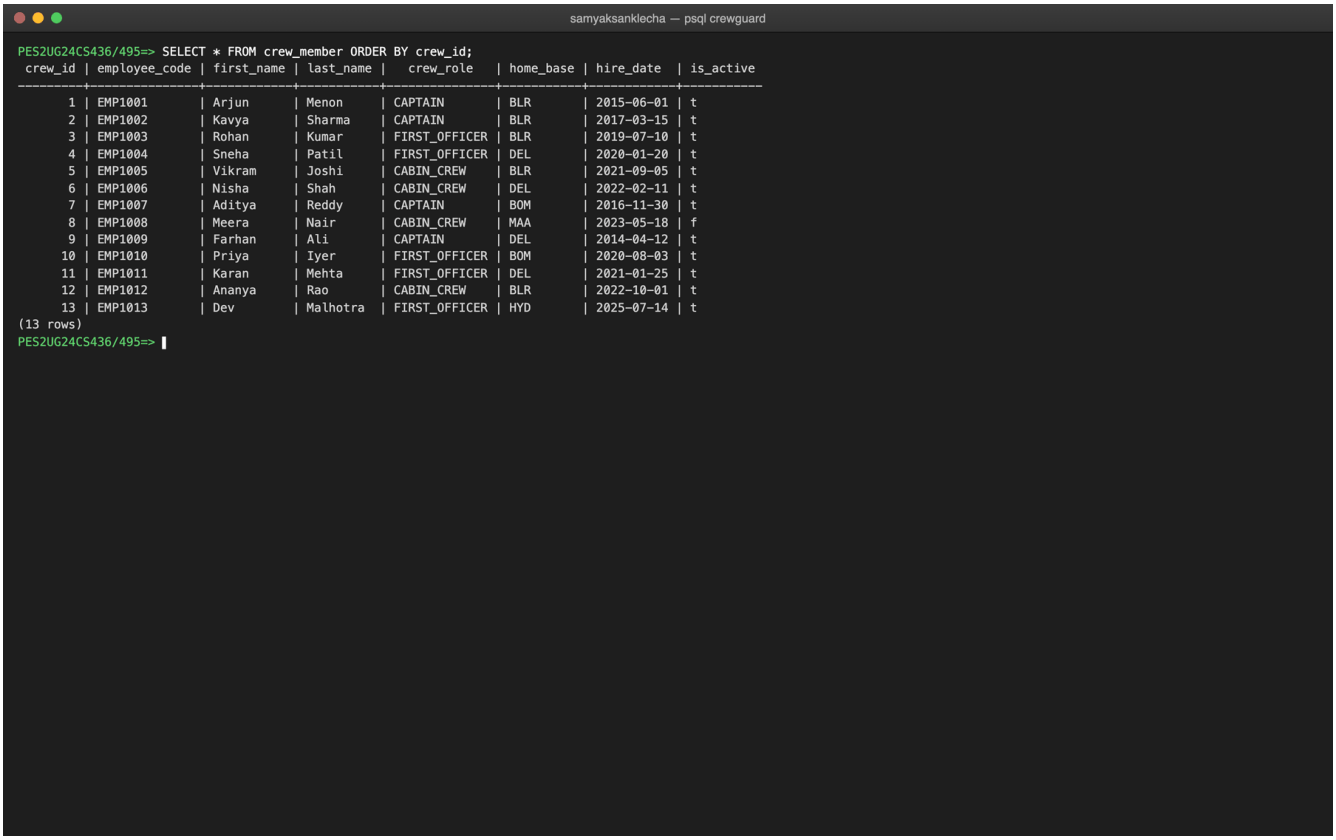


Figure 6.4: *Rows in crew_member*

Table certification

```
INSERT INTO certification (cert_id, crew_id, type_code, issued_on, expires_on) VALUES
(1,  1,  'A320', '2024-01-10', '2027-01-09'),
(2,  1,  'A321', '2024-02-15', '2026-12-20'),   -- expiring soon
(3,  2,  'A320', '2023-11-01', '2026-10-31'),   -- expired, but ...
(4,  2,  'A320', '2026-10-20', '2029-10-19'),   -- ... renewed before it lapsed
(5,  2,  'B738', '2025-01-05', '2028-01-04'),
(6,  3,  'A320', '2024-05-20', '2027-05-19'),
    ...  -- remaining rows omitted, see db/04_seed.sql
```

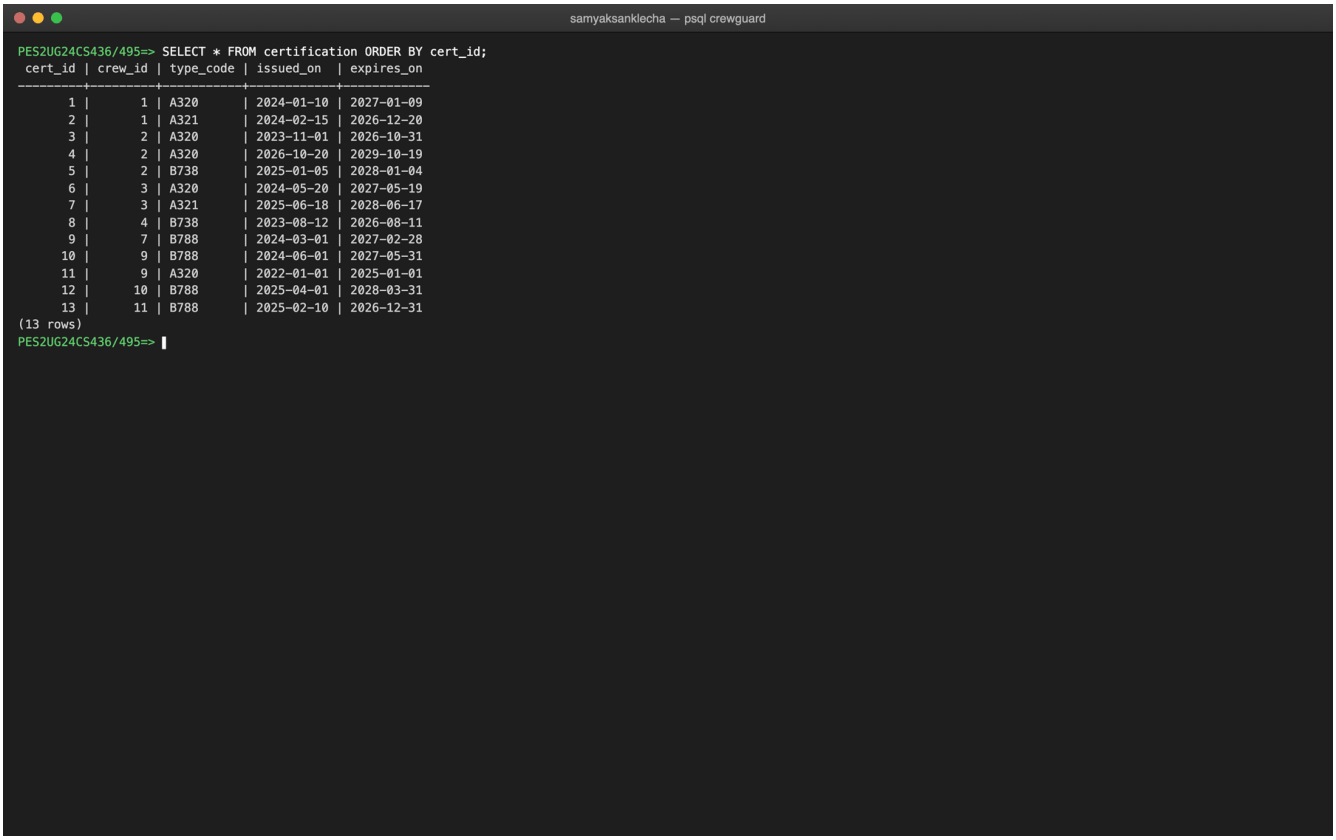


Figure 6.5: *Rows in certification*

Table flight

```
INSERT INTO flight (flight_id, flight_number, type_code, dep_airport, arr_airport,
sched_dep_utc, sched_arr_utc) VALUES
-- week 1, domestic and regional
(1,  'CG101', 'A320', 'BLR', 'DEL', '2026-11-02 01:00+00', '2026-11-02 03:45+00'),
(2,  'CG102', 'A320', 'DEL', 'BLR', '2026-11-02 05:30+00', '2026-11-02 08:15+00'),
(3,  'CG201', 'B738', 'BLR', 'BOM', '2026-11-03 02:00+00', '2026-11-03 03:45+00'),
(4,  'CG202', 'B738', 'BOM', 'BLR', '2026-11-03 16:00+00', '2026-11-03 17:45+00'),
(5,  'CG301', 'A321', 'BLR', 'MAA', '2026-11-04 01:30+00', '2026-11-04 02:40+00'),
    ...  -- remaining rows omitted, see db/04_seed.sql
```

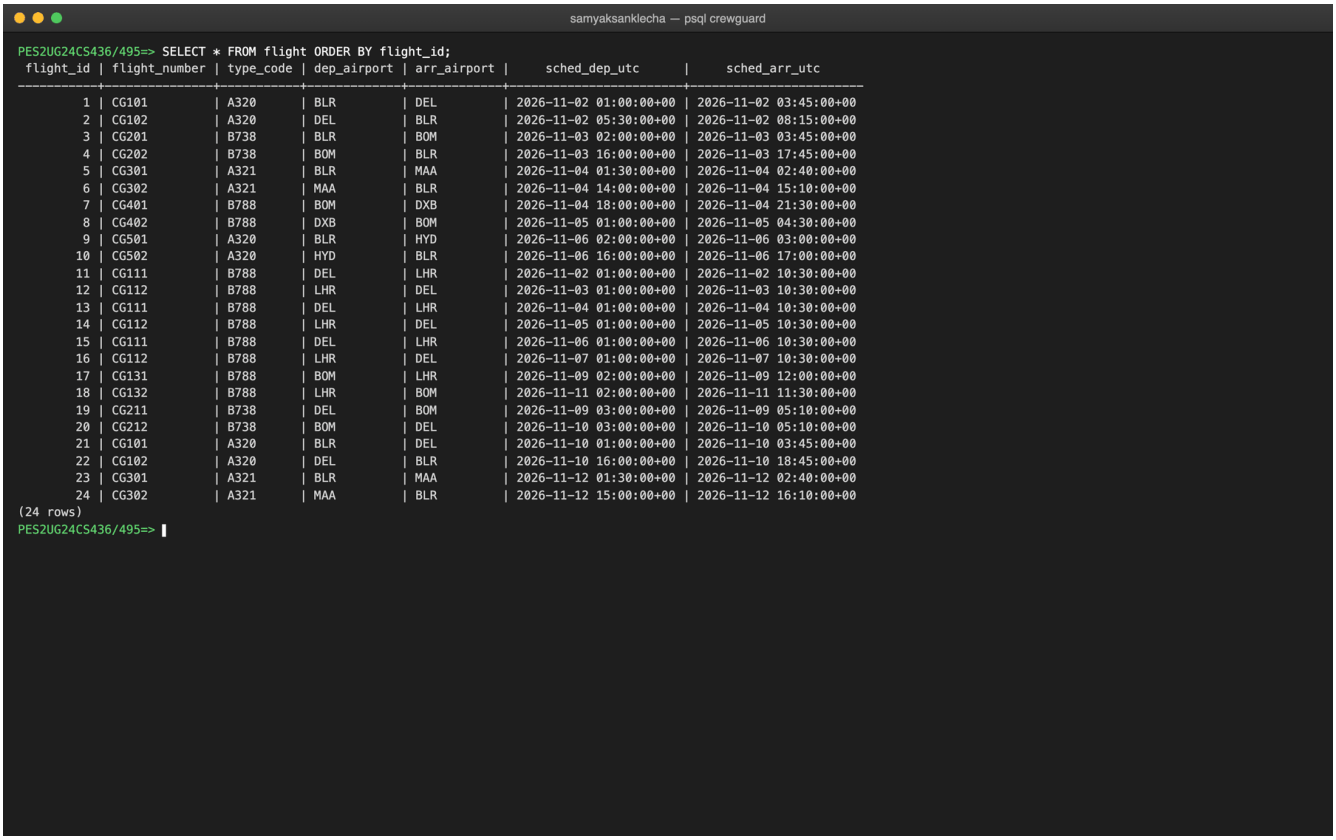


Figure 6.6: *Rows in flight*

Table duty_assignment

```
INSERT INTO duty_assignment (crew_id, flight_id, assigned_role, duty_start_utc, duty_end_utc,
created_by)
SELECT r.crew_id,
       f.flight_id,
       cm.crew_role,
       f.sched_dep_utc - INTERVAL '1 hour',
       f.sched_arr_utc + INTERVAL '30 minutes' + r.delay,
       1                                                              -- rostered by sched_sam
FROM (VALUES
    -- (flight_id, crew_id, extra delay)
    (1, 1, INTERVAL '0'), (1, 3, INTERVAL '0'), (1, 5, INTERVAL '0'),
    (2, 1, INTERVAL '0'), (2, 3, INTERVAL '0'), (2, 5, INTERVAL '0'),
    (3, 2, INTERVAL '0'), (3, 3, INTERVAL '0'), (3, 12, INTERVAL '0'),
    (4, 2, INTERVAL '0'), (4, 3, INTERVAL '0'), (4, 12, INTERVAL '0'),
    (5, 1, INTERVAL '0'), (5, 5, INTERVAL '0'),
    ...  -- remaining rows omitted, see db/04_seed.sql

SELECT setval('app_user_user_id_seq',       (SELECT MAX(user_id)   FROM app_user));
SELECT setval('crew_member_crew_id_seq',    (SELECT MAX(crew_id)   FROM crew_member));
SELECT setval('certification_cert_id_seq',  (SELECT MAX(cert_id)   FROM certification));
SELECT setval('flight_flight_id_seq',       (SELECT MAX(flight_id) FROM flight));
```

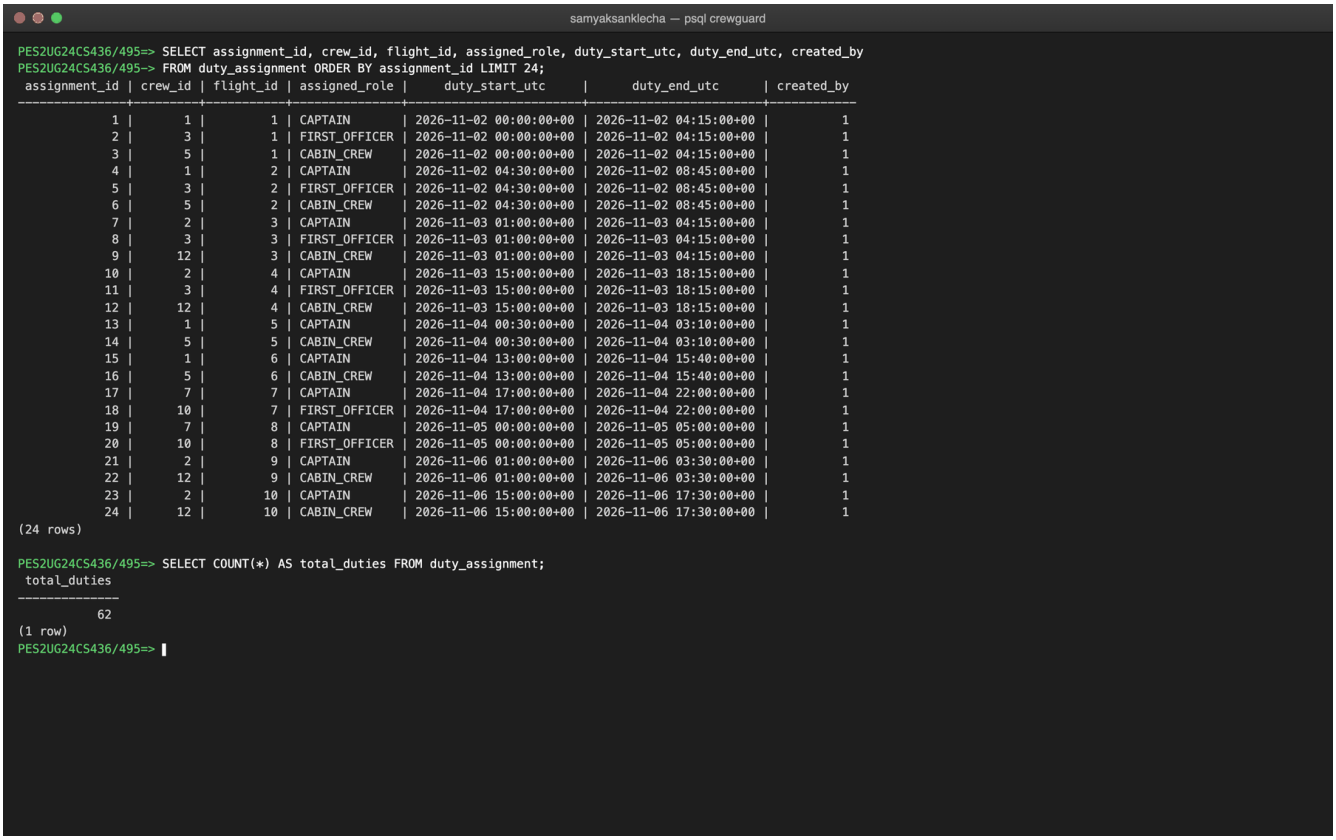


Figure 6.7: *Rows in duty_assignment (first 24 of 62)*

Table rule_definition

```
INSERT INTO rule_definition (rule_code, rule_name, description, regulation_ref, threshold_value,
threshold_unit, window_days, severity, is_active) VALUES
('RST', 'Minimum Rest Between Duties',
        'A crew member must receive a minimum continuous rest period between the end of one duty
and the start of the next duty, so that fatigue from one duty does not carry into the next.',
        'DGCA CAR Section 7 Series J Part III', 10.00, 'HOURS', NULL, 'HIGH', TRUE),
    ...  -- remaining rows omitted, see db/04_seed.sql
```

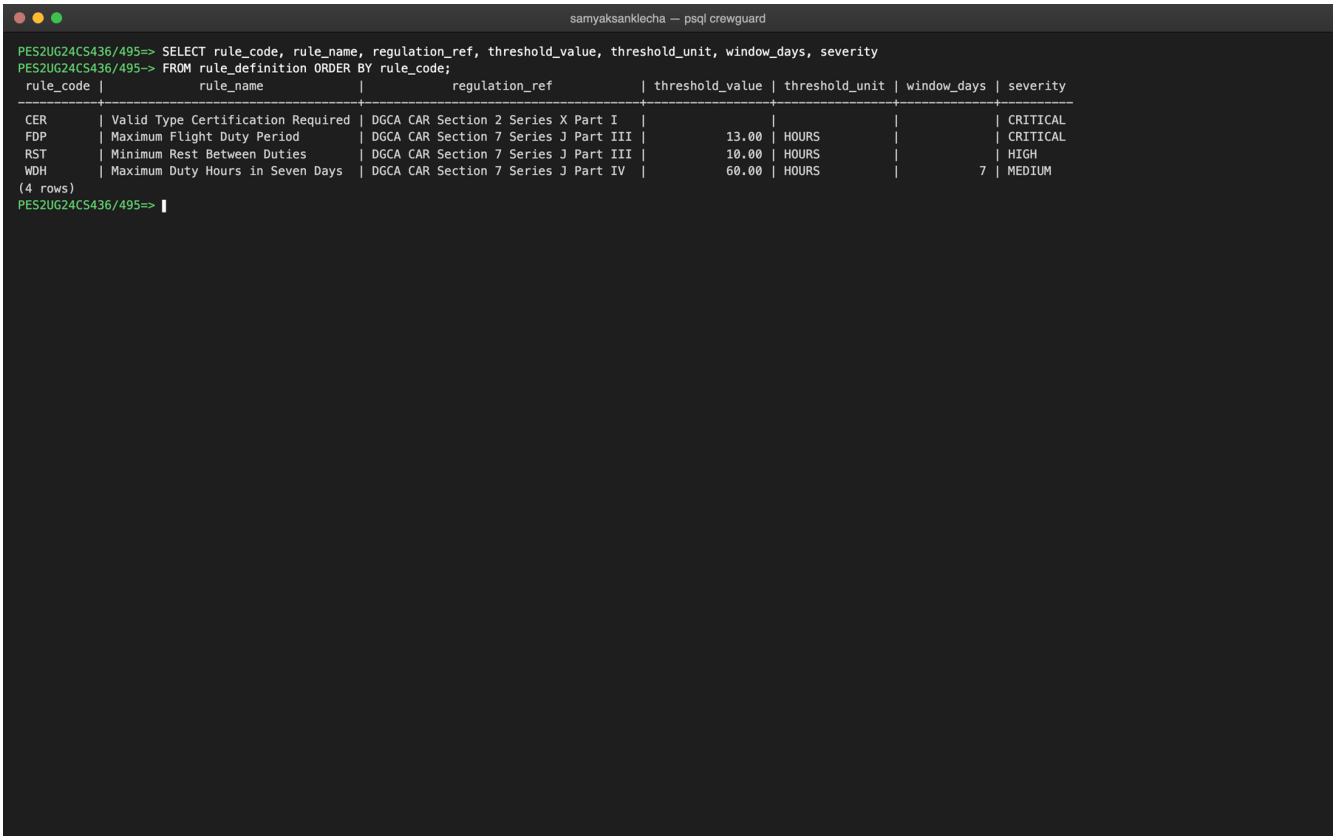


Figure 6.8: *Rows in rule_definition*

Compliance tables

validation_run, violation and violation_assignment are filled by the validator, not by the seed script. The screenshots below show them after the auditor ran a validation from 2 to 15 November in the application and reviewed two violations. evidence_doc_id holds the ObjectId of each violation's MongoDB document.

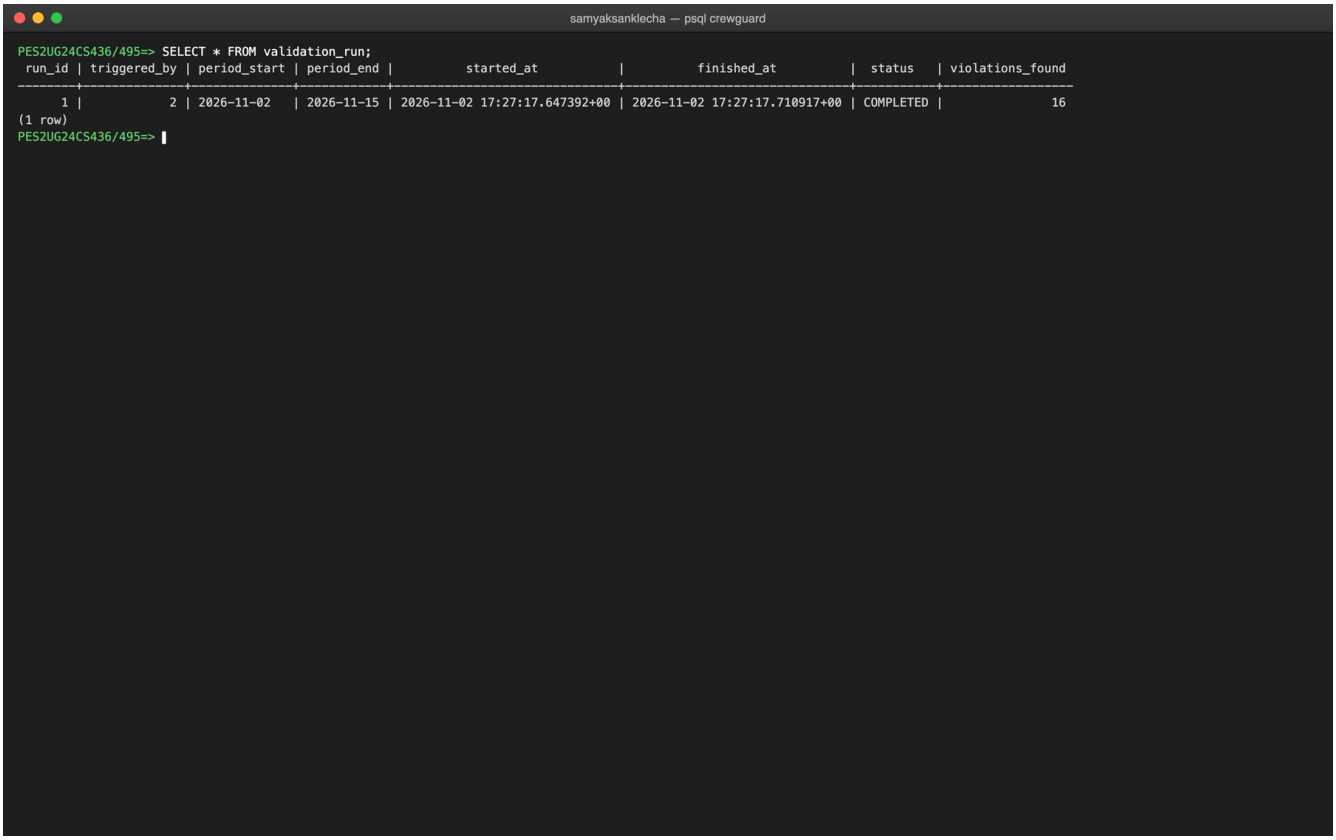


Figure 6.9: *Rows in validation_run after one run*

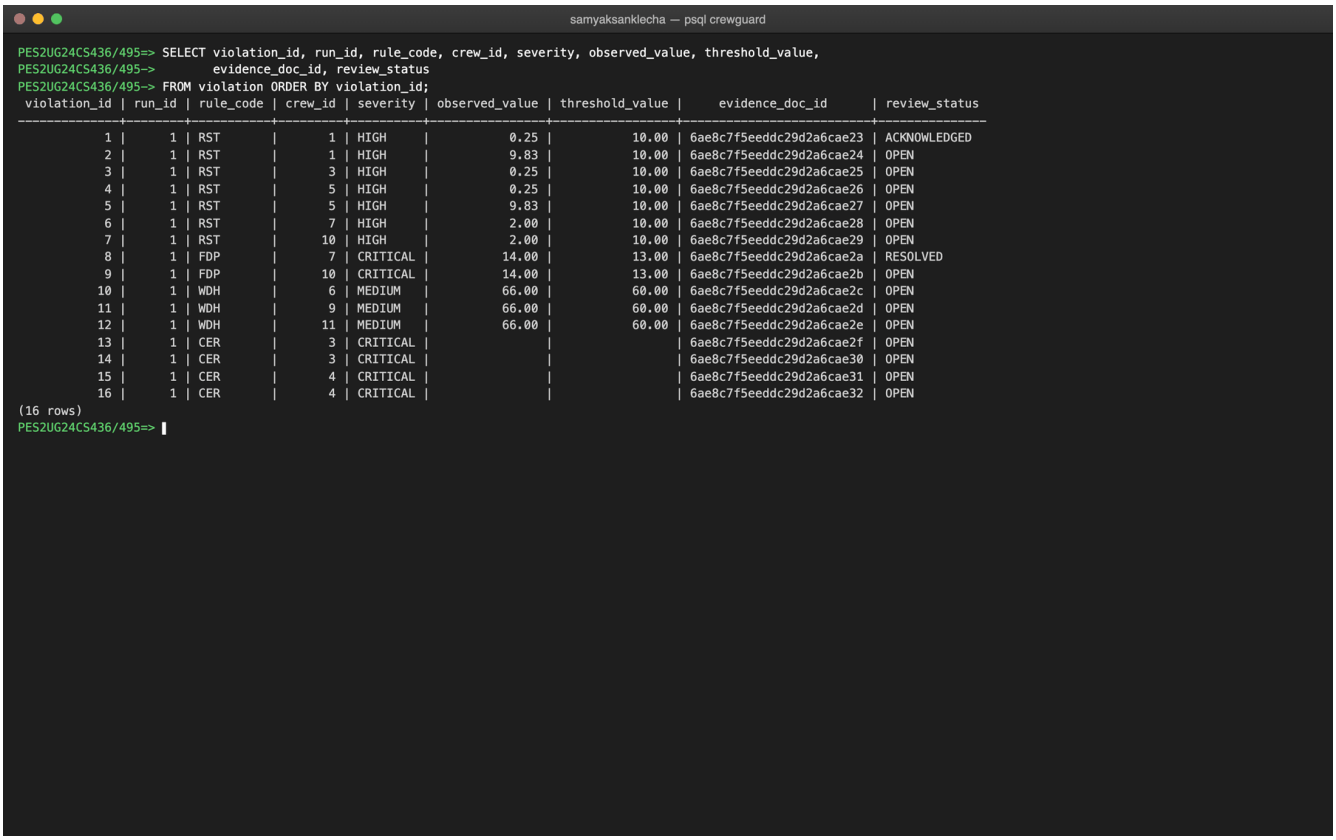


Figure 6.10: *Rows in violation: the 16 planted breaches*

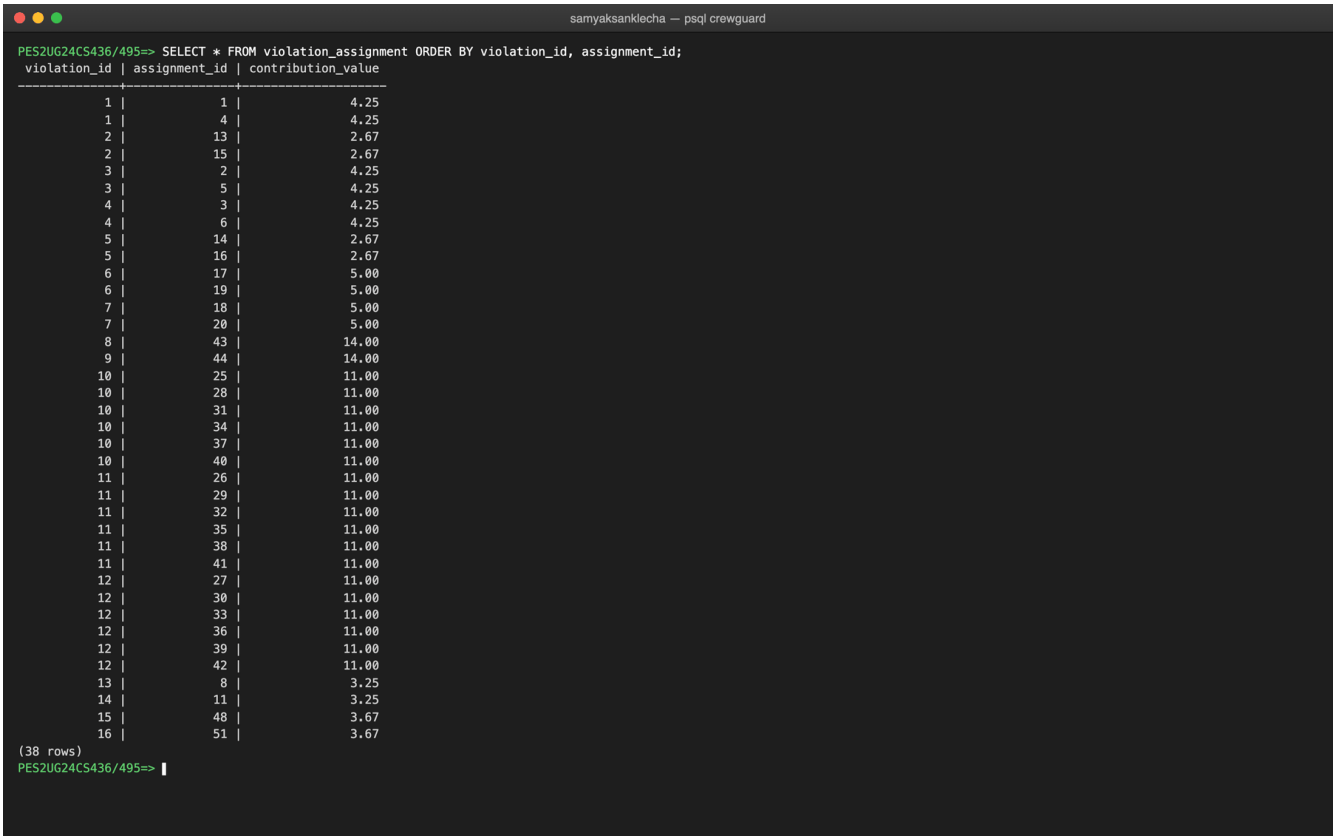


Figure 6.11: *Rows in violation_assignment: the duties behind each violation*

Inserting through the constraints

A valid insert succeeds and returns the generated key. Invalid inserts are rejected by the database, and the error names the constraint that fired: UNIQUE on employee_code, CHECK on the role, CHECK on distinct airports, and CHECK on the rating dates.

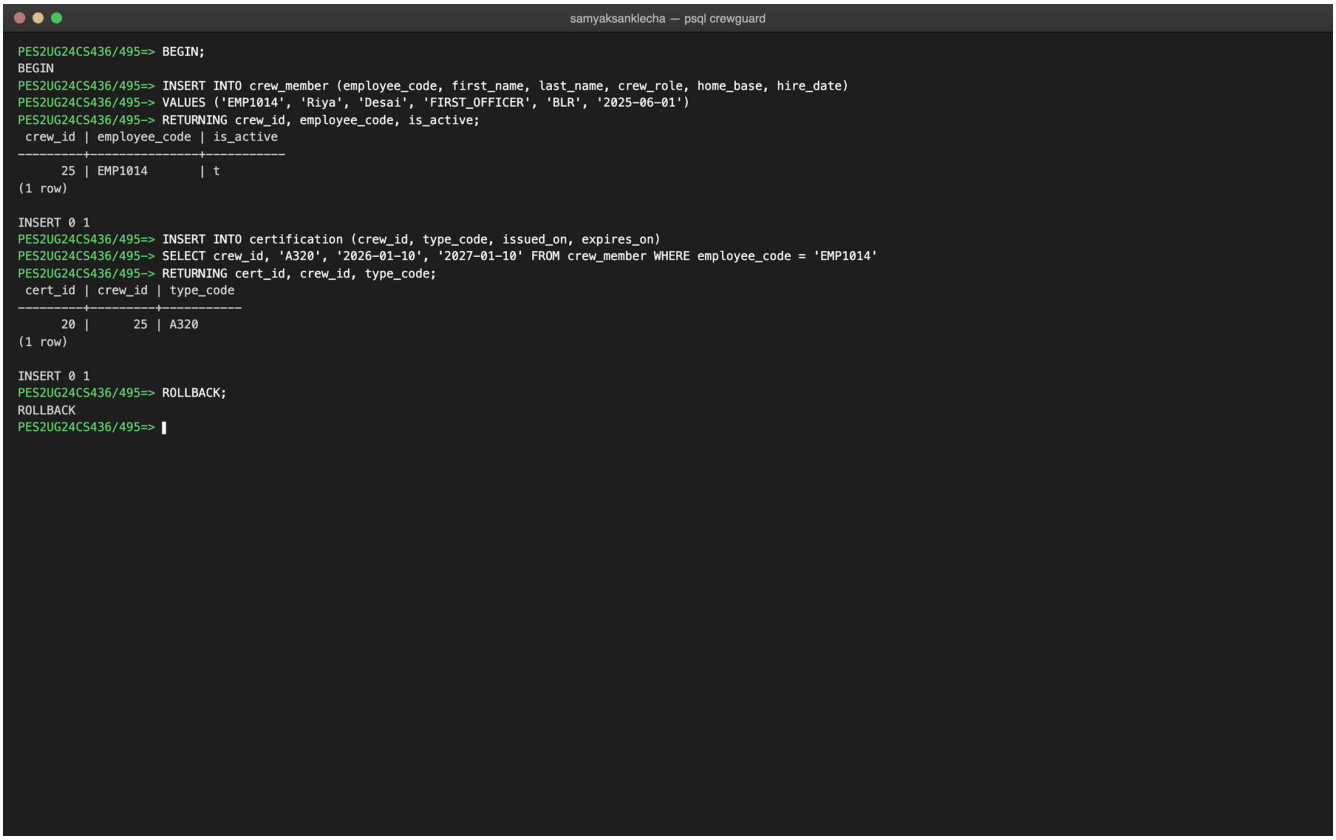


Figure 6.12: *A valid INSERT inside a transaction (rolled back afterwards)*

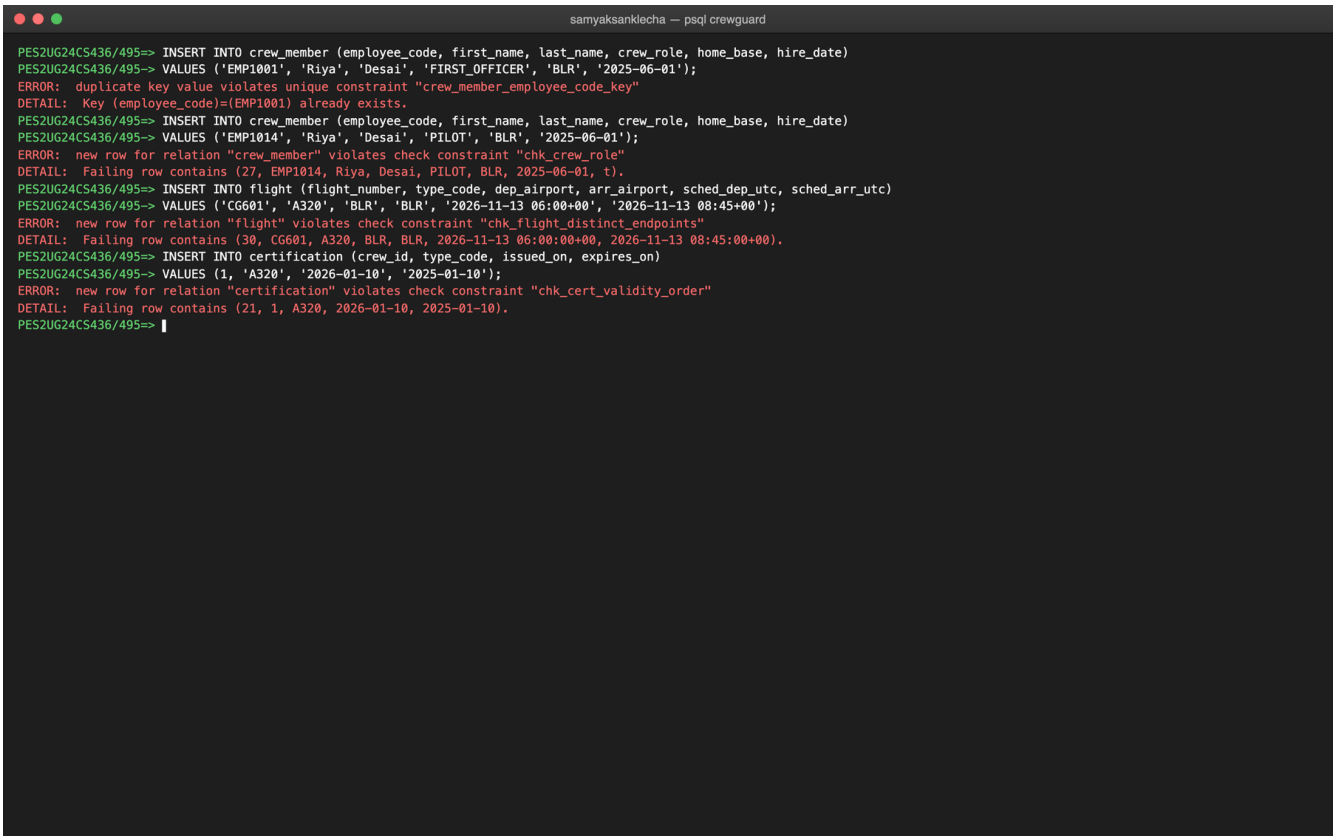


Figure 6.13: *Invalid INSERTs rejected by UNIQUE and CHECK constraints*

7. QUERIES

All queries below were run in psql on the sample data after one validation run. Statements that change data were run inside BEGIN ... ROLLBACK so the sample data stays intact. db/queries.sql in the repository contains these and further queries.

7.1 SIMPLE QUERY WITH GROUP BY, AGGREGATE

The first query aggregates the roster by role using COUNT, COUNT(DISTINCT), SUM, AVG and MAX over the v_duty_roster view. The second uses HAVING to keep only crew with more than 30 rostered hours. The third counts violations per rule and uses an aggregate FILTER clause to count the ones still open.

```
SELECT assigned_role,
       COUNT(*)                    AS duties,
       COUNT(DISTINCT crew_id)     AS crew,
       ROUND(SUM(duty_hours), 2)   AS total_hours,
       ROUND(AVG(duty_hours), 2)   AS avg_hours,
       MAX(duty_hours)             AS longest_duty
FROM v_duty_roster
GROUP BY assigned_role
ORDER BY total_hours DESC;

SELECT crew_name, COUNT(*) AS duties, SUM(duty_hours) AS hours
FROM v_duty_roster
GROUP BY crew_id, crew_name
HAVING SUM(duty_hours) > 30
ORDER BY hours DESC;

SELECT rule_code,
       COUNT(*)                                          AS violations,
       COUNT(*) FILTER (WHERE review_status = 'OPEN')    AS still_open
FROM violation
GROUP BY rule_code
ORDER BY violations DESC;
```

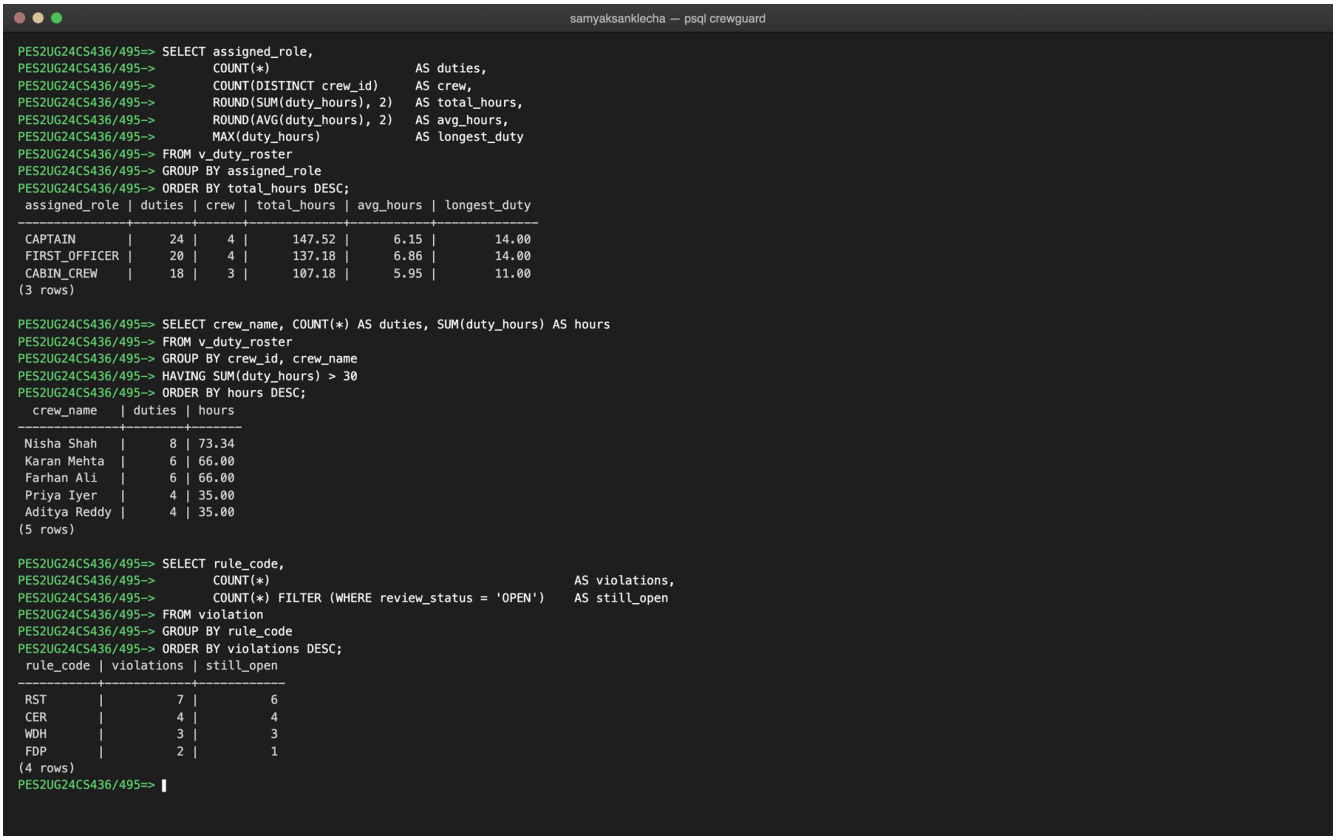


Figure 7.1: *GROUP BY with aggregates, HAVING and FILTER*

7.2 UPDATE OPERATION

The first UPDATE moves a crew member to another base. The second records an auditor's review; the trg_violation_review trigger stamps reviewed_at automatically, as RETURNING shows. The third tries to make a flight arrive before it departs and is rejected by the CHECK constraint chk_flight_time_order, which aborts the transaction.

```
BEGIN;
UPDATE crew_member SET home_base = 'HYD'
WHERE employee_code = 'EMP1006'
RETURNING crew_id, first_name, last_name, home_base;

UPDATE violation
SET review_status = 'ACKNOWLEDGED', review_note = 'Weekly limit breach confirmed.', reviewed_by
= 2
WHERE violation_id = 11
RETURNING violation_id, rule_code, review_status, reviewed_by, reviewed_at;

UPDATE flight SET sched_arr_utc = sched_dep_utc - INTERVAL '1 hour'
WHERE flight_number = 'CG101';
ROLLBACK;
```

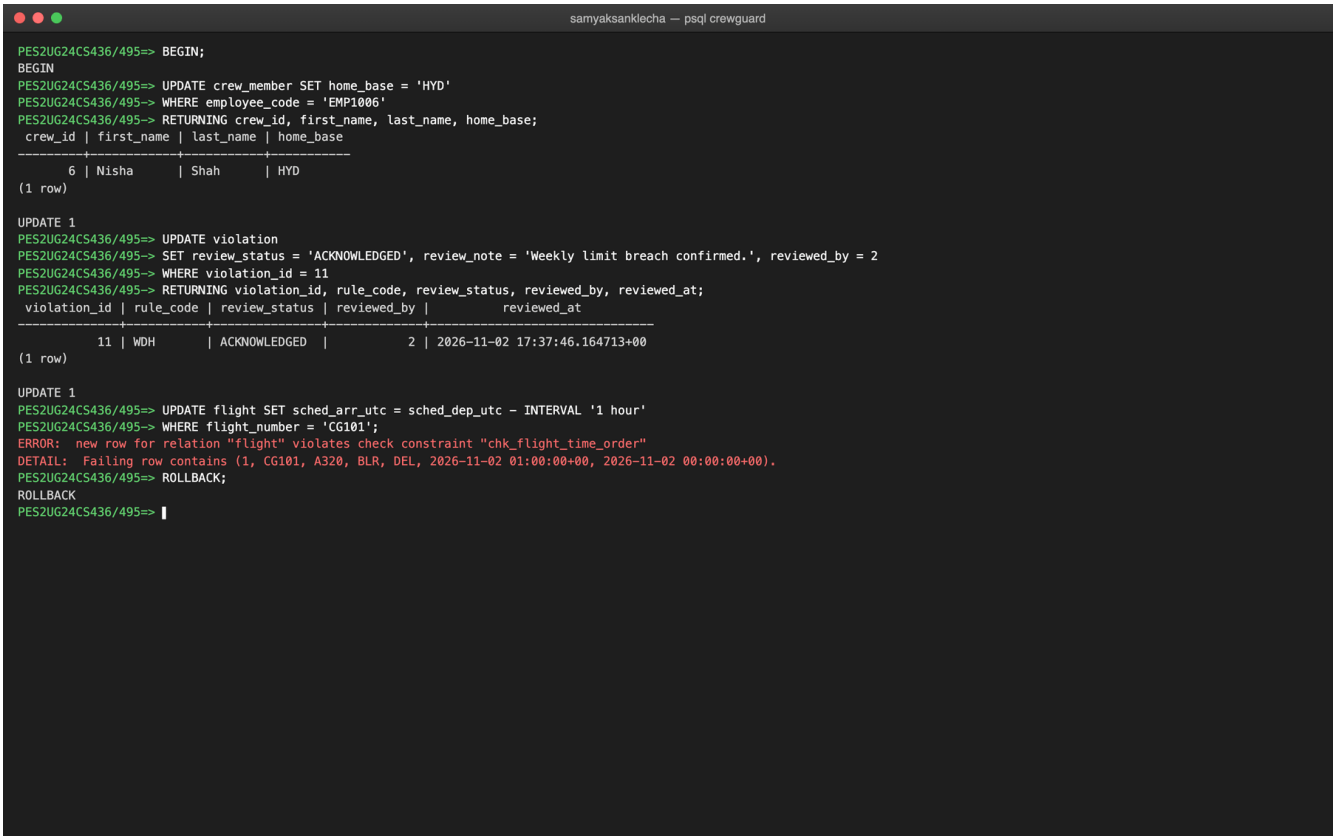


Figure 7.2: *UPDATE statements, including one rejected by a CHECK constraint*

7.3 DELETE OPERATION

Deleting a crew member cascades: ON DELETE CASCADE removes the member's certificates, duties and violations in the same statement, as the before and after counts show. Deleting an airport that is still in use is rejected by ON DELETE RESTRICT on the foreign keys that reference it.

```
BEGIN;
SELECT (SELECT COUNT(*) FROM certification   WHERE crew_id = 3) AS certificates,
       (SELECT COUNT(*) FROM duty_assignment WHERE crew_id = 3) AS duties,
       (SELECT COUNT(*) FROM violation       WHERE crew_id = 3) AS violations;

DELETE FROM crew_member WHERE crew_id = 3
RETURNING crew_id, employee_code, first_name, last_name;

SELECT (SELECT COUNT(*) FROM certification   WHERE crew_id = 3) AS certificates,
       (SELECT COUNT(*) FROM duty_assignment WHERE crew_id = 3) AS duties,
       (SELECT COUNT(*) FROM violation       WHERE crew_id = 3) AS violations;
ROLLBACK;

DELETE FROM airport WHERE iata_code = 'BLR';
```

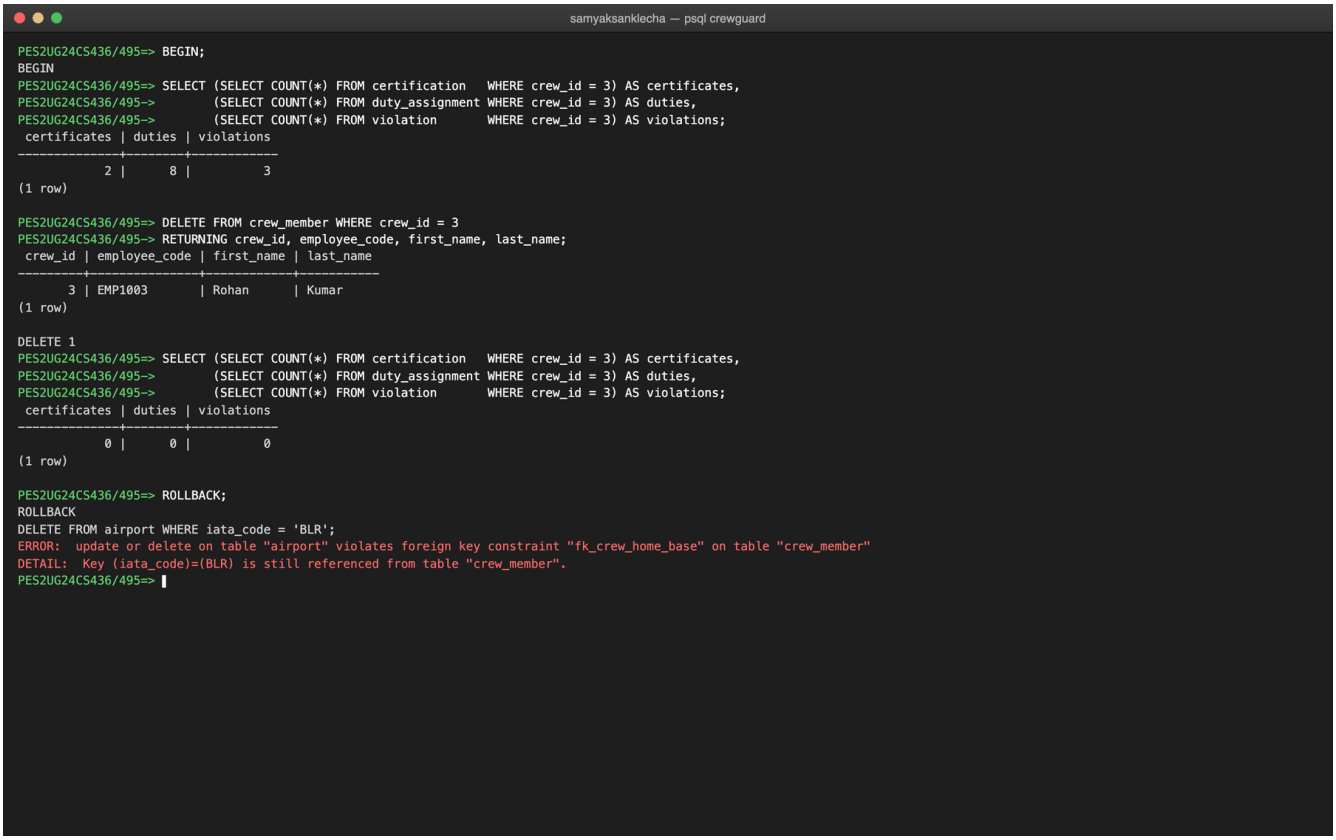


Figure 7.3: *DELETE with ON DELETE CASCADE, and a DELETE blocked by RESTRICT*

7.4 CORRELATED QUERY

A correlated sub-query refers to the outer row, so it is evaluated once per outer row. The first query counts, for each airport, the crew based there and the flights departing and arriving. The second lists crew whose total duty hours exceed the average of their own role; the inner query reads the outer row's crew_role.

```
SELECT a.iata_code, a.city,
       (SELECT COUNT(*) FROM crew_member c WHERE c.home_base   = a.iata_code) AS crew_based,
       (SELECT COUNT(*) FROM flight f      WHERE f.dep_airport = a.iata_code) AS departures,
       (SELECT COUNT(*) FROM flight f      WHERE f.arr_airport = a.iata_code) AS arrivals
FROM airport a
ORDER BY departures DESC, a.iata_code;

SELECT s.crew_name, s.crew_role, s.total_duty_hours
FROM v_crew_summary s
WHERE s.total_duty_hours > (SELECT AVG(t.total_duty_hours)
                            FROM v_crew_summary t
                            WHERE t.crew_role = s.crew_role)
ORDER BY s.crew_role, s.total_duty_hours DESC;
```

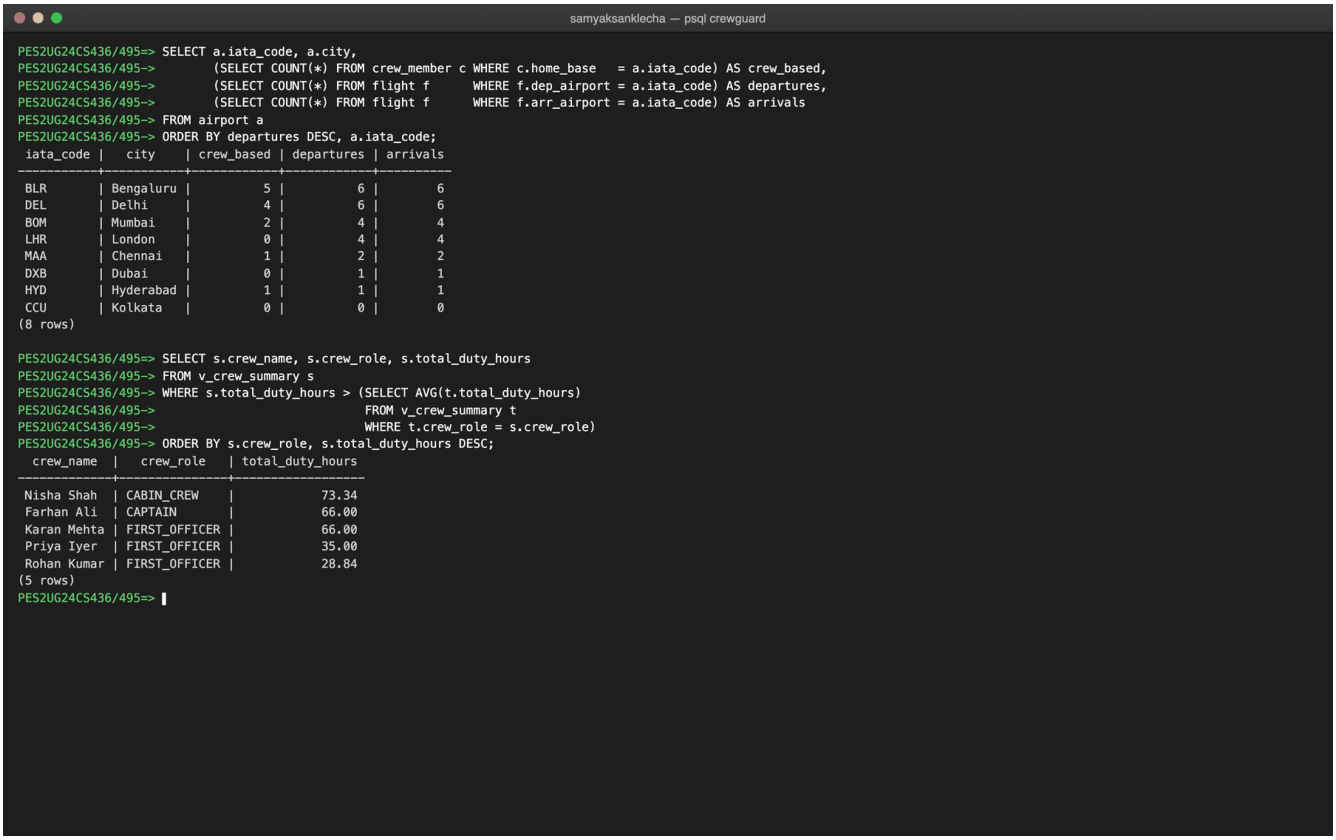


Figure 7.4: *Correlated sub-queries*

7.5 NESTED QUERY

The first query uses NOT IN with an independent sub-query to find crew who have never been rostered (an inactive cabin crew member and a newly hired first officer). The second uses a nested NOT EXISTS anti-join to find pilots who flew an aircraft type without a rating valid on the day of the flight, the same logic the validator uses for the CER rule.

```
SELECT employee_code, first_name || ' ' || last_name AS name, crew_role, is_active
FROM crew_member
WHERE crew_id NOT IN (SELECT crew_id FROM duty_assignment);

SELECT DISTINCT cm.employee_code, cm.first_name, f.flight_number, f.type_code
FROM duty_assignment da
JOIN crew_member cm USING (crew_id)
JOIN flight f USING (flight_id)
WHERE da.assigned_role <> 'CABIN_CREW'
  AND NOT EXISTS (SELECT 1 FROM certification c
                  WHERE c.crew_id = da.crew_id
                    AND c.type_code = f.type_code
                    AND f.sched_dep_utc::date BETWEEN c.issued_on AND c.expires_on)
ORDER BY cm.employee_code, f.flight_number;
```

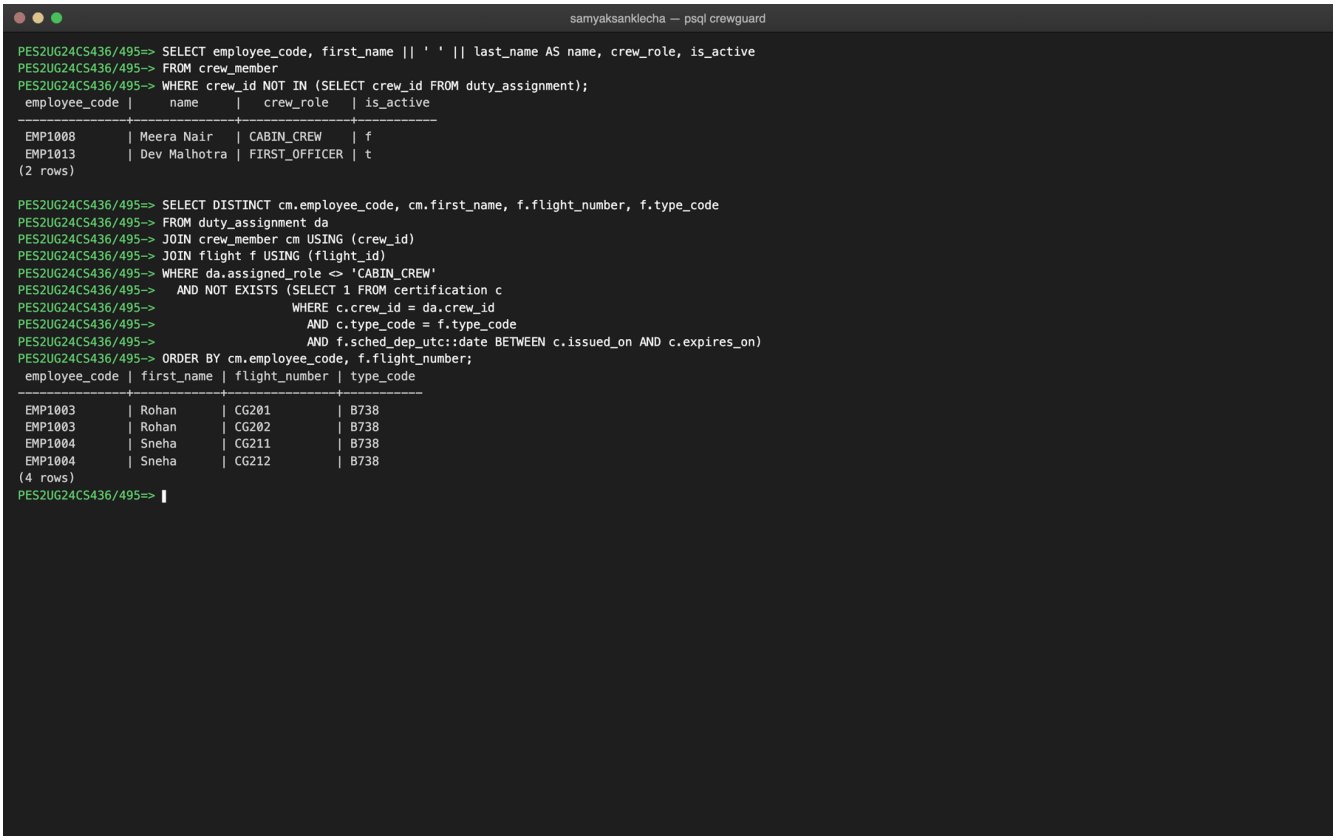


Figure 7.5: *Nested queries with NOT IN and NOT EXISTS*

7.6 VIEWS

Three views keep join logic inside the database. v_duty_roster joins duty_assignment, crew_member and flight. v_crew_summary gives one row per crew member; it LEFT JOINs two GROUP BY sub-queries so crew with no duties still appear and the duty and certificate counts do not multiply each other. v_violation_report gives one readable row per violation with its rule, crew member, reviewer and number of linked duties.

```
CREATE OR REPLACE VIEW v_crew_summary AS
SELECT cm.crew_id,
       cm.employee_code,
       cm.first_name,
       cm.last_name,
       cm.first_name || ' ' || cm.last_name AS crew_name,
       cm.crew_role,
       cm.home_base,
       cm.hire_date,
       cm.is_active,
       COALESCE(d.duty_count, 0)  AS duty_count,
       COALESCE(d.duty_hours, 0)  AS total_duty_hours,
       d.last_duty_end,
       COALESCE(c.valid_certs, 0) AS valid_certs
FROM crew_member cm
LEFT JOIN (
    SELECT crew_id,
           COUNT(*)::int                                        AS duty_count,
           SUM(fn_duty_hours(duty_start_utc, duty_end_utc))  AS duty_hours,
```

```
           MAX(duty_end_utc)                                    AS last_duty_end
    FROM duty_assignment
    GROUP BY crew_id
) d ON d.crew_id = cm.crew_id
LEFT JOIN (
    SELECT crew_id, COUNT(*)::int AS valid_certs
    FROM certification
    WHERE expires_on >= CURRENT_DATE
    GROUP BY crew_id
) c ON c.crew_id = cm.crew_id;

SELECT crew_name, crew_role, home_base, duty_count, total_duty_hours, valid_certs
FROM v_crew_summary
ORDER BY total_duty_hours DESC;

SELECT violation_id, rule_code, crew_name, observed_value, threshold_value,
       review_status, reviewer, duty_count
FROM v_violation_report
ORDER BY violation_id;
```

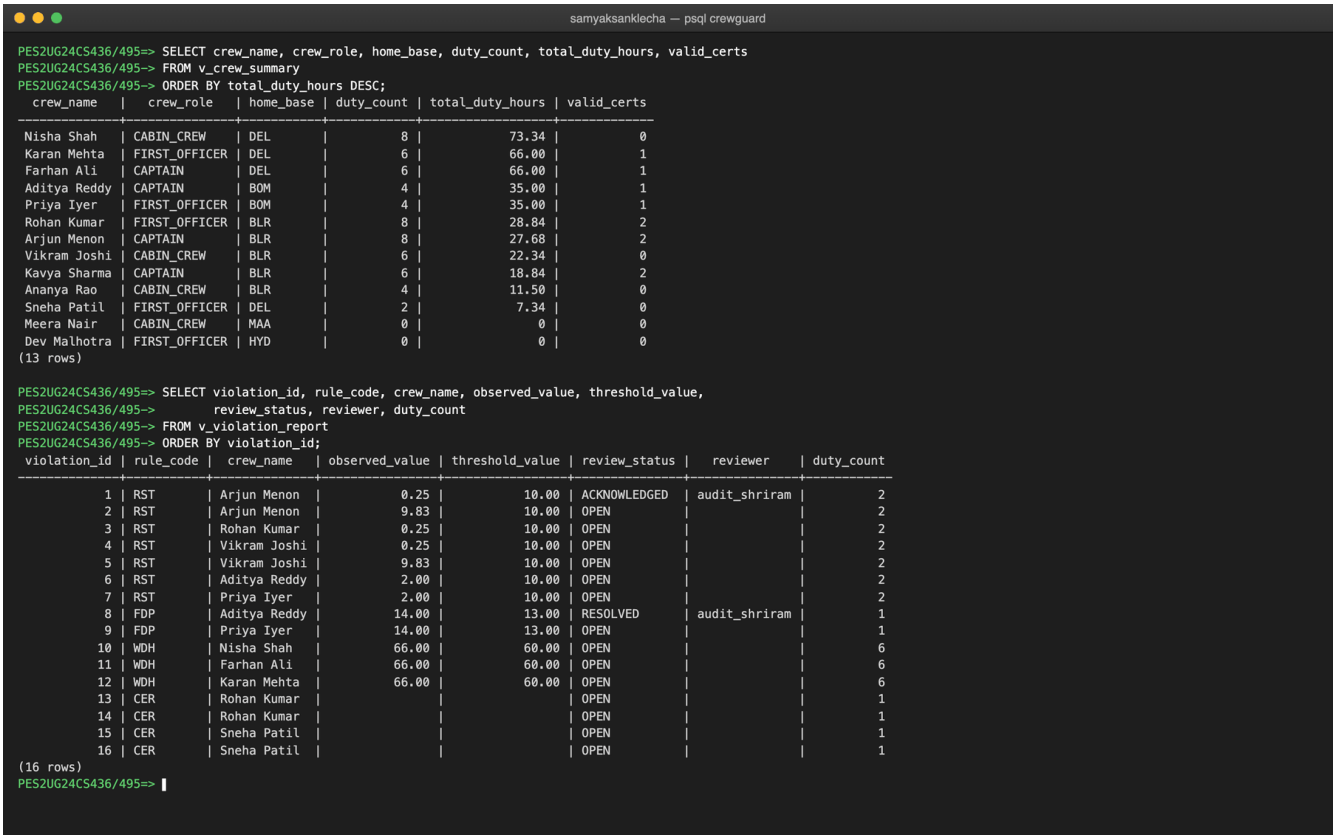


Figure 7.6: *Querying the v_crew_summary and v_violation_report views*

7.7 WINDOW FUNCTIONS

LAG reads the previous row of the same partition, so the rest before each duty is the gap since the same crew member's previous duty ended; a running SUM gives cumulative hours. Arjun Menon's 0.25-hour rest between CG101 and CG102 is visible directly.

```
SELECT crew_name, flight_number, duty_start_utc, duty_end_utc,
       ROUND(EXTRACT(EPOCH FROM duty_start_utc - LAG(duty_end_utc) OVER w) / 3600, 2) AS
rest_hours,
       SUM(duty_hours) OVER w AS running_hours
```

```
FROM v_duty_roster
WHERE crew_id = 1
WINDOW w AS (PARTITION BY crew_id ORDER BY duty_start_utc);
```

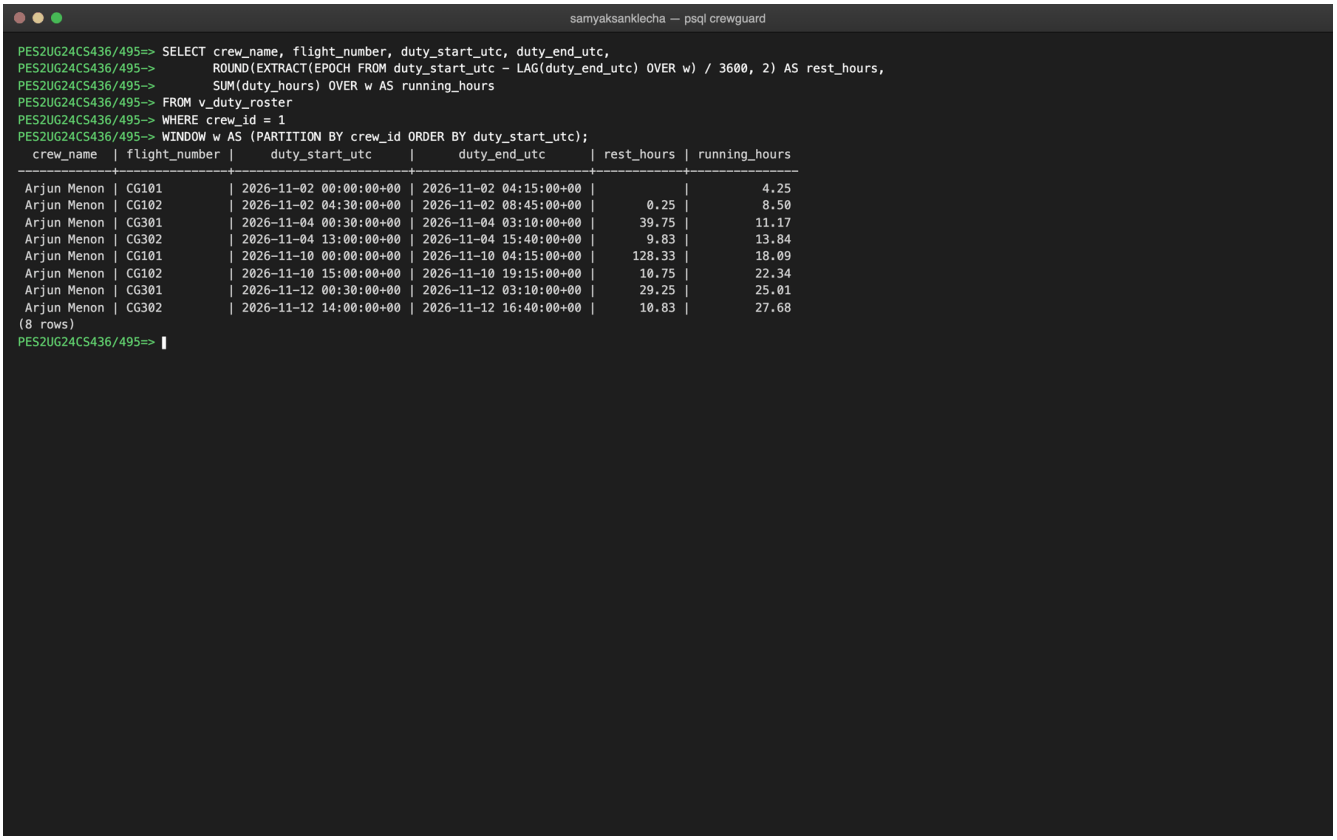


***Figure** 7.7: LAG and running SUM over one crew member's duties*

7.8 FULL-TEXT SEARCH AND INDEX USE

The rule catalogue is searched with to_tsvector and websearch_to_tsquery and ranked with ts_rank; the expression matches the GIN index exactly so the index can be used. EXPLAIN ANALYZE shows the composite index idx_duty_crew_start serving a crew member's duties in time order (sequential scans are switched off for the demonstration because the sample table is small).

```
SELECT rule_code, rule_name,
       ts_rank(to_tsvector('english', rule_name || ' ' || description || ' ' ||
COALESCE(regulation_ref, '')),
               websearch_to_tsquery('english', 'rest')) AS rank
FROM rule_definition
WHERE to_tsvector('english', rule_name || ' ' || description || ' ' || COALESCE(regulation_ref,
''))
      @@ websearch_to_tsquery('english', 'rest');

SET enable_seqscan = off;
EXPLAIN ANALYZE
SELECT * FROM duty_assignment WHERE crew_id = 1 ORDER BY duty_start_utc;
RESET enable_seqscan;
```

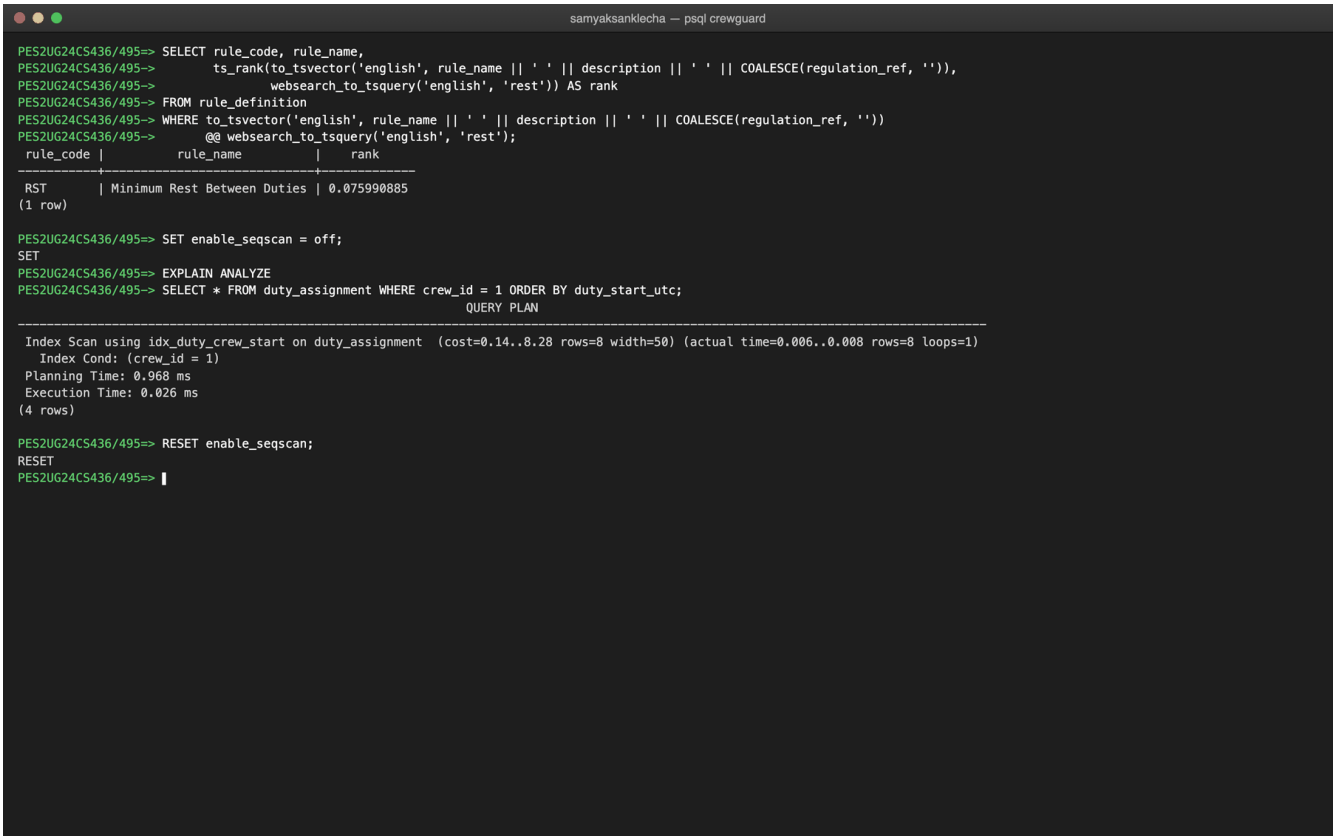


Figure 7.8: *Full-text search and EXPLAIN ANALYZE using the composite index*

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

Four SQL functions are defined in db/02_roster.sql. fn_duty_hours is IMMUTABLE (same input, same output) and the others are STABLE because they read tables. fn_crew_roster is table-valued: it returns rows and is used like a table in FROM.

```
CREATE OR REPLACE FUNCTION fn_duty_hours(p_start TIMESTAMPTZ, p_end TIMESTAMPTZ)
RETURNS NUMERIC
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT ROUND((EXTRACT(EPOCH FROM (p_end - p_start)) / 3600)::numeric, 2);
$$;

CREATE OR REPLACE FUNCTION fn_crew_duty_hours(p_crew_id INTEGER,
                                              p_from    TIMESTAMPTZ,
                                              p_to      TIMESTAMPTZ)
RETURNS NUMERIC
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(SUM(fn_duty_hours(duty_start_utc, duty_end_utc)), 0)
    FROM duty_assignment
    WHERE crew_id = p_crew_id
      AND duty_start_utc >= p_from
      AND duty_start_utc <  p_to;
$$;

CREATE OR REPLACE FUNCTION fn_cert_status(p_expires_on DATE, p_on DATE DEFAULT CURRENT_DATE)
RETURNS TEXT
LANGUAGE sql
STABLE
AS $$
    SELECT CASE
             WHEN p_expires_on <  p_on              THEN 'EXPIRED'
             WHEN p_expires_on <  p_on + 60         THEN 'EXPIRING'
             ELSE 'VALID'
           END;
$$;

CREATE OR REPLACE FUNCTION fn_crew_roster(p_crew_id INTEGER)
RETURNS TABLE (assignment_id  INTEGER,
               flight_id      INTEGER,
               flight_number  VARCHAR(10),
               type_code      VARCHAR(10),
               sector         TEXT,
               assigned_role  VARCHAR(20),
               duty_start_utc TIMESTAMPTZ,
               duty_end_utc   TIMESTAMPTZ,
               duty_hours     NUMERIC)
LANGUAGE sql
STABLE
AS $$
    SELECT da.assignment_id, f.flight_id, f.flight_number, f.type_code,
           f.dep_airport || ' → ' || f.arr_airport,
           da.assigned_role, da.duty_start_utc, da.duty_end_utc,
           fn_duty_hours(da.duty_start_utc, da.duty_end_utc)
    FROM duty_assignment da
    JOIN flight f ON f.flight_id = da.flight_id
    WHERE da.crew_id = p_crew_id
```

```
    ORDER BY da.duty_start_utc;
$$;
```

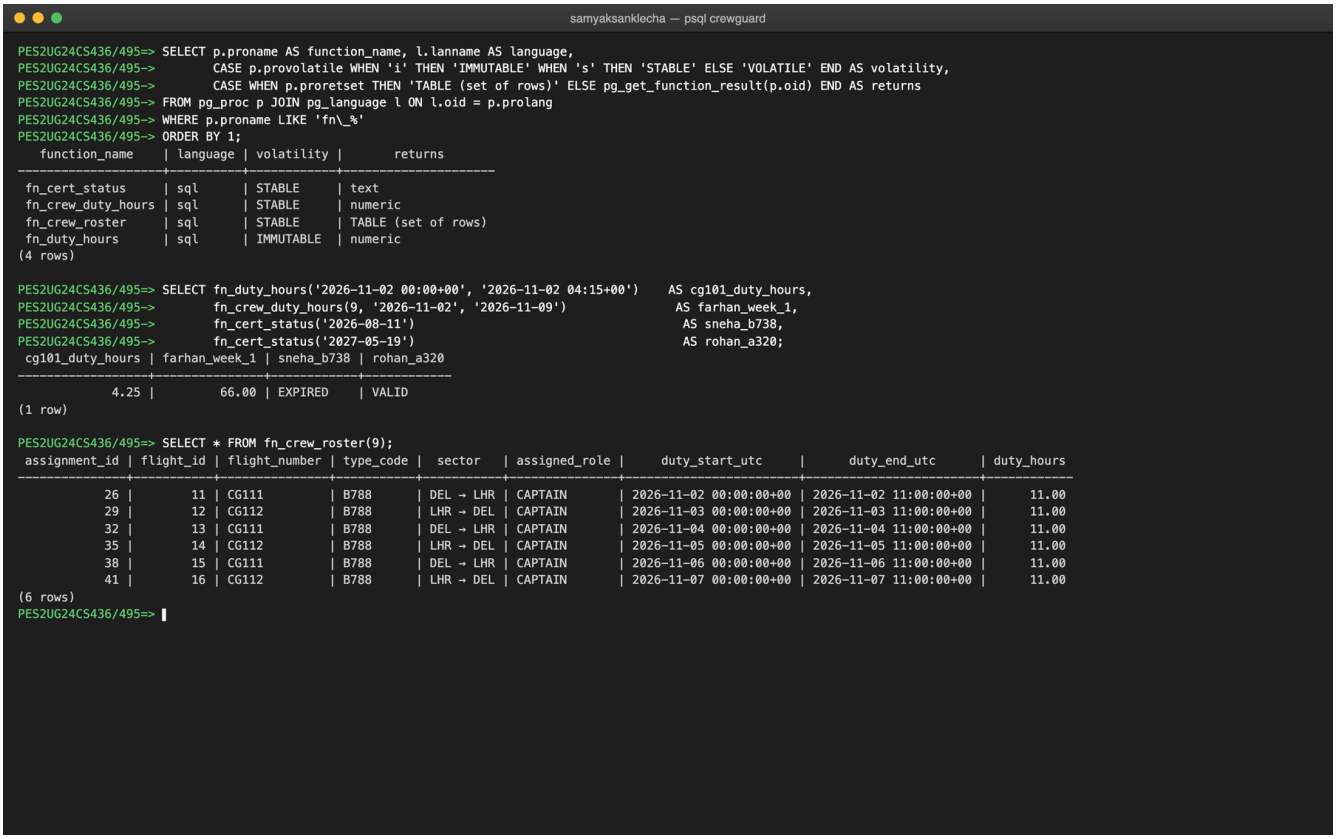


Figure 8.1: *The four functions and their results*

The validator is the stored procedure sp_run_validation in db/03_compliance.sql. It opens a validation_run, reads each rule's threshold and severity from rule_definition (no limit is hard-coded), checks the four rules over the period with set-based SQL, inserts one violation per breach and links the causing duties in violation_assignment. It does not COMMIT itself: the caller runs it inside a transaction, so a run is saved completely or not at all. RST uses LAG over each crew member's duties, FDP compares one duty's length with the limit, WDH uses a rolling SUM over a seven-day RANGE window and keeps each crew member's worst window with DISTINCT ON, and CER uses a NOT EXISTS anti-join on certification.

```
CREATE OR REPLACE PROCEDURE sp_run_validation(p_user_id INTEGER,
                                              p_start   DATE,
                                              p_end     DATE,
                                              INOUT p_run_id INTEGER DEFAULT NULL)
LANGUAGE plpgsql
AS $$
DECLARE
    -- AT TIME ZONE 'UTC': the result must not depend on the session time zone
    v_from TIMESTAMPTZ := p_start::timestamp AT TIME ZONE 'UTC';
    v_to   TIMESTAMPTZ := (p_end + 1)::timestamp AT TIME ZONE 'UTC';
    v_rule rule_definition%ROWTYPE;
    v_id   INTEGER;
    r      RECORD;
BEGIN
    INSERT INTO validation_run (triggered_by, period_start, period_end)
```

```
    VALUES (p_user_id, p_start, p_end)
    RETURNING run_id INTO p_run_id;

    -- ---------- RST: minimum rest between two consecutive duties ----------
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'RST';
    IF v_rule.is_active THEN
        FOR r IN
            -- LAG reads the same crew member's previous duty, in time order
            SELECT d.*, fn_duty_hours(d.prev_end, d.duty_start_utc) AS rest_hours
            FROM (
                SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
                       LAG(da.assignment_id)  OVER w AS prev_id,
                       LAG(da.duty_start_utc) OVER w AS prev_start,
                       LAG(da.duty_end_utc)   OVER w AS prev_end
                FROM duty_assignment da
                WINDOW w AS (PARTITION BY da.crew_id ORDER BY da.duty_start_utc)
            ) d
            WHERE d.prev_id IS NOT NULL
              AND d.duty_start_utc >= v_from AND d.duty_start_utc < v_to
              AND fn_duty_hours(d.prev_end, d.duty_start_utc) < v_rule.threshold_value
            ORDER BY d.crew_id, d.duty_start_utc
        LOOP
            INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                                   observed_value, threshold_value)
            VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.prev_end, r.duty_start_utc,
                    r.rest_hours, v_rule.threshold_value)
            RETURNING violation_id INTO v_id;

            -- explanation: the two duties the short rest sits between
            INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
            VALUES (v_id, r.prev_id,       fn_duty_hours(r.prev_start, r.prev_end)),
                   (v_id, r.assignment_id, fn_duty_hours(r.duty_start_utc, r.duty_end_utc));
        END LOOP;
    END IF;

    -- ---------- FDP: maximum length of a single duty ----------
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'FDP';
    IF v_rule.is_active THEN
        FOR r IN
            SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
                   fn_duty_hours(da.duty_start_utc, da.duty_end_utc) AS hours
            FROM duty_assignment da
            WHERE da.duty_start_utc >= v_from AND da.duty_start_utc < v_to
              AND fn_duty_hours(da.duty_start_utc, da.duty_end_utc) > v_rule.threshold_value
            ORDER BY da.crew_id, da.duty_start_utc
        LOOP
            INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                                   observed_value, threshold_value)
            VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.duty_start_utc, r.duty_end_utc,
                    r.hours, v_rule.threshold_value)
            RETURNING violation_id INTO v_id;

            INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
            VALUES (v_id, r.assignment_id, r.hours);
        END LOOP;
    END IF;

    -- ---------- WDH: maximum duty hours in any rolling window ----------
    -- The window function gives every duty the total of that crew member's duties
    -- in the window ending at it. DISTINCT ON then keeps each crew member's single
    -- worst window (the earliest if tied), so one overworked week is ONE violation
    -- per crew per run, not one for every duty inside that week.
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'WDH';
    IF v_rule.is_active THEN
        FOR r IN
            SELECT DISTINCT ON (d.crew_id) d.crew_id, d.duty_start_utc, d.duty_end_utc, d.rolling_hours
            FROM (
                SELECT da.crew_id, da.duty_start_utc, da.duty_end_utc,
                       SUM(fn_duty_hours(da.duty_start_utc, da.duty_end_utc)) OVER (
                           PARTITION BY da.crew_id
                           ORDER BY da.duty_start_utc
                           RANGE BETWEEN make_interval(days => v_rule.window_days) PRECEDING AND CURRENT ROW
                       ) AS rolling_hours
                FROM duty_assignment da
            ) d
            WHERE d.duty_start_utc >= v_from AND d.duty_start_utc < v_to
              AND d.rolling_hours > v_rule.threshold_value
            ORDER BY d.crew_id, d.rolling_hours DESC, d.duty_start_utc
        LOOP
            INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                                   observed_value, threshold_value)
            VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity,
                    r.duty_start_utc - make_interval(days => v_rule.window_days), r.duty_end_utc,
                    r.rolling_hours, v_rule.threshold_value)
```

```
            RETURNING violation_id INTO v_id;

            -- explanation: every duty inside the window, each with its hours
            INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
            SELECT v_id, da.assignment_id, fn_duty_hours(da.duty_start_utc, da.duty_end_utc)
            FROM duty_assignment da
            WHERE da.crew_id = r.crew_id
              AND da.duty_start_utc >= r.duty_start_utc - make_interval(days => v_rule.window_days)
              AND da.duty_start_utc <= r.duty_start_utc;
        END LOOP;
    END IF;

    -- ---------- CER: pilot must hold a valid type rating ----------
    -- Anti-join: no certificate covers this aircraft type on the duty date.
    -- A renewal is a separate row, so an expired certificate does not trigger this
    -- as long as a newer one covers the date. Cabin crew are exempt.
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'CER';
    IF v_rule.is_active THEN
        FOR r IN
            SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
                   fn_duty_hours(da.duty_start_utc, da.duty_end_utc) AS hours
            FROM duty_assignment da
            JOIN flight f ON f.flight_id = da.flight_id
            WHERE da.duty_start_utc >= v_from AND da.duty_start_utc < v_to
              AND da.assigned_role <> 'CABIN_CREW'
              AND NOT EXISTS (
                  SELECT 1 FROM certification c
                  WHERE c.crew_id = da.crew_id
                    AND c.type_code = f.type_code
                    AND (da.duty_start_utc AT TIME ZONE 'UTC')::date BETWEEN c.issued_on AND c.expires_on)
            ORDER BY da.crew_id, da.duty_start_utc
        LOOP
            -- there is no number to compare here, so observed and threshold stay NULL
            INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end)
            VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.duty_start_utc, r.duty_end_utc)
            RETURNING violation_id INTO v_id;

            INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
            VALUES (v_id, r.assignment_id, r.hours);
        END LOOP;
    END IF;

    -- trg_run_finish stamps finished_at; trg_violation_count already kept the count
    UPDATE validation_run SET status = 'COMPLETED' WHERE run_id = p_run_id;
END;
$$;
```

Calling the procedure for the first week inside a transaction creates run 2 with 12 violations (RST 7, WDH 3, CER 2); the triggers keep violations_found and finished_at correct. The explanation query then joins violation_assignment to v_duty_roster: Farhan Ali's WDH violation is caused by six 11-hour duties whose contributions add up to the observed 66 hours.

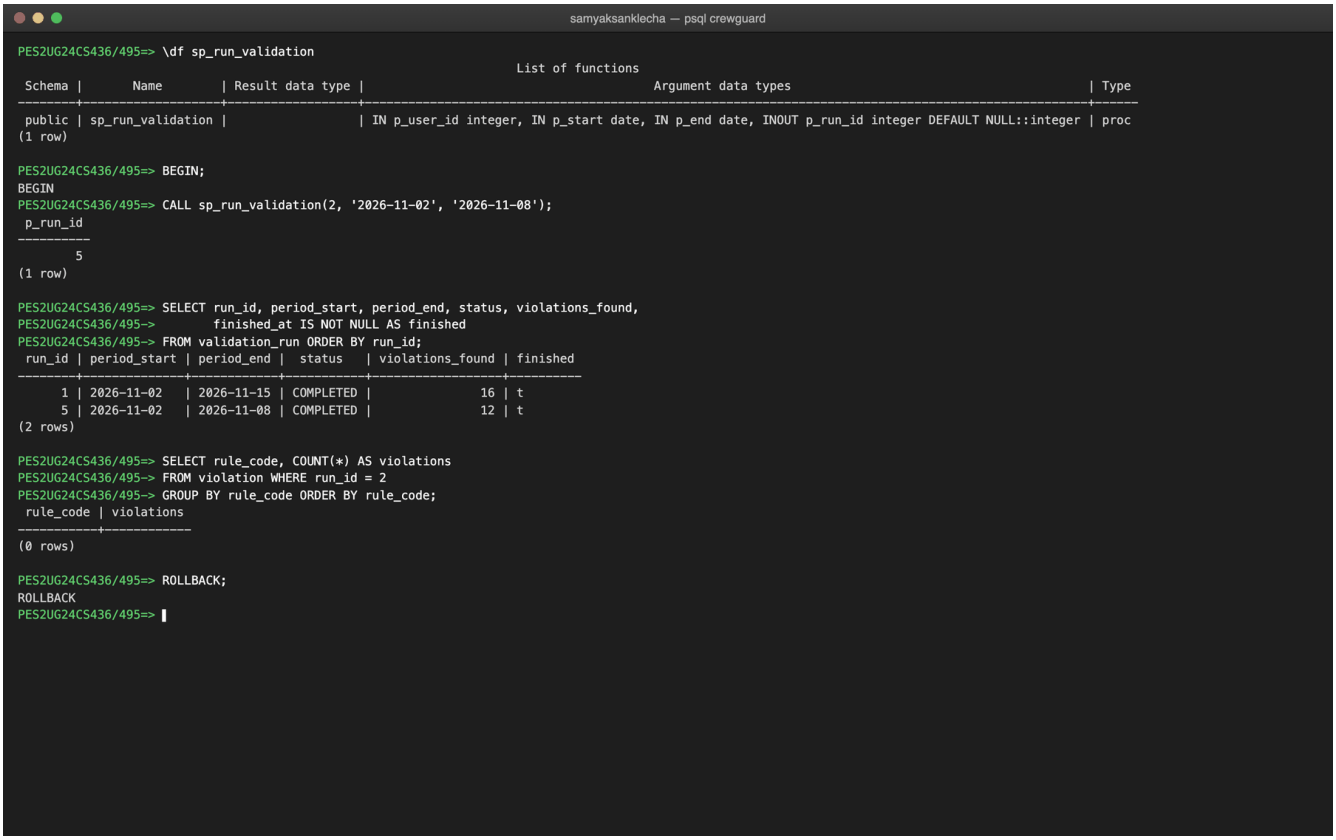


Figure 8.2: *CALL sp_run_validation for the first week*

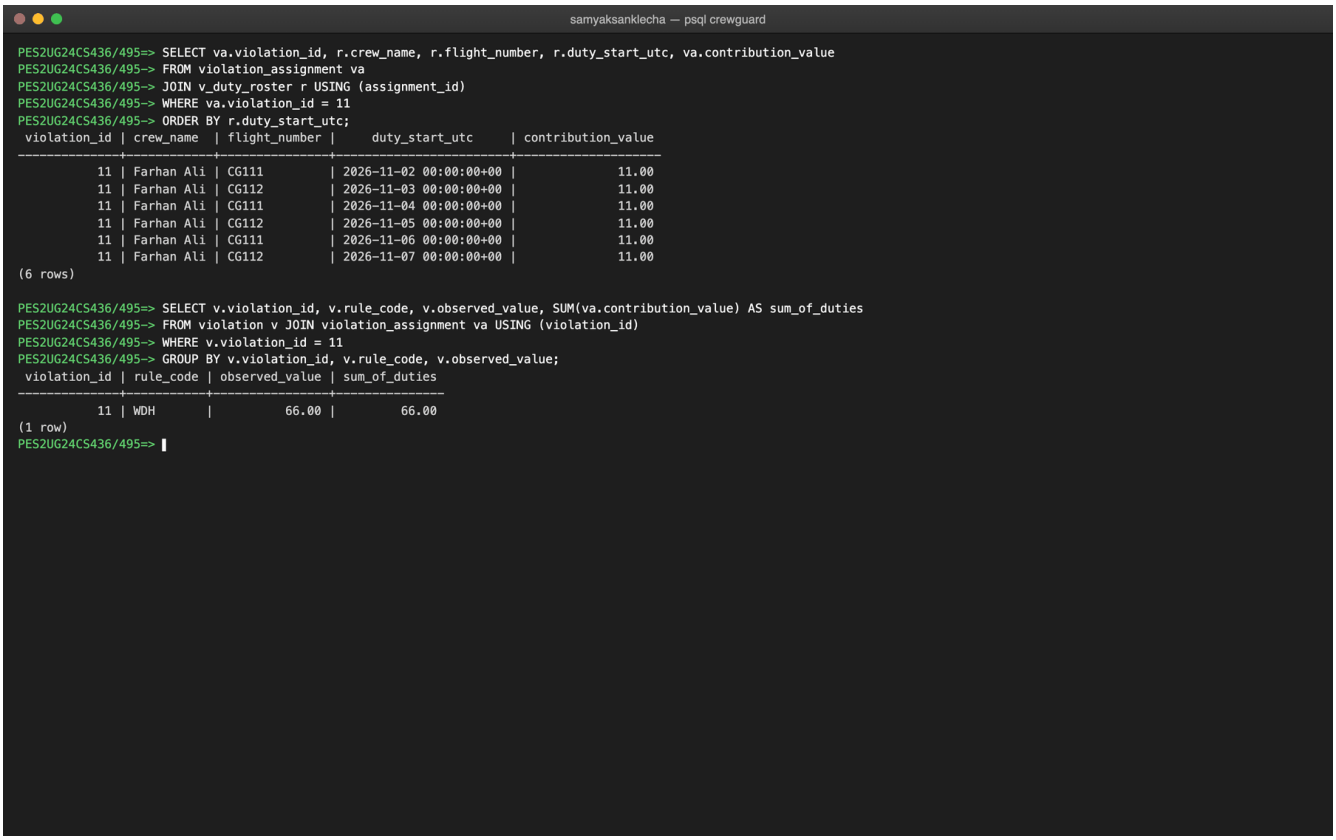


Figure 8.3: *The explanation: the duties behind a violation add up to the observed value*

8.2 TRIGGERS

Four triggers are used. trg_validate_duty prevents bad duties; the other three maintain derived values that the application must never have to keep in step itself.

Trigger	Table and timing	What it does
trg_validate_duty	duty_assignment, BEFORE INSERT OR UPDATE, per row	Rejects a duty if the crew member is inactive, if the assigned role differs from the crew member's role, or if the duty does not cover the whole flight. A CHECK constraint cannot do this because it needs data from other tables.
trg_violation_count	violation, AFTER INSERT OR DELETE, per row	Keeps validation_run.violations_found equal to the real number of violations.
trg_run_finish	validation_run, BEFORE UPDATE, per row	Stamps finished_at when a run becomes COMPLETED or FAILED.
trg_violation_review	violation, BEFORE UPDATE OF review_status, per row	Stamps reviewed_at when the review status changes.

Table 8.1: *Triggers*

```
CREATE OR REPLACE FUNCTION trg_fn_validate_duty()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_crew   crew_member%ROWTYPE;
    v_flight flight%ROWTYPE;
BEGIN
    SELECT * INTO v_crew   FROM crew_member WHERE crew_id   = NEW.crew_id;
    SELECT * INTO v_flight FROM flight      WHERE flight_id = NEW.flight_id;

    IF NOT v_crew.is_active THEN
        RAISE EXCEPTION 'Crew member % is inactive and cannot be rostered', v_crew.employee_code
            USING ERRCODE = 'check_violation', CONSTRAINT = 'trg_duty_crew_active';
    END IF;

    IF NEW.assigned_role <> v_crew.crew_role THEN
        RAISE EXCEPTION 'Crew member % is a %, not a %',
                        v_crew.employee_code, v_crew.crew_role, NEW.assigned_role
            USING ERRCODE = 'check_violation', CONSTRAINT = 'trg_duty_role_match';
    END IF;

    IF NEW.duty_start_utc > v_flight.sched_dep_utc OR NEW.duty_end_utc < v_flight.sched_arr_utc THEN
        RAISE EXCEPTION 'Duty must start before departure and end after arrival of flight %',
                        v_flight.flight_number
            USING ERRCODE = 'check_violation', CONSTRAINT = 'trg_duty_covers_flight';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validate_duty ON duty_assignment;
CREATE TRIGGER trg_validate_duty
BEFORE INSERT OR UPDATE ON duty_assignment
FOR EACH ROW
EXECUTE FUNCTION trg_fn_validate_duty();
```

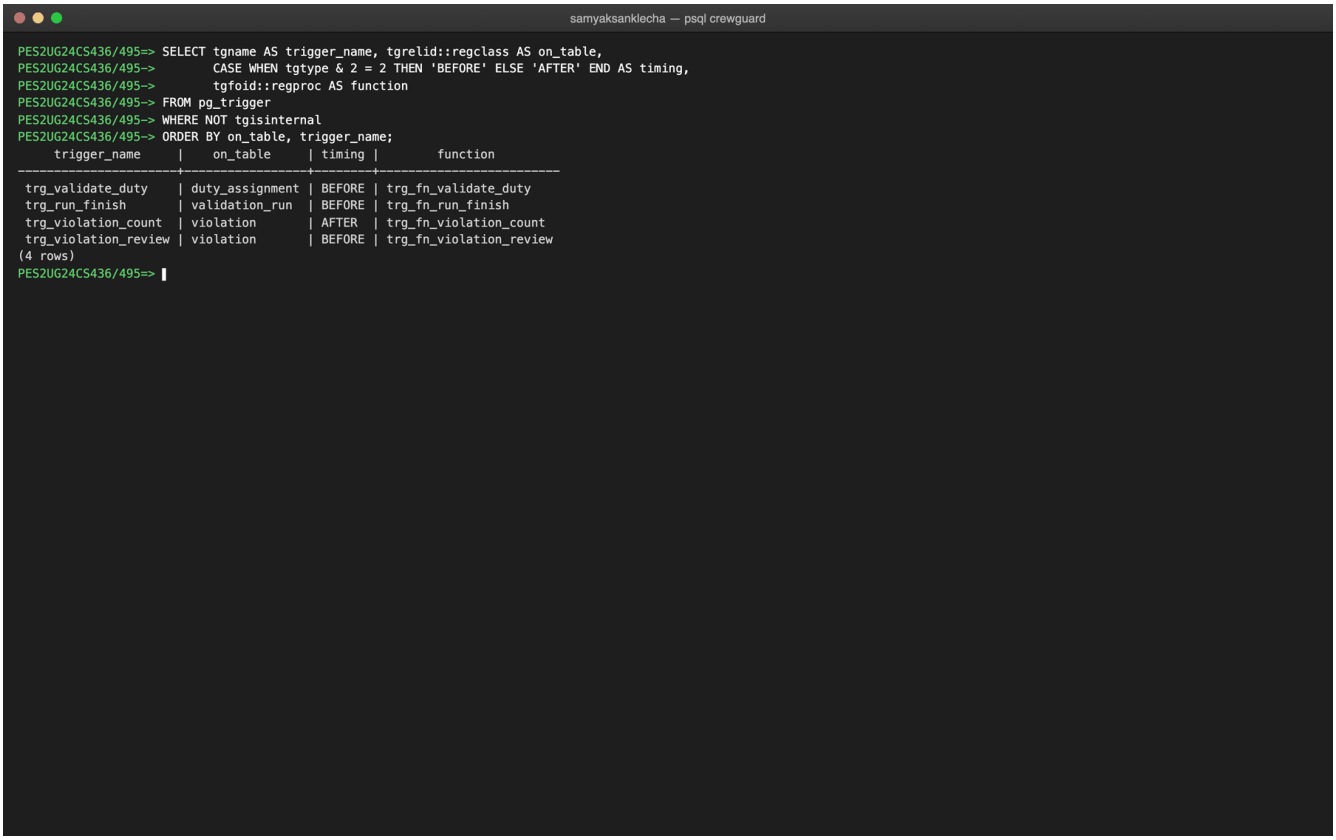


Figure 8.4: *The triggers installed in the database*

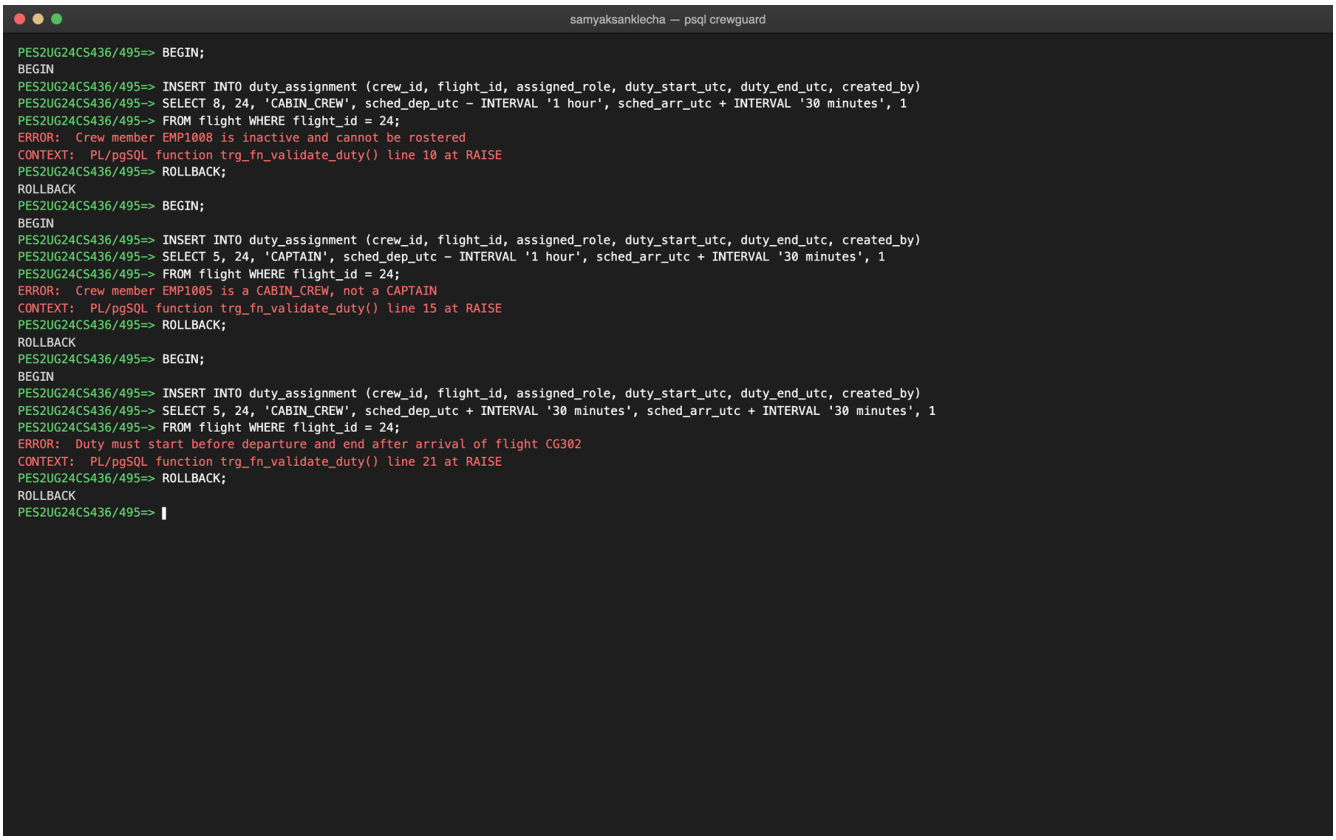


Figure 8.5: *trg_validate_duty rejecting an inactive crew member, a wrong role and a duty that does not cover its flight*

```
CREATE OR REPLACE FUNCTION trg_fn_violation_count()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE validation_run
        SET violations_found = violations_found + 1
        WHERE run_id = NEW.run_id;
        RETURN NEW;
    END IF;
    -- GREATEST: never goes below 0, which chk_run_violation_count would reject
    UPDATE validation_run
    SET violations_found = GREATEST(violations_found - 1, 0)
    WHERE run_id = OLD.run_id;
    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS trg_violation_count ON violation;
CREATE TRIGGER trg_violation_count
AFTER INSERT OR DELETE ON violation
FOR EACH ROW
EXECUTE FUNCTION trg_fn_violation_count();

-- Stamps finished_at when a run is closed.
-- clock_timestamp(), not NOW(): NOW() is frozen at the start of the transaction,
-- so a run closed in the same transaction that opened it would show 0 ms.
CREATE OR REPLACE FUNCTION trg_fn_run_finish()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status IN ('COMPLETED', 'FAILED') AND NEW.finished_at IS NULL THEN
        NEW.finished_at := clock_timestamp();
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_run_finish ON validation_run;
CREATE TRIGGER trg_run_finish
BEFORE UPDATE ON validation_run
FOR EACH ROW
EXECUTE FUNCTION trg_fn_run_finish();

-- Stamps reviewed_at when the review status actually changes.
-- UPDATE OF fires only if review_status is in the SET list, and the IF ignores
-- an update that writes the same status again (e.g. editing only the note).
CREATE OR REPLACE FUNCTION trg_fn_violation_review()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.review_status IS DISTINCT FROM OLD.review_status THEN
        NEW.reviewed_at := NOW();
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_violation_review ON violation;
CREATE TRIGGER trg_violation_review
BEFORE UPDATE OF review_status ON violation
FOR EACH ROW
EXECUTE FUNCTION trg_fn_violation_review();
```

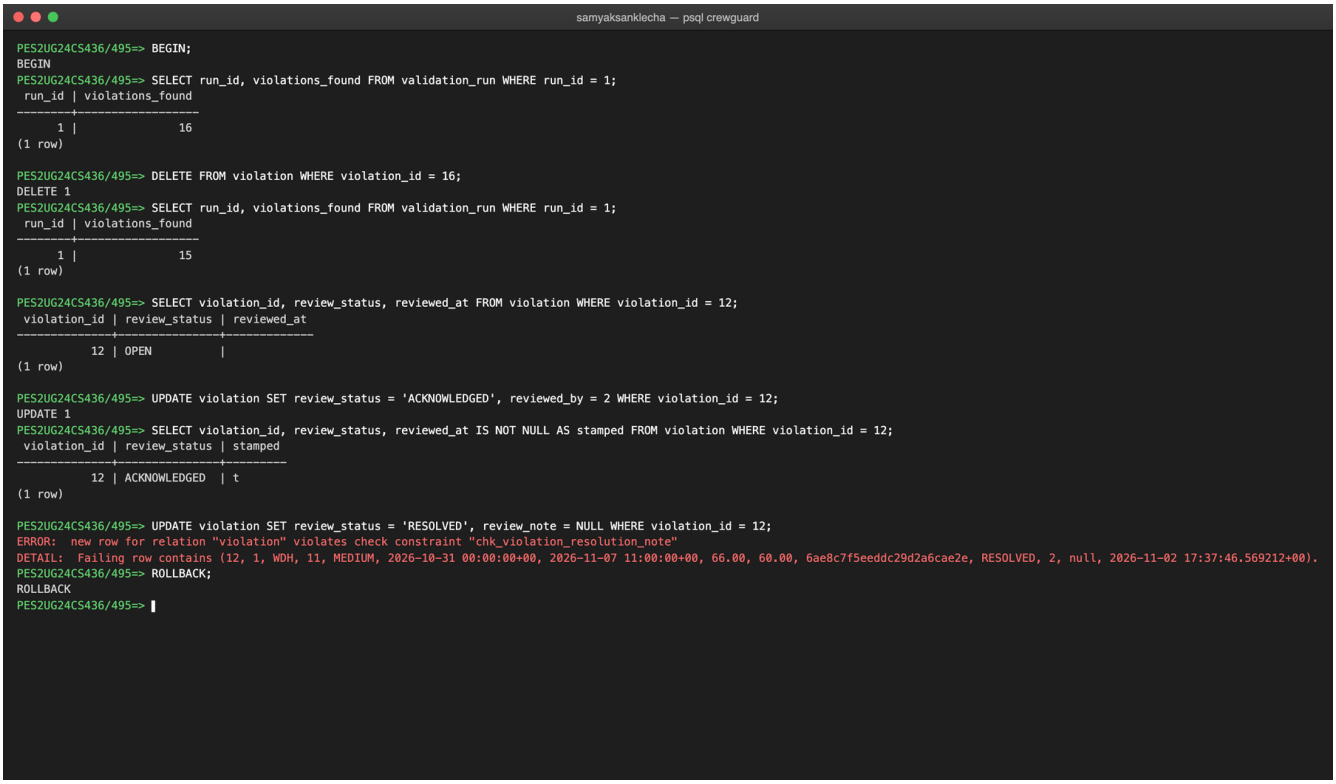


Figure 8.6: *Compliance triggers maintaining the count and the review time, and the resolution-note CHECK*

9. FRONT END DEVELOPMENT

The front end is a Next.js 16 application written in TypeScript with React 19 and Tailwind CSS. Pages are server components that query the database directly on the server; forms send JSON to API route handlers that check the user's role and run parameterised SQL. The browser never connects to a database.

9.1 System architecture

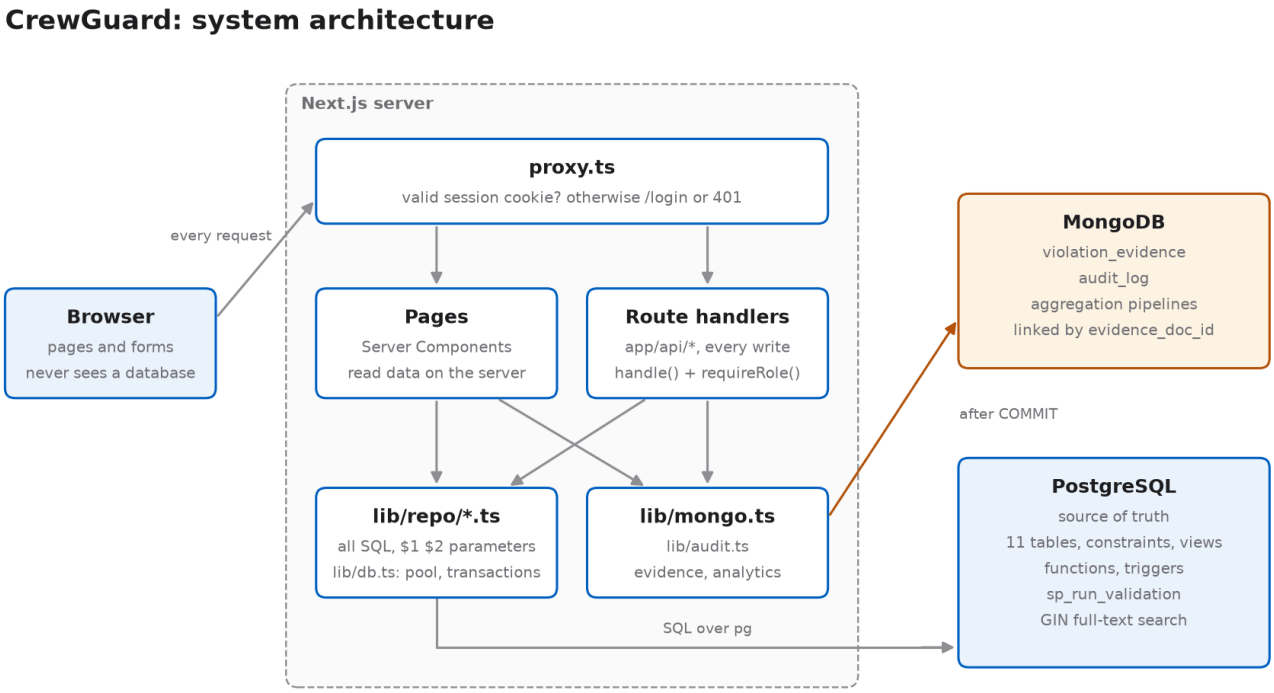


Figure 9.1: *System architecture*

A request travels through the following layers. As an example, when the scheduler assigns crew to a flight:

- **Form:** components/assign-crew-form.tsx posts the selected crew and duty times as JSON to /api/flights/[id]/duties.
- **Session check:** proxy.ts runs before every request and rejects anyone without a valid signed session cookie.
- **Route handler:** app/api/flights/[id]/duties/route.ts calls requireRole("SCHEDULER"), so an auditor receives HTTP 403.

- **SQL:** lib/repo/flights.ts runs one INSERT per crew member inside withTransaction from lib/db.ts (BEGIN, the INSERTs, COMMIT or ROLLBACK), so the assignment is all or nothing.
- **Database:** PostgreSQL fires trg_validate_duty and checks the CHECK, UNIQUE, EXCLUDE and FOREIGN KEY constraints.
- **Error mapping:** if a constraint fails, lib/api.ts maps its name to a readable message, and the page shows the message together with the name of the constraint that rejected the row.

9.2 Connecting the backend to the databases

lib/db.ts creates one PostgreSQL connection pool for the server process with the node-postgres driver. The connection string comes from the environment, never from code. Every query passes its values separately as $1, $2, ... so user input can never be executed as SQL, which prevents SQL injection. withTransaction runs several statements on a single pooled connection between BEGIN and COMMIT, and rolls back if any of them fails.

```
export const pool =
  globalForPg.pgPool ??
  new Pool({
    connectionString: process.env.DATABASE_URL, // from .env.local, never hard-coded
    options: "-c timezone=UTC", // the roster is in UTC, so date casts are too
    max: 10,
  });

if (process.env.NODE_ENV
...
export async function query<T extends QueryResultRow>(text: string, params: unknown[] = []):
Promise<T[]> {
  const { rows } = await pool.query<T>(text, params);
  return rows;
}

export async function withTransaction<T>(fn: (client: PoolClient) => Promise<T>): Promise<T> {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    const result = await fn(client);
    await client.query("COMMIT");
    return result;
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release(); // give the connection back to the pool
  }
}
```

lib/mongo.ts does the same for MongoDB: one shared client, the URI from the environment, a two-second server-selection timeout and idempotent index creation on start-up. PostgreSQL is the source of truth: the evidence is written to MongoDB only after the validation transaction commits, and if MongoDB is unavailable the violations still exist and the page reports that the evidence is unavailable.

9.3 MongoDB: evidence, activity log and aggregation

The violation_evidence collection holds one document per violation, whose evidence part has a different shape for each rule (a rest gap, a seven-day breakdown, certificate details), which a flexible document stores more cleanly than many nullable columns. Its ObjectId is saved in violation.evidence_doc_id, the link between the two databases. The audit_log collection is append-only and records every login, roster change, validation run and review. The dashboard's charts are aggregation pipelines ($match, $group, $sort, $limit, $avg).

Data	Store	Reason
Airports, aircraft, users, crew, ratings, flights, duties, rules, runs, violations	PostgreSQL	Related data that needs keys, constraints, transactions and temporal SQL
violation_evidence	MongoDB	Shape differs per rule; a frozen snapshot taken at run time
audit_log	MongoDB	Append-only, write-heavy, different payload per action, never joined

Table 9.1: *Choice of database for each kind of data*

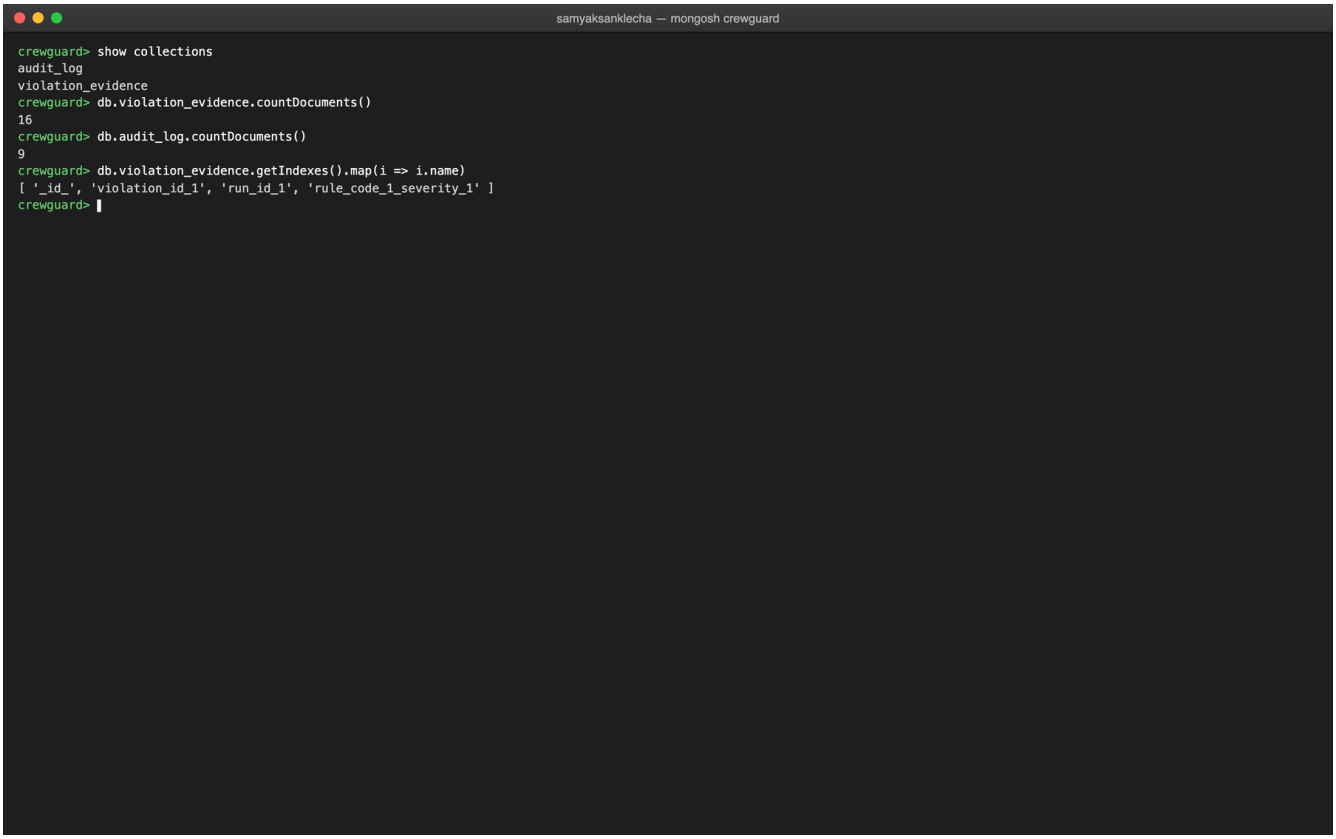


Figure 9.2: *MongoDB collections, document counts and indexes*

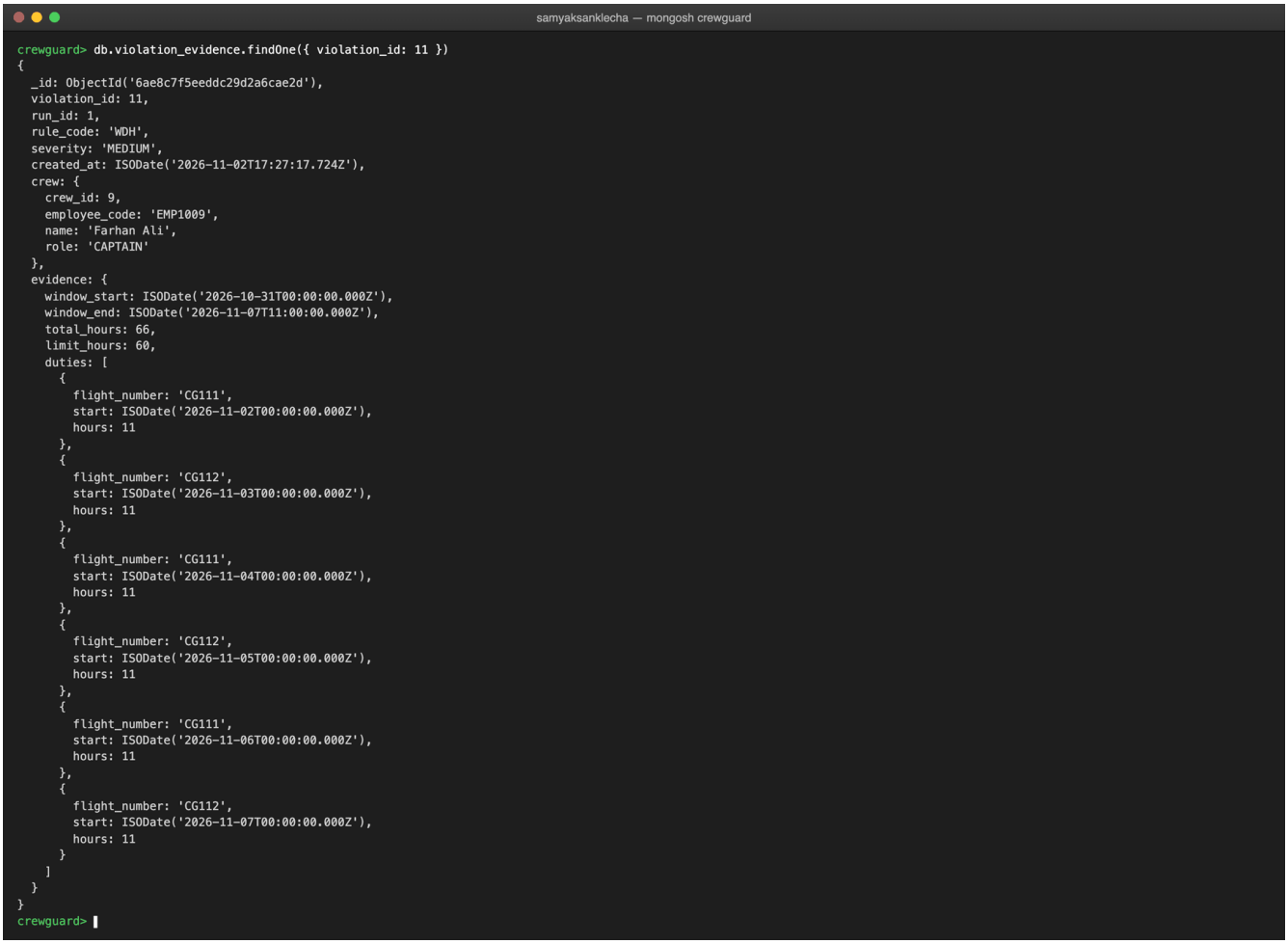


Figure 9.3: *A WDH evidence document (violation 11, Farhan Ali)*

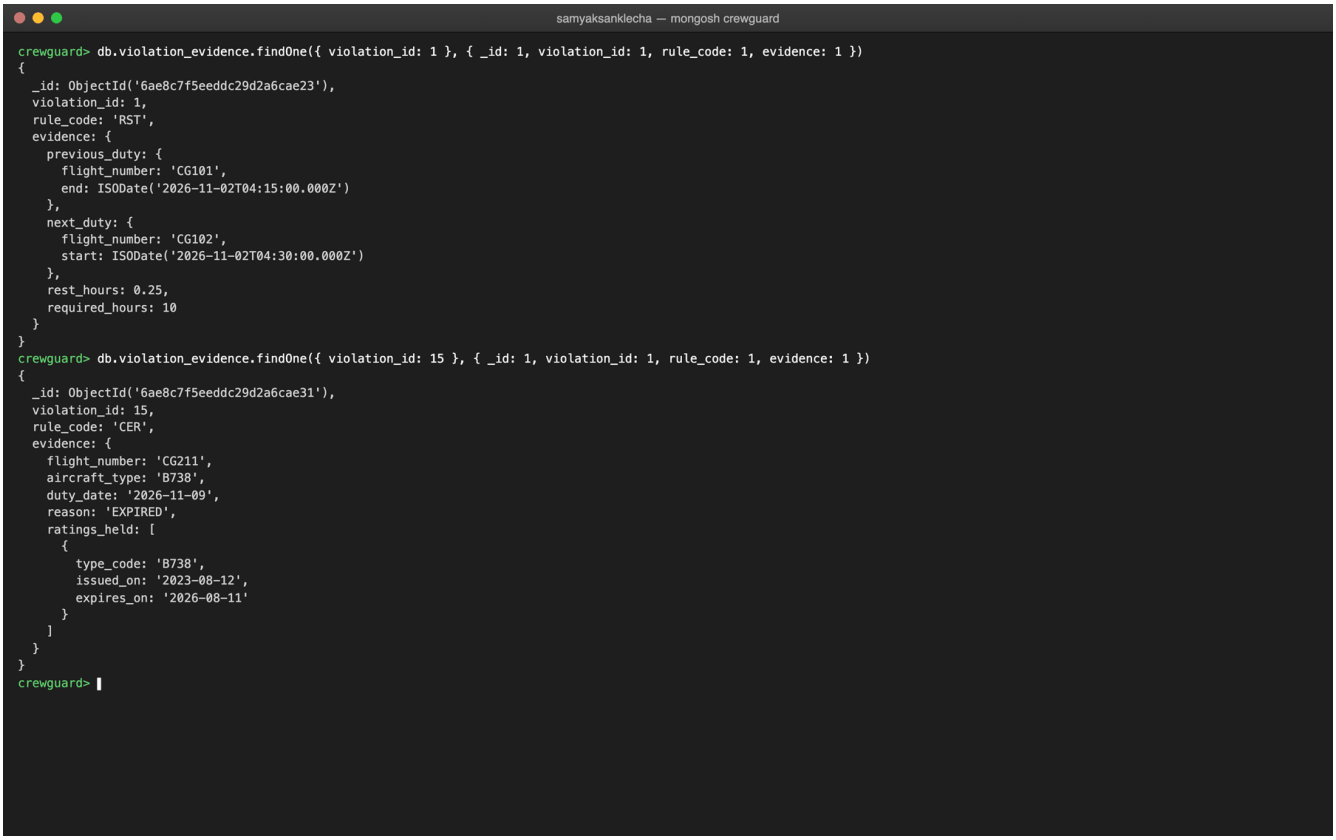


Figure 9.4: *Evidence documents for an RST and a CER violation: different shapes per rule*

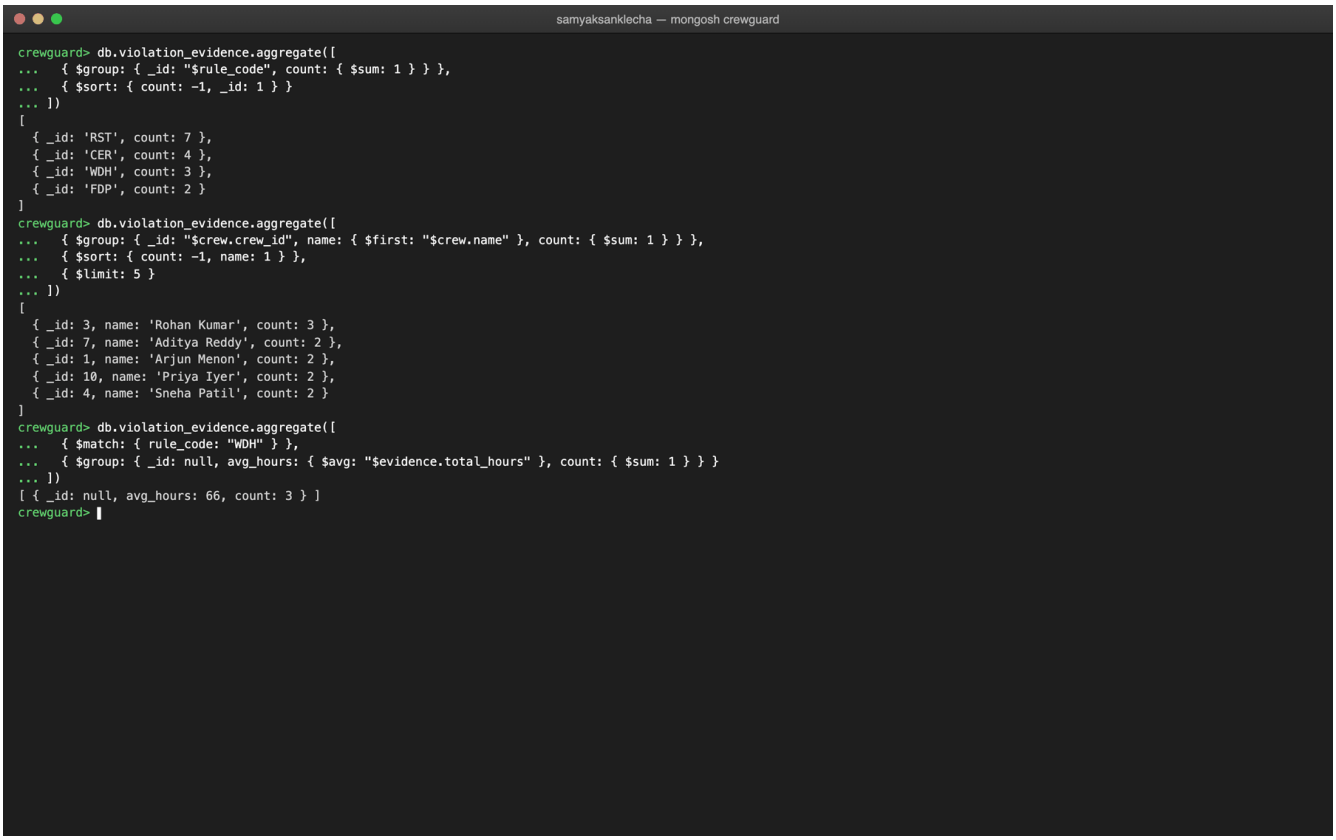


Figure 9.5: *Aggregation pipelines behind the dashboard*

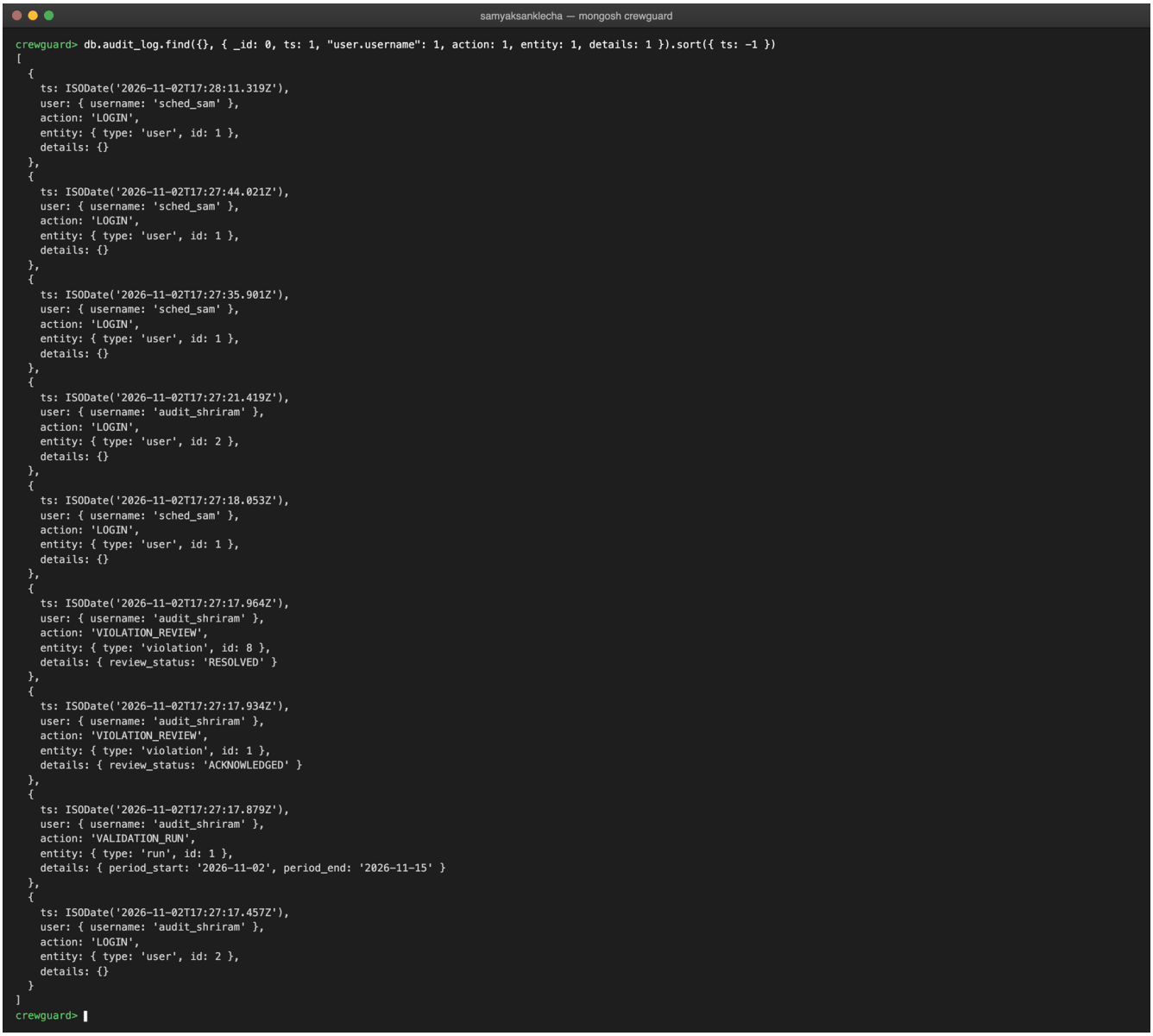


Figure 9.6: *The activity log in audit_log*

9.4 Application screens

Screens are shown for both roles. The scheduler (sched_sam) manages the roster; the auditor (audit_shriram) runs validations and reviews violations. Flight and duty times are in UTC, as airlines use; times of events such as logins, runs and reviews are shown in IST.

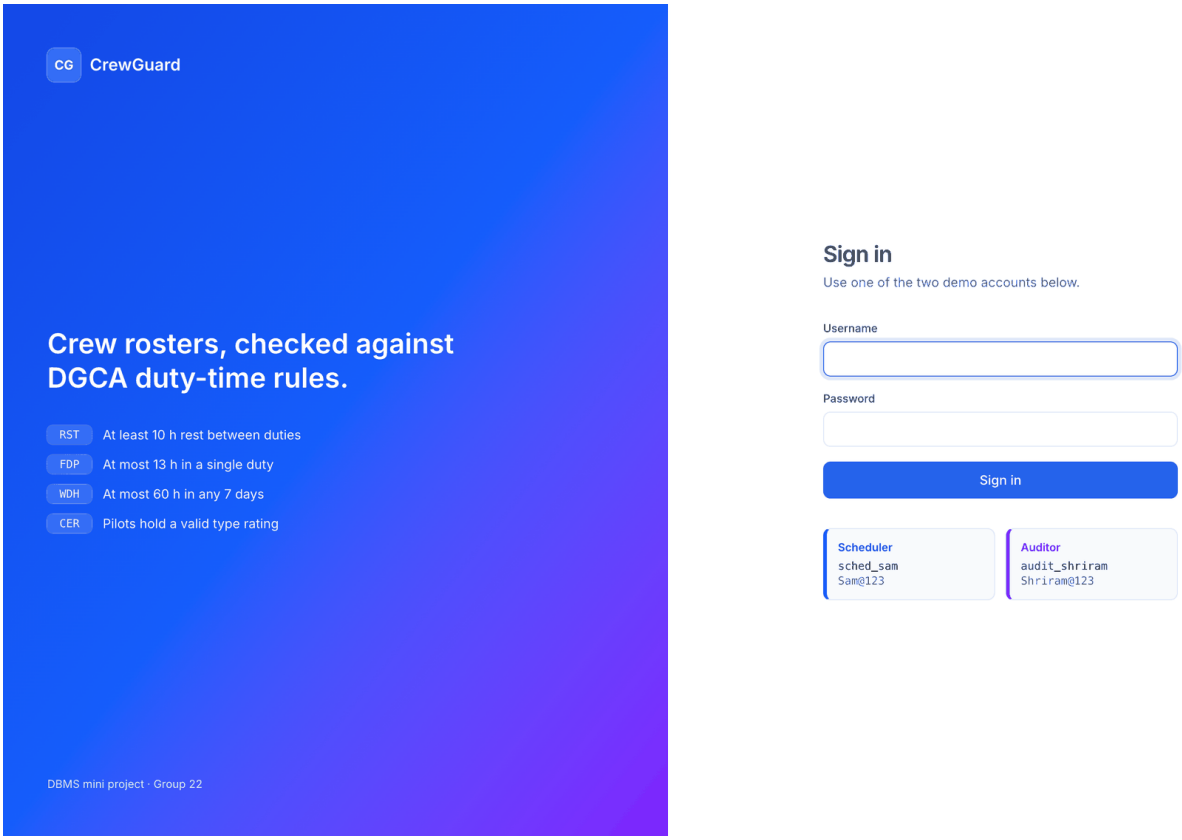


Figure 9.7: *Sign-in page with the two demo accounts*

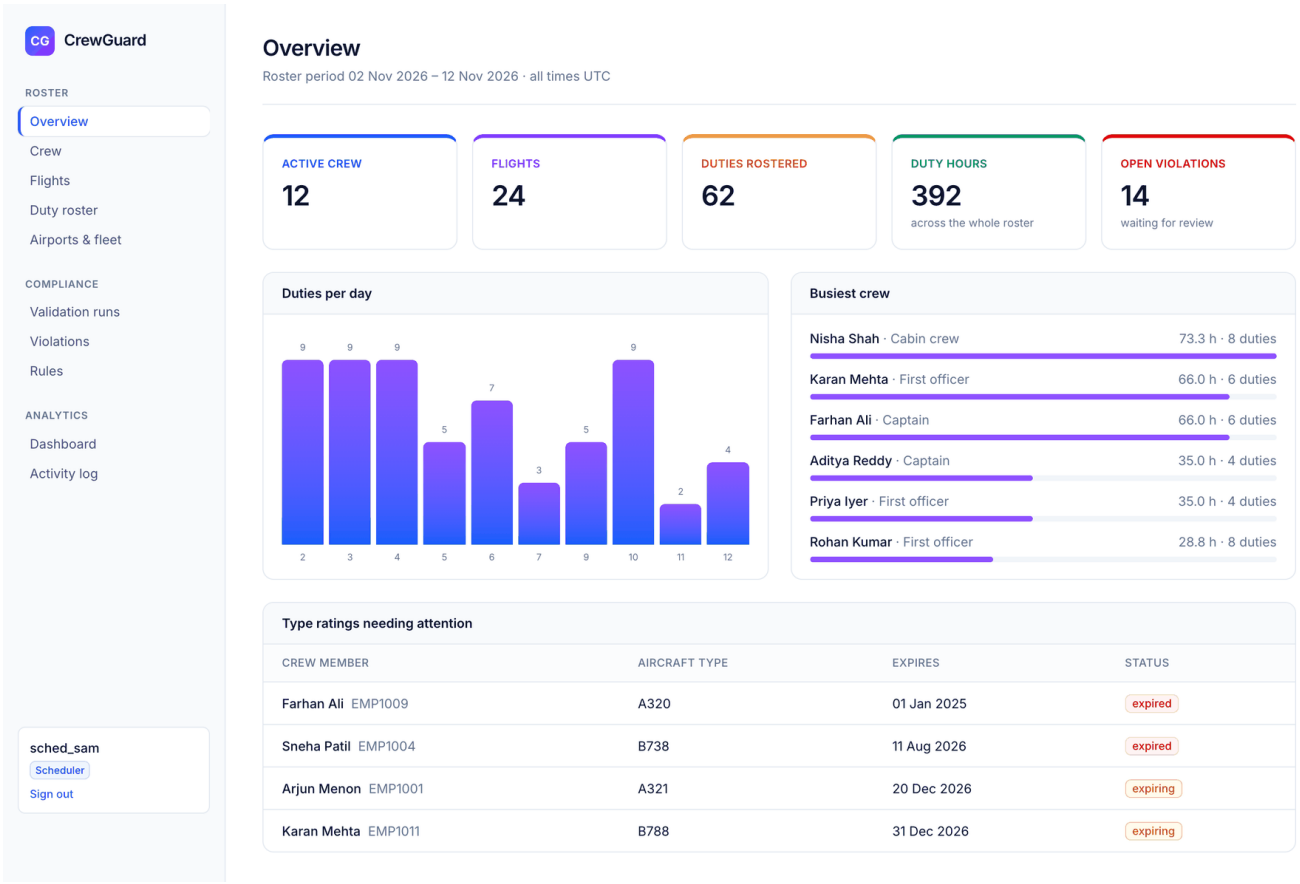


Figure 9.8: *Overview (scheduler): roster statistics, duties per day, busiest crew and ratings needing attention*

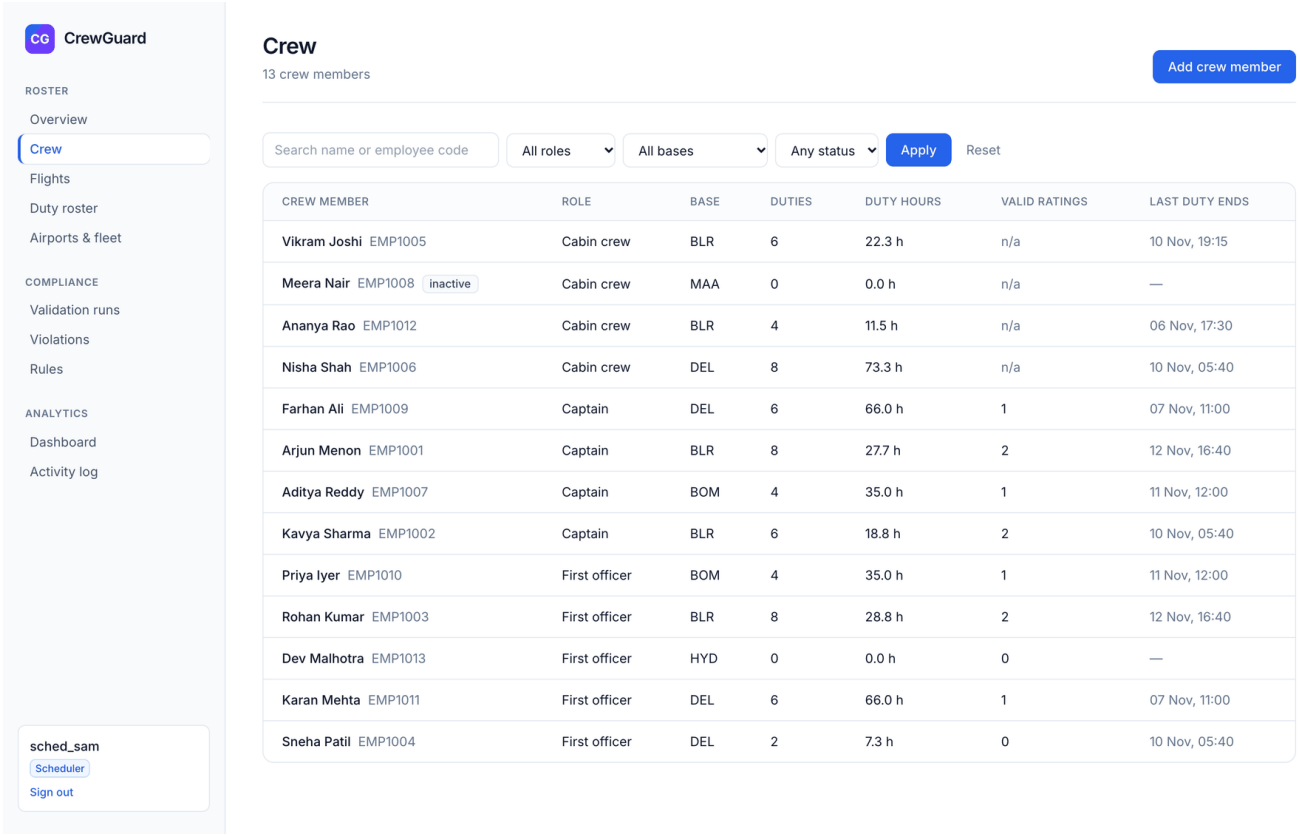


Figure 9.9: *Crew list with search and filters, read from v_crew_summary*

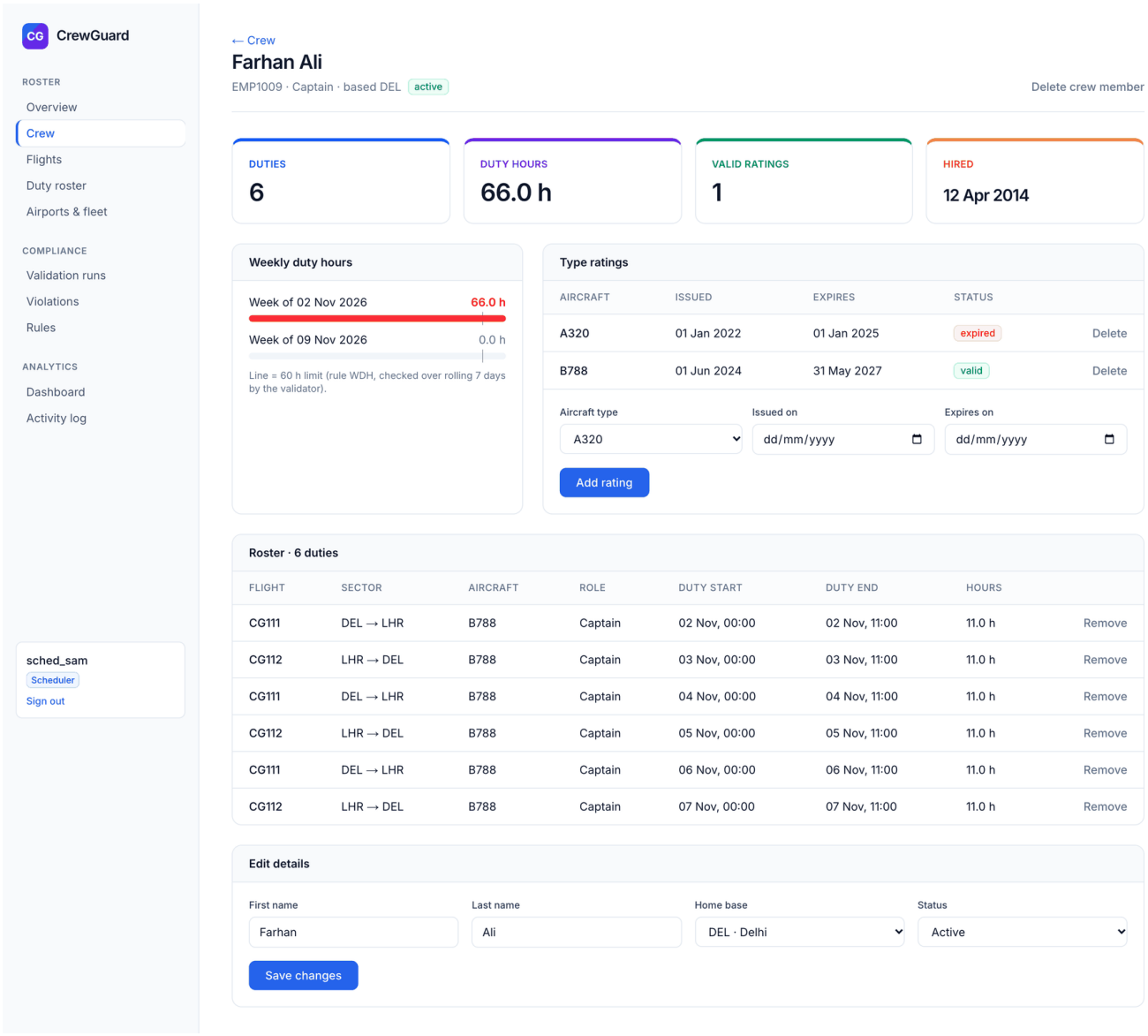


Figure 9.10: *Crew detail: weekly duty hours against the 60-hour limit, type ratings and roster*

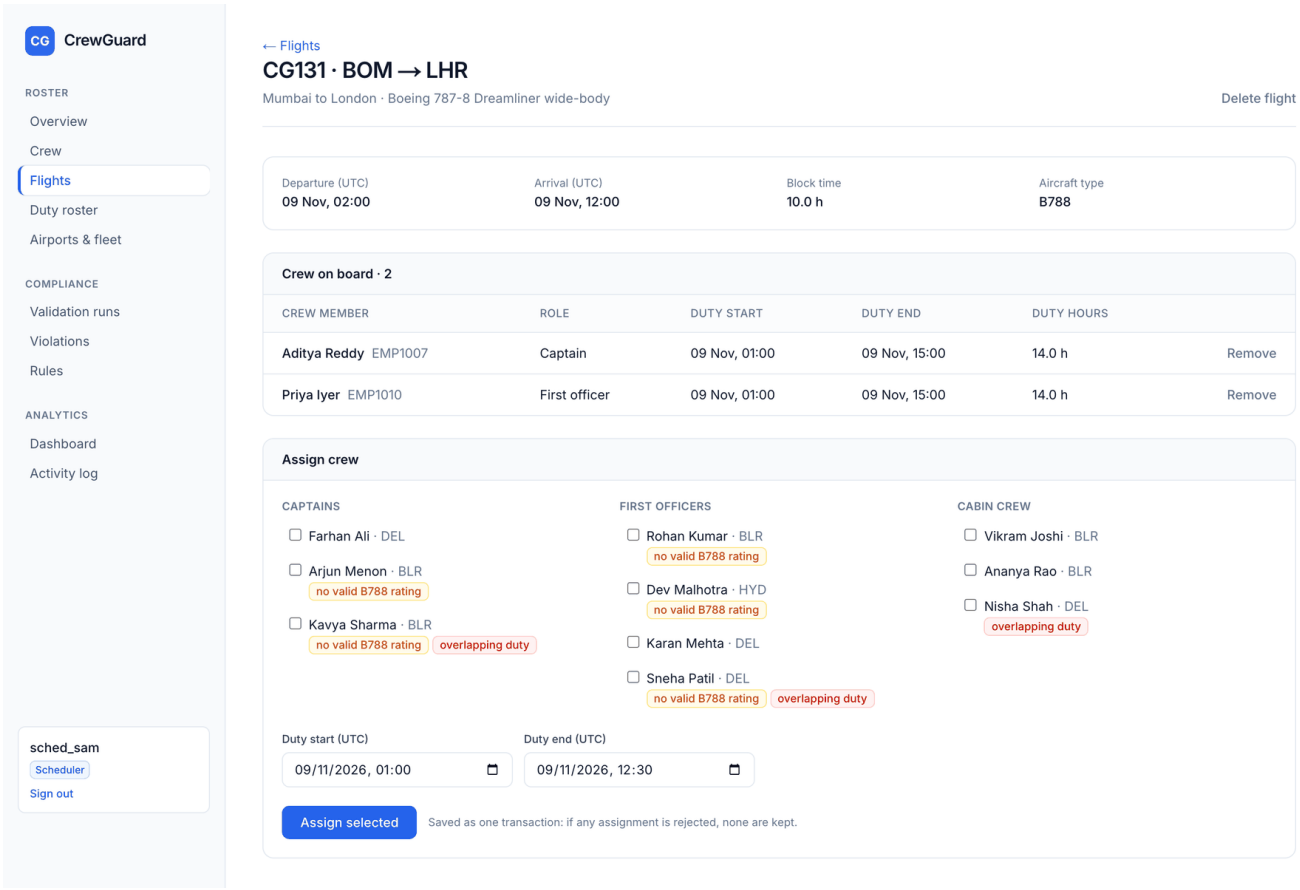


Figure 9.11: *Flight detail with the assign-crew form and warnings computed with EXISTS and range overlap*

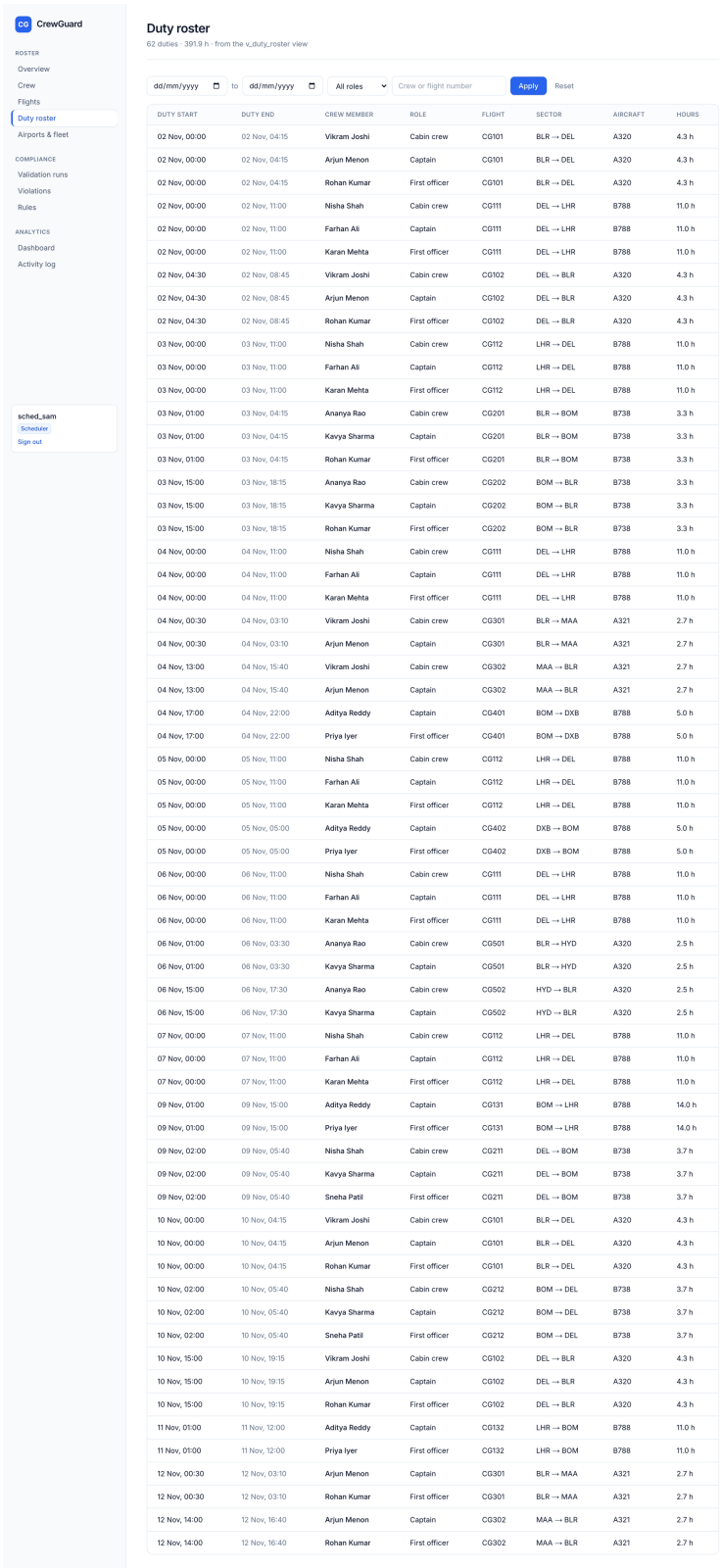


Figure 9.12: *Duty roster read from v_duty_roster*

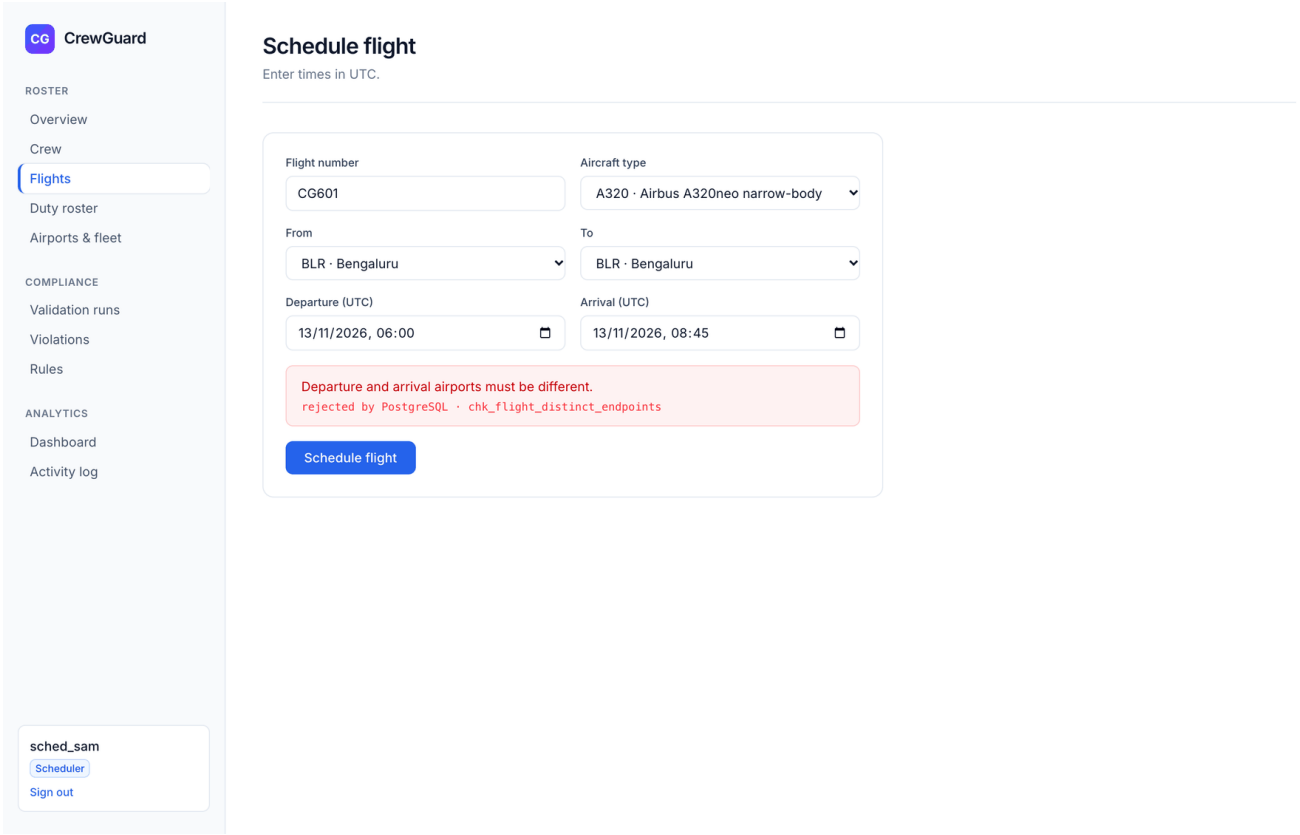


Figure 9.13: *Database rejecting a flight with the same departure and arrival airport (CHECK)*

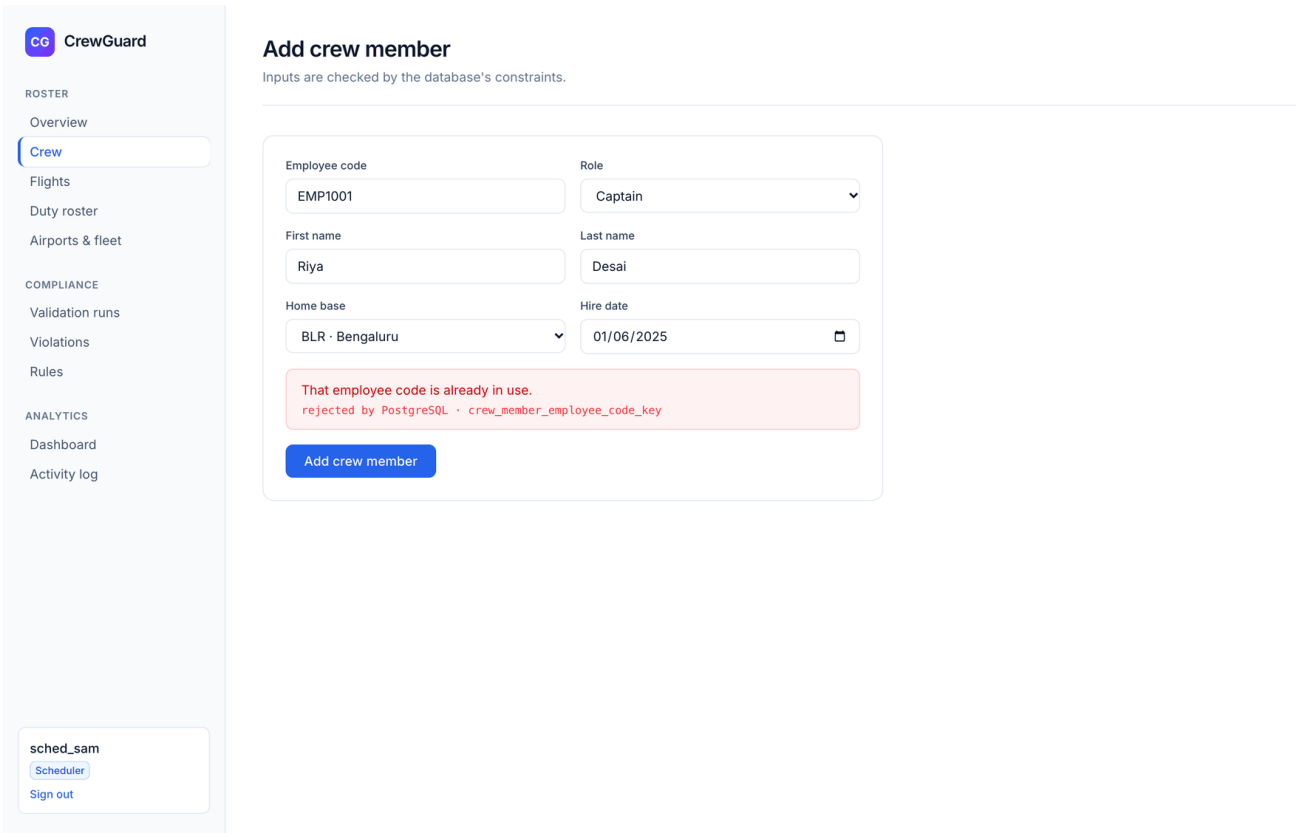


Figure 9.14: *Database rejecting a duplicate employee code (UNIQUE)*

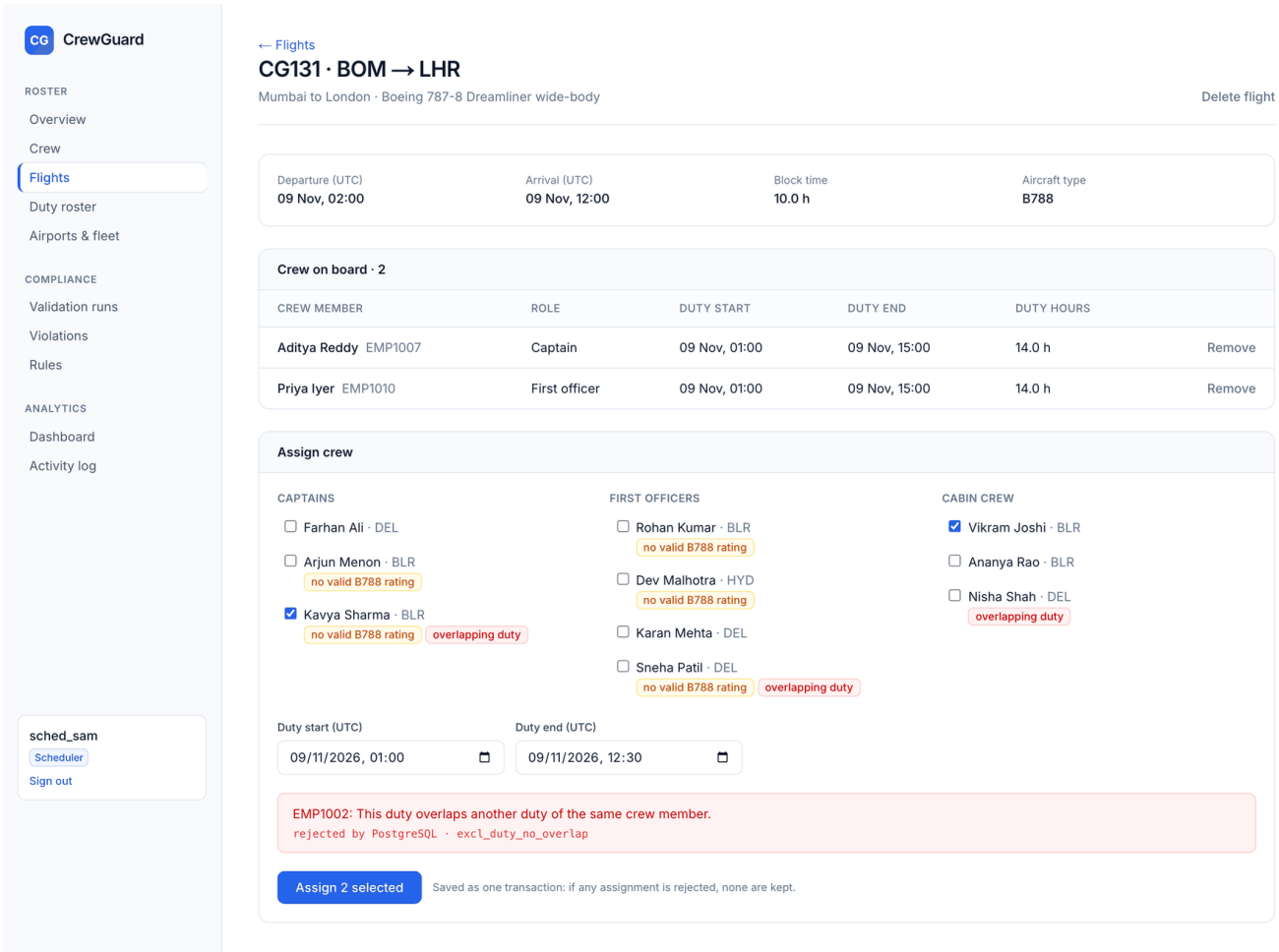


Figure 9.15: *Assigning two crew in one transaction: the overlap is rejected by EXCLUDE and neither duty is saved*

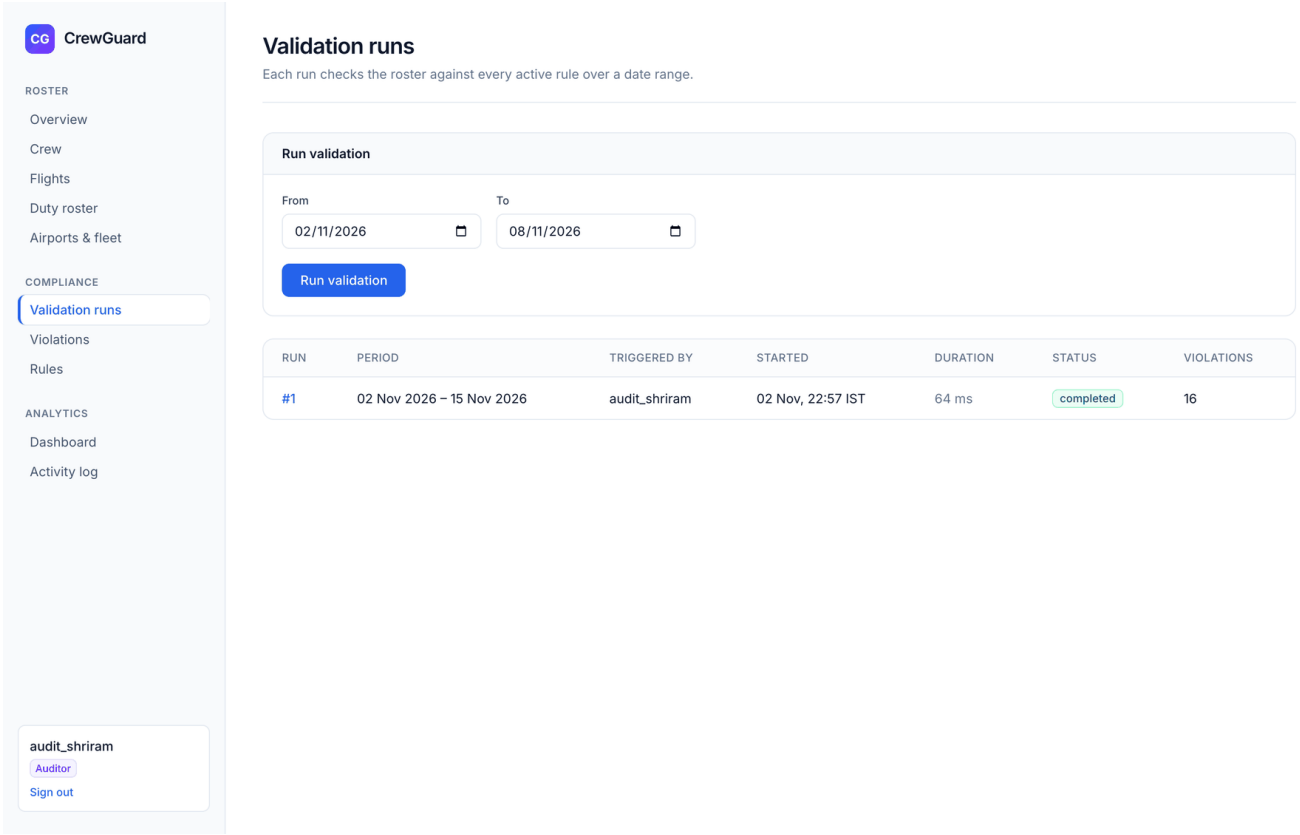


Figure 9.16: *Validation runs (auditor) with the run form*

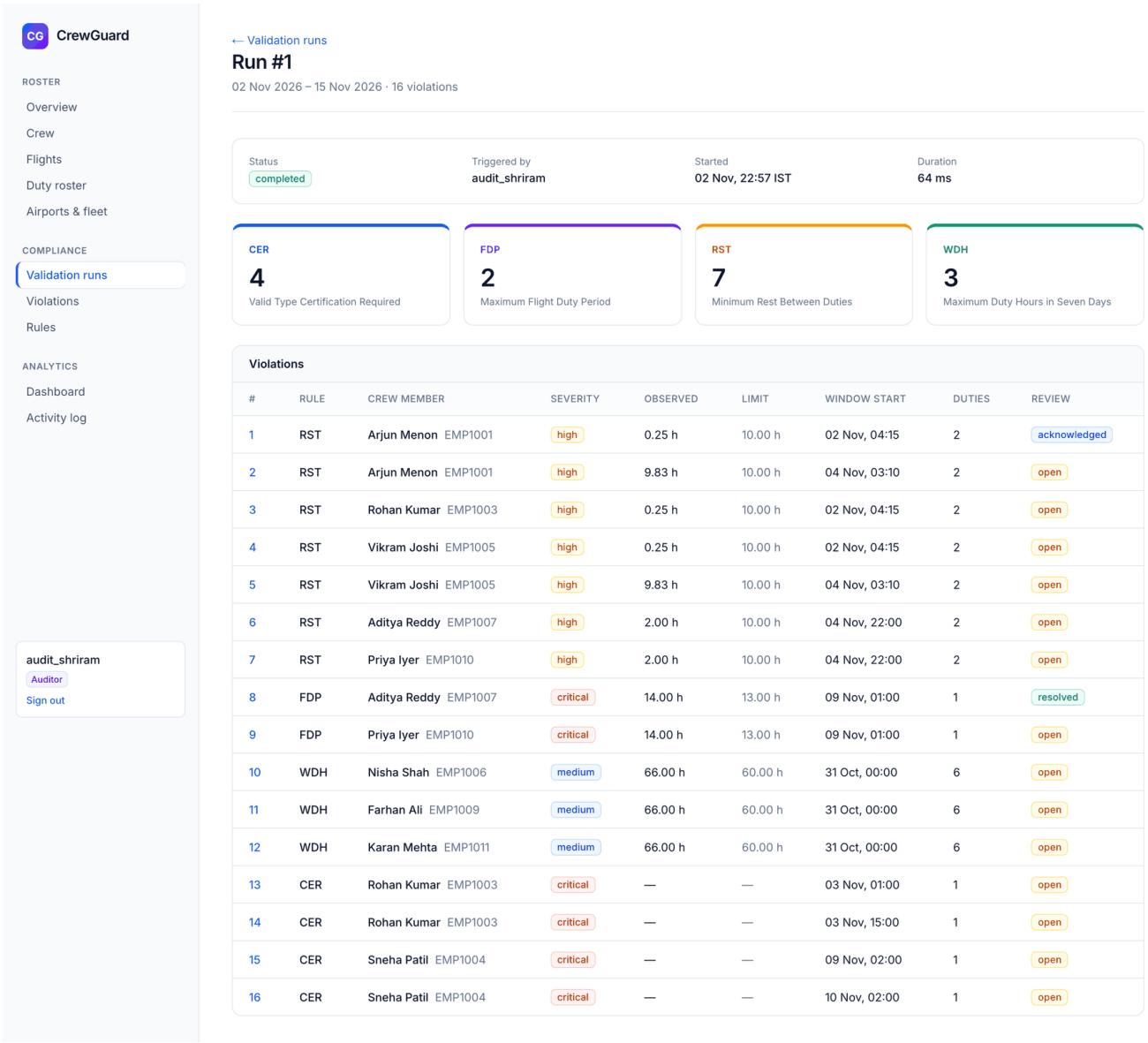


Figure 9.17: *Result of one run: violations per rule*

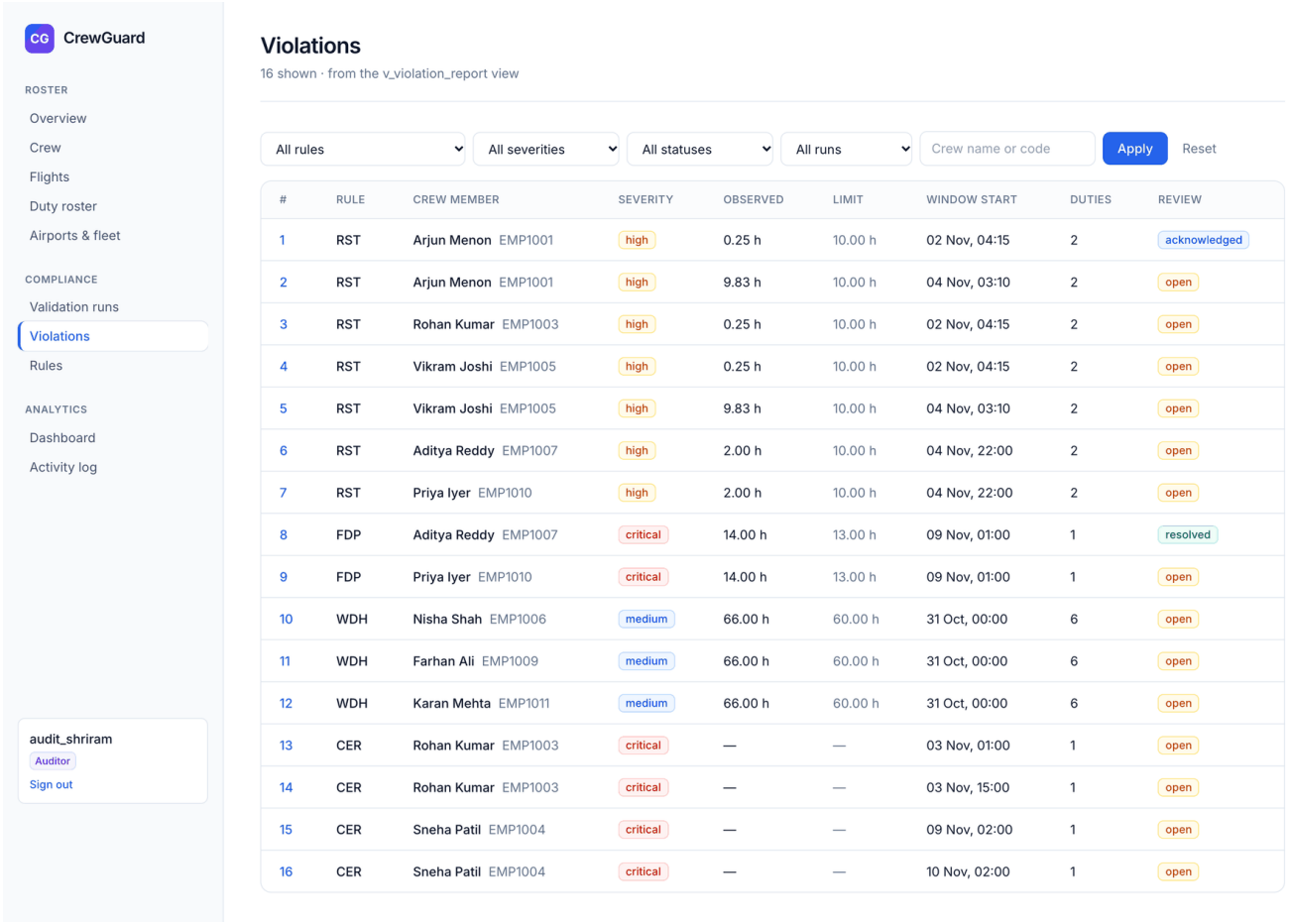


Figure 9.18: *Violations list from v_violation_report with filters*

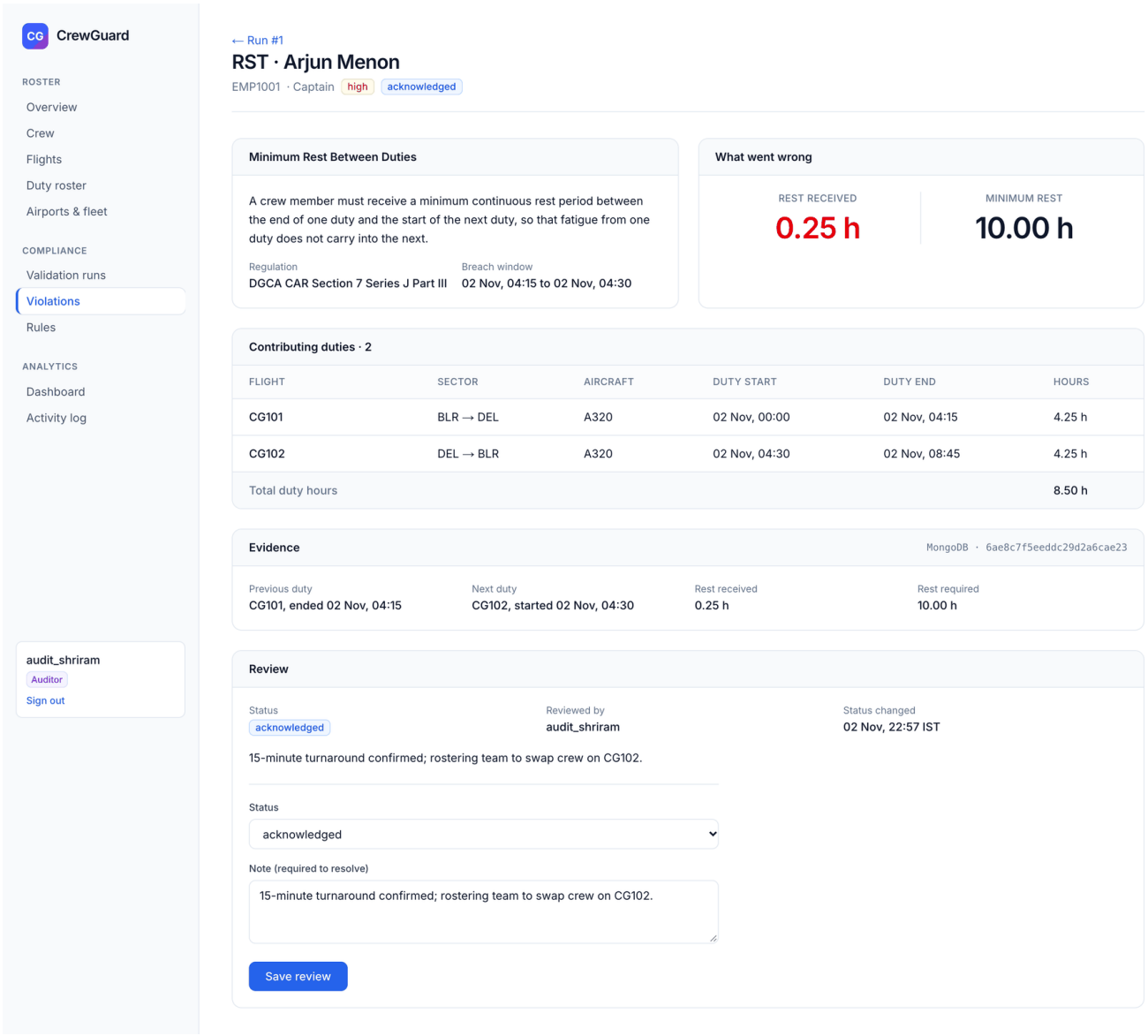


Figure 9.19: *Explanation page for an RST violation: rule, observed value, causing duties, MongoDB evidence and review*

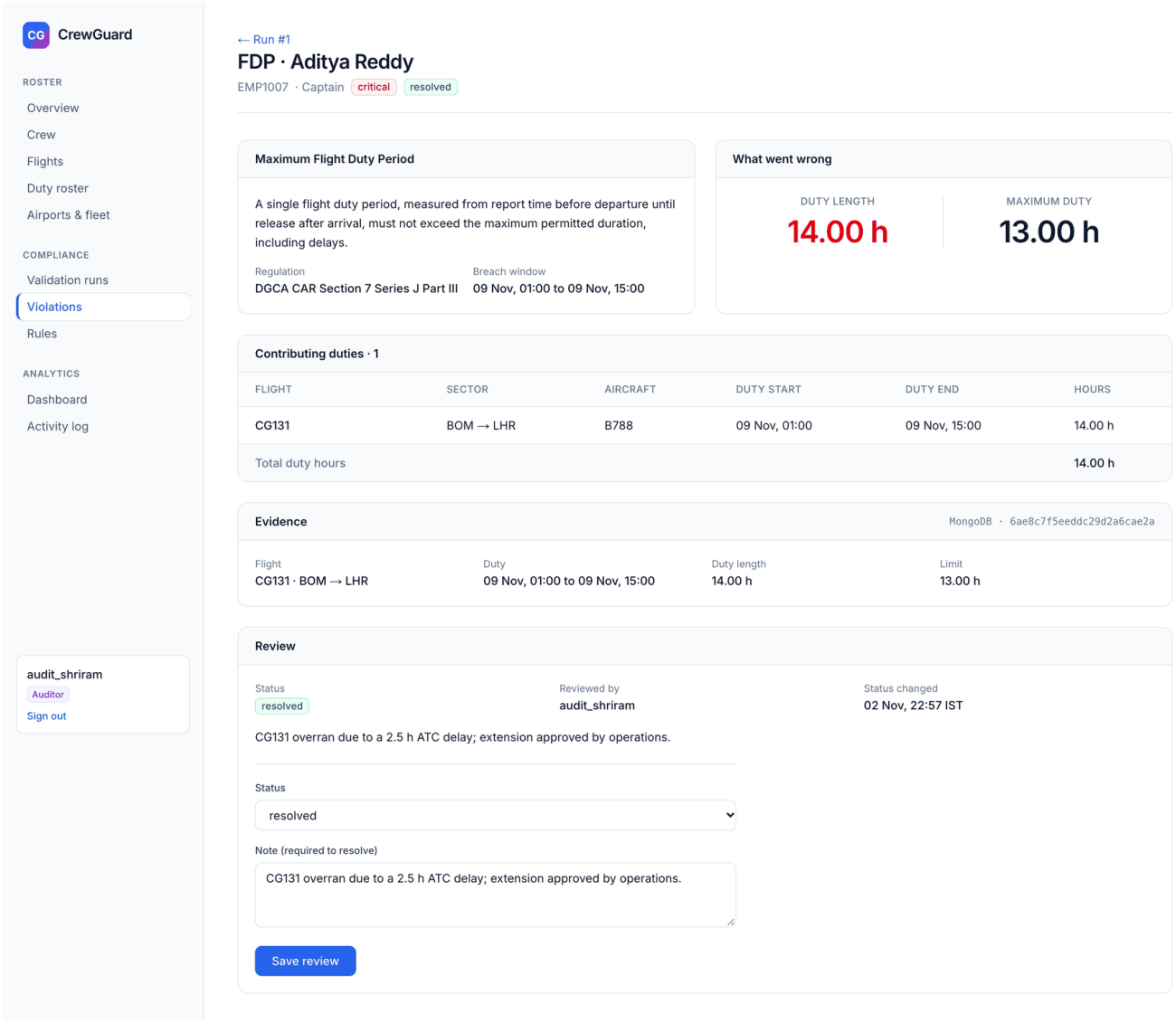


Figure 9.20: *Explanation page for an FDP violation (resolved with a note)*

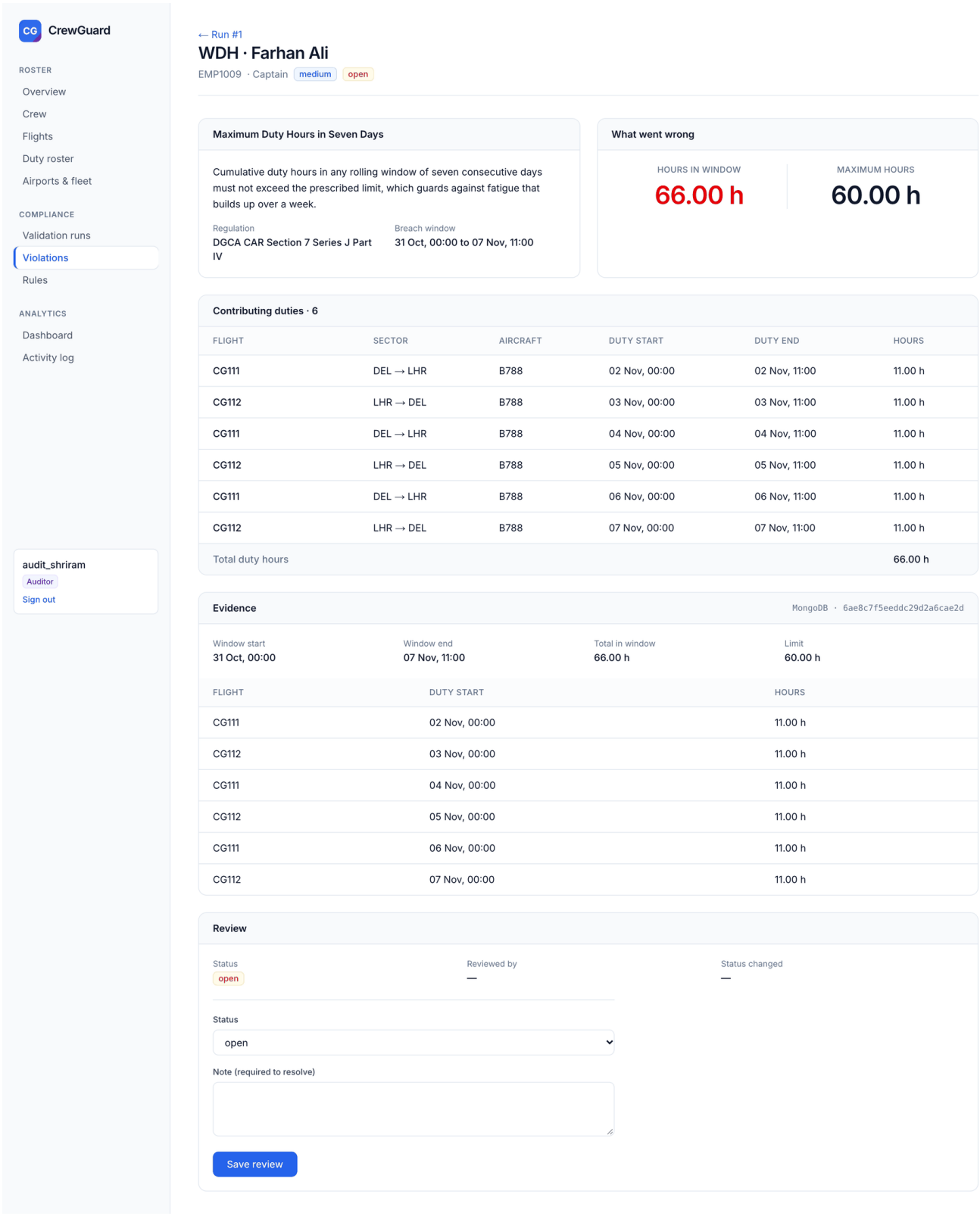


Figure 9.21: *Explanation page for a WDH violation: six duties adding up to 66 hours*

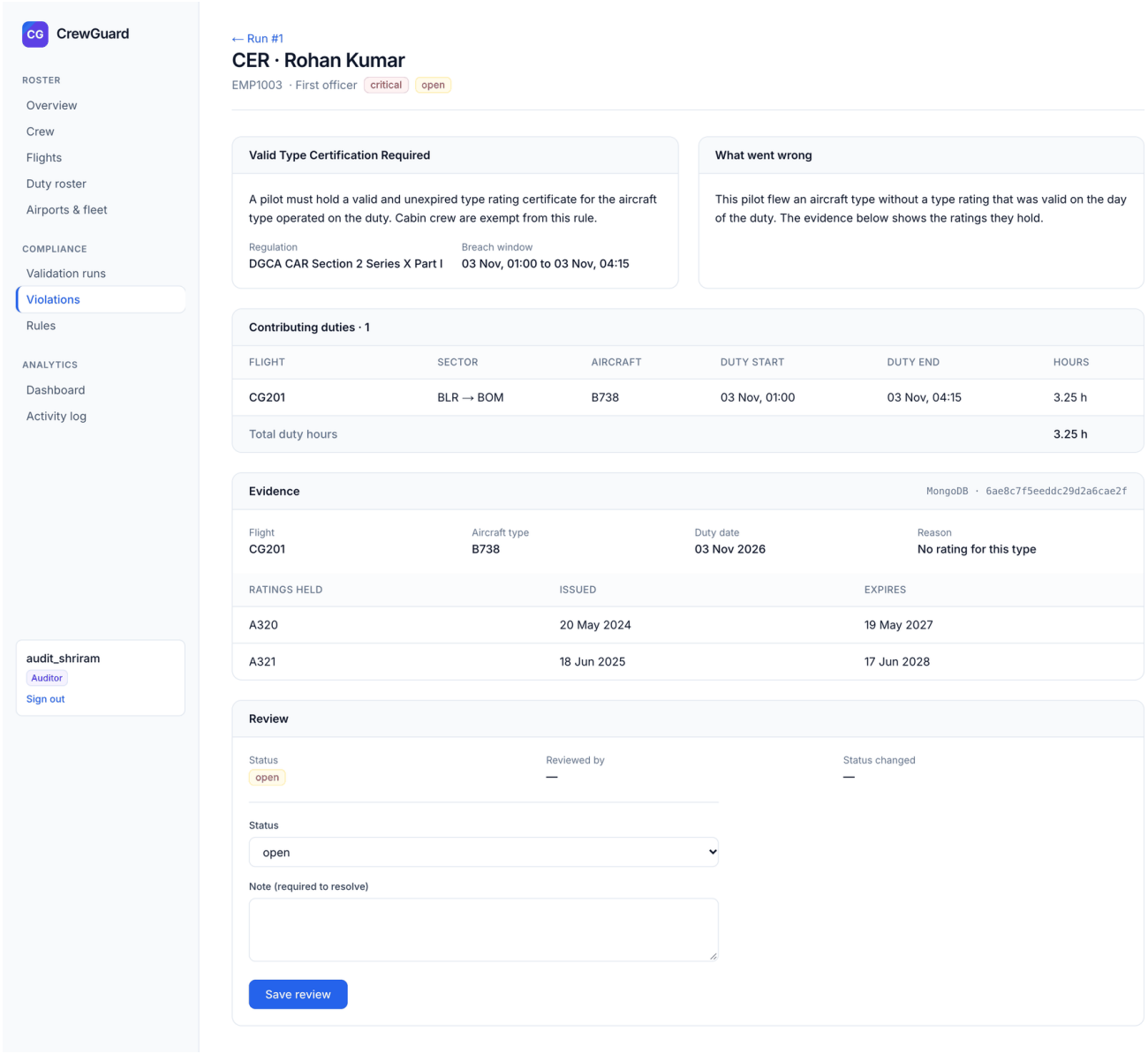


Figure 9.22: *Explanation page for a CER violation: no rating for the aircraft type*

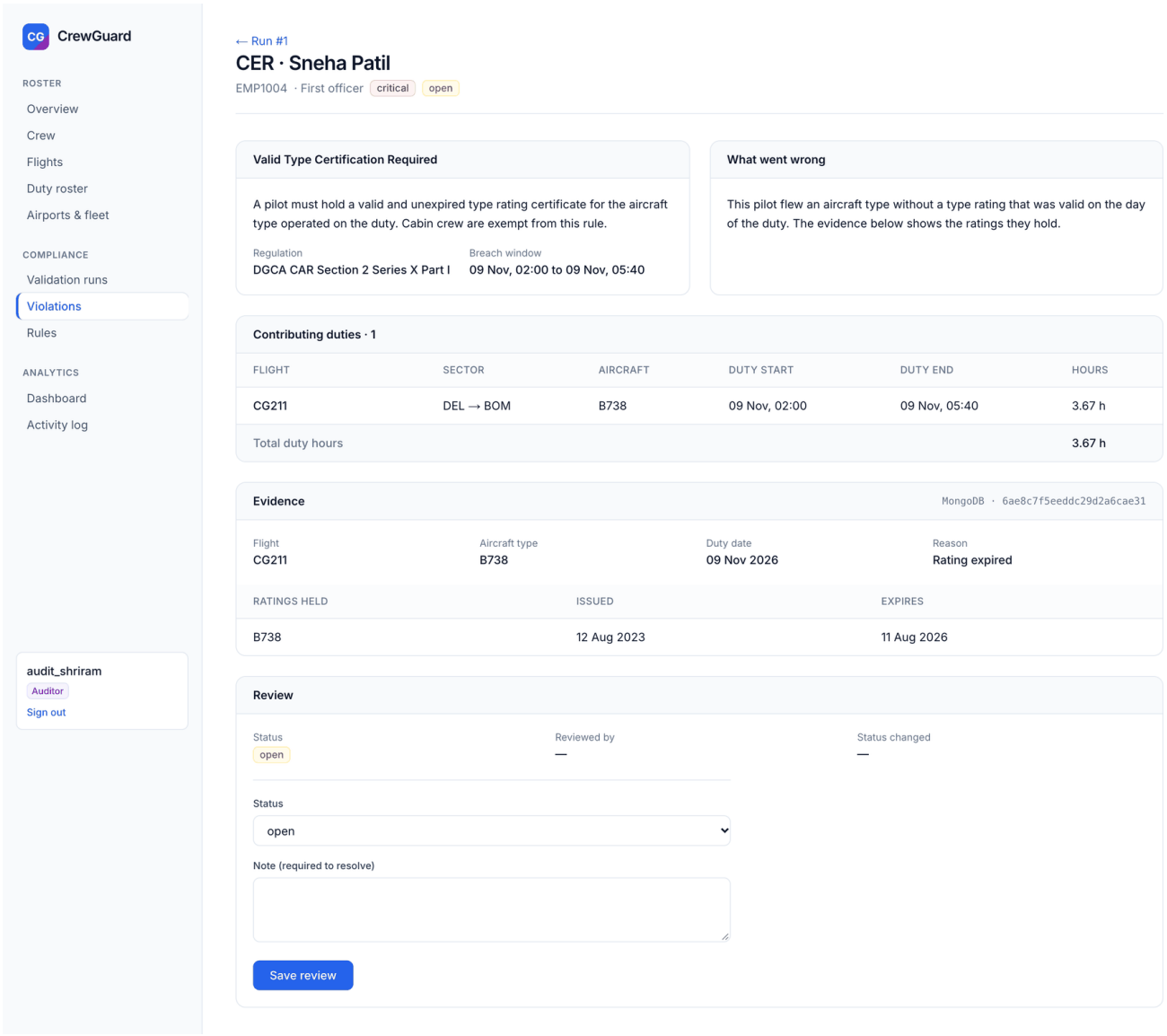


Figure 9.23: *Explanation page for a CER violation: rating expired*

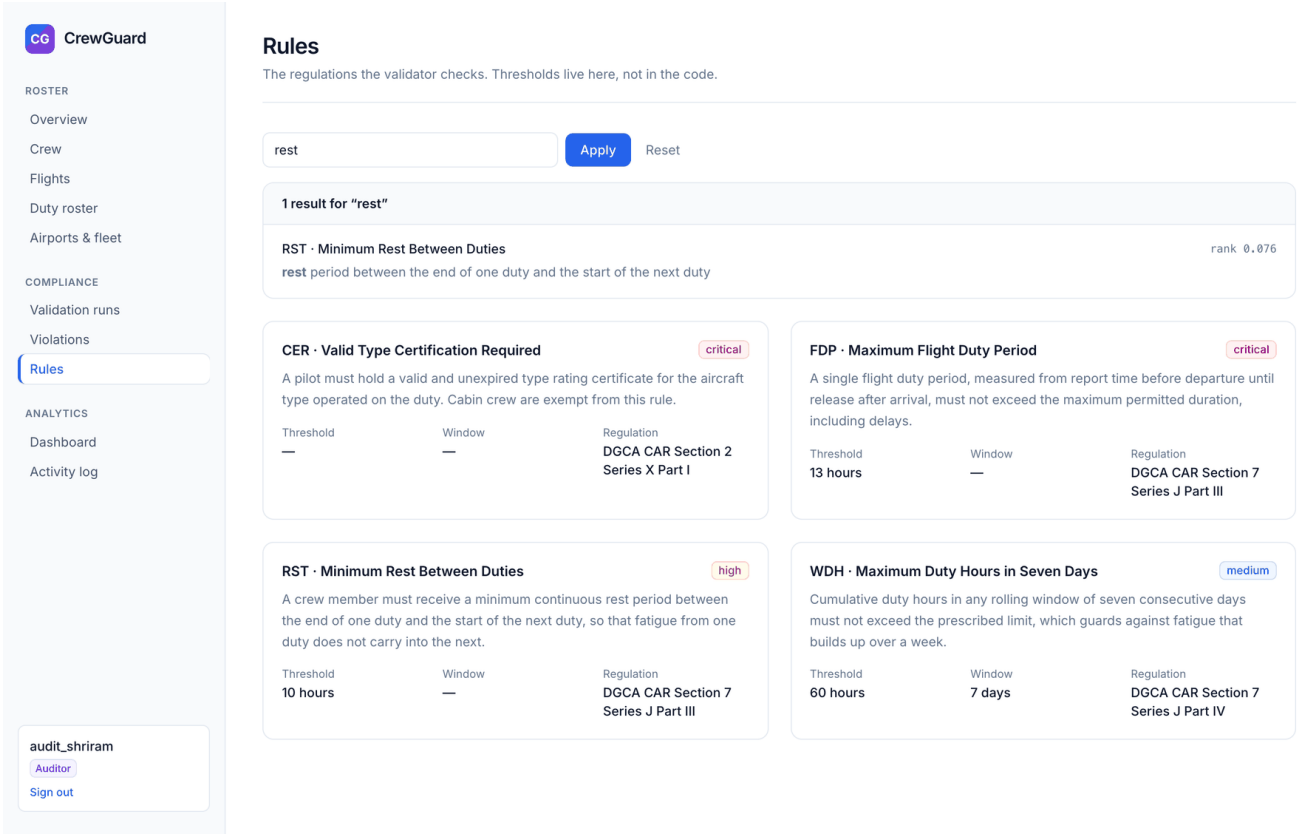


Figure 9.24: *Rule catalogue with full-text search*

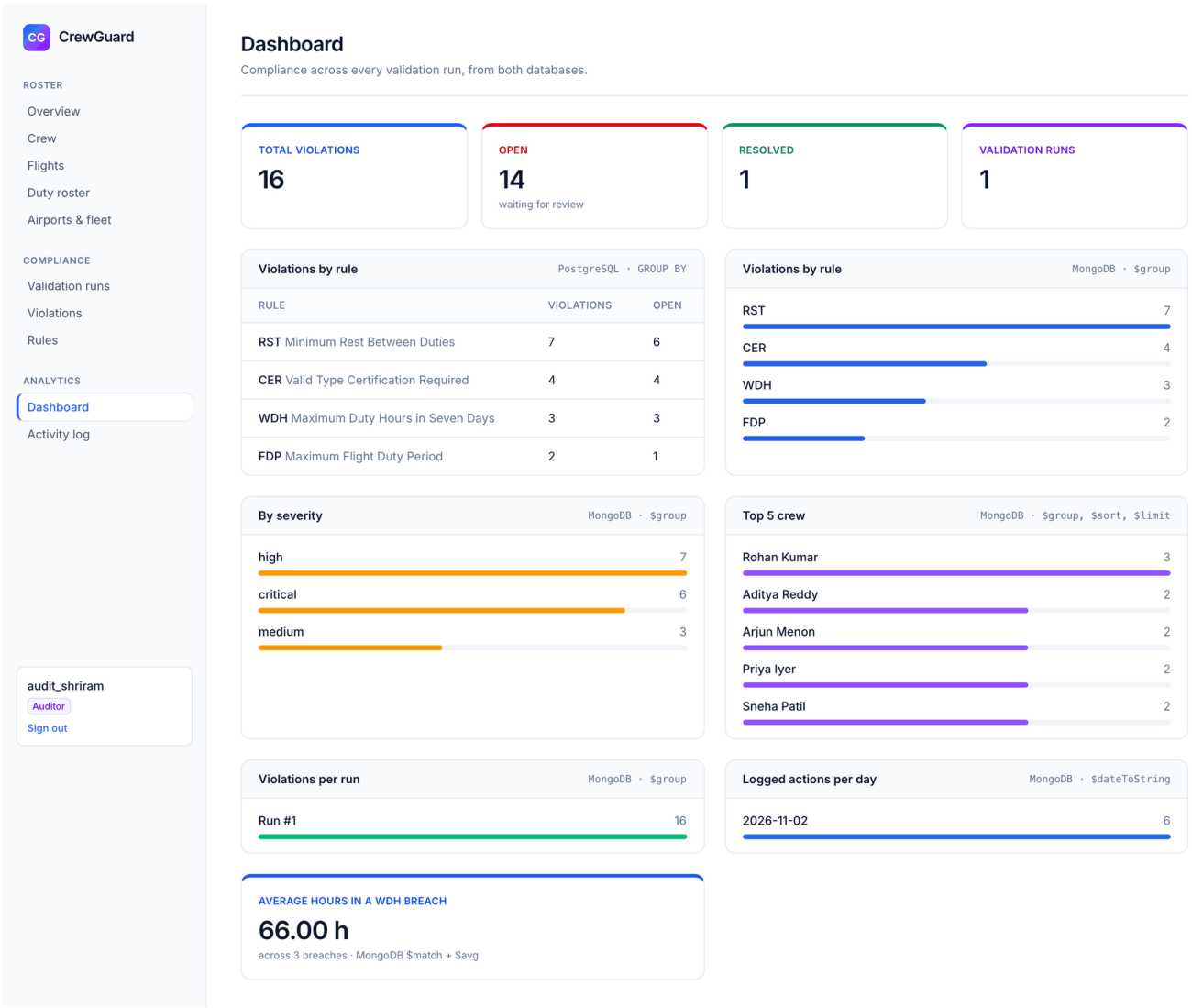


Figure 9.25: *Dashboard: PostgreSQL GROUP BY next to MongoDB aggregation pipelines*

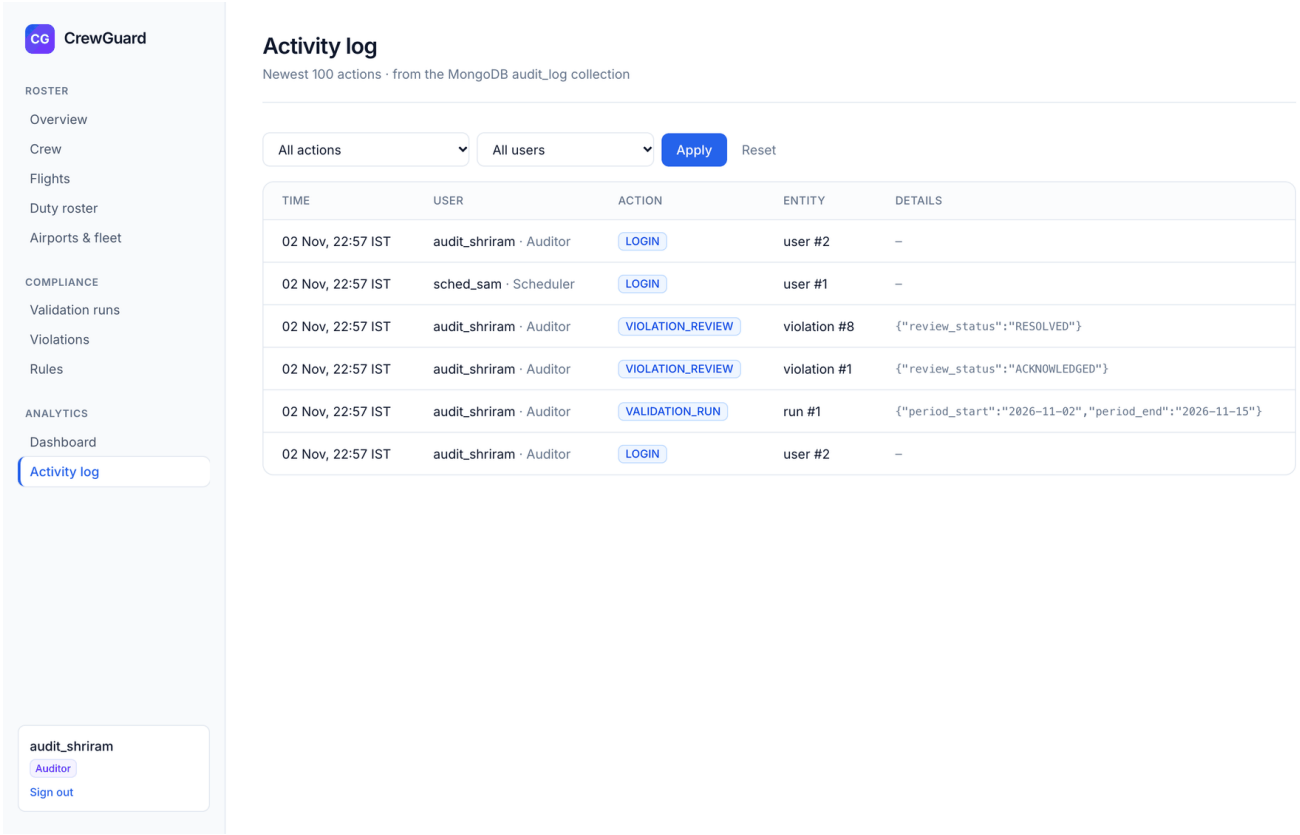


Figure 9.26: *Activity log read from the MongoDB audit_log collection*

9.5 Testing

The test suite (npm test) runs 46 tests with the Node.js test runner against the real PostgreSQL and MongoDB databases. Each database test runs inside a transaction that is rolled back, so the sample data is never changed. The tests cover every constraint, cascade and restrict rule, the duty trigger, transaction rollback, the functions and views, the validator's expected results (16 violations for the full period, 12 for week 1 and 4 for week 2), the explanation links, the compliance triggers, the review CHECK constraints, full-text search and the MongoDB collections and pipelines. All 46 pass.

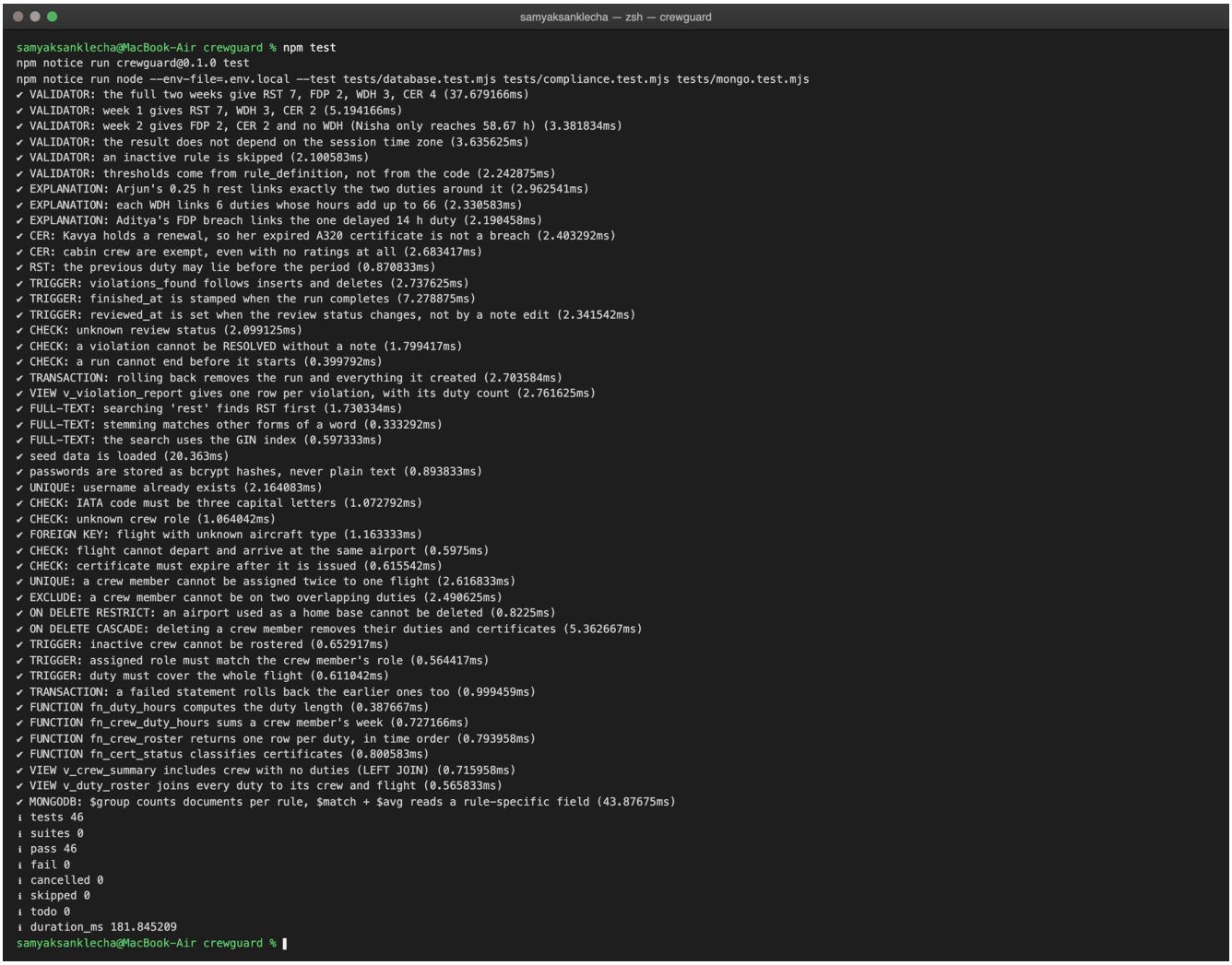


Figure 9.27: *Output of npm test: 46 tests, 46 passed*

10. CONCLUSION AND FUTURE ENHANCEMENTS

CrewGuard shows how much of an application's correctness can live inside the database. Impossible rosters are rejected by constraints, an exclusion constraint and a trigger; regulatory breaches in otherwise valid data are found by a single stored procedure using window functions and anti-joins; and every breach is explained through a many-to-many link to the duties that caused it. Transactions keep multi-row writes atomic, views keep join logic in one place, and MongoDB complements PostgreSQL where the data is document-shaped. On the sample roster the validator finds exactly the sixteen planted violations, and the full test suite passes.

Possible enhancements:

- More DGCA rules, such as cumulative duty limits over 28 days and a weekly day off, added as new rows in rule_definition plus a section in the procedure.
- Time-zone-aware rules such as night duty and acclimatisation, using each airport's stored time zone.
- Suggesting a legal replacement crew member when a violation is found, using the same EXISTS and range-overlap queries as the assign form.
- Semantic search over the full regulation text using vector embeddings.
- Scheduled automatic validation runs and notifications to the auditor.

Individual contributions

Area	Samyak Sanklecha (PES2UG24CS436)	Shriram Chandrasekar (PES2UG24CS495)
PostgreSQL	Schema with 11 tables and all constraints including EXCLUDE; indexes; functions, views and trg_validate_duty (02_roster.sql); seed data; queries	03_compliance.sql: review columns, three compliance triggers, v_violation_report, sp_run_validation, full-text search
MongoDB	–	violation_evidence and audit_log collections, aggregation pipelines
Application	Backend–database layer (pool, parameterised queries, transactions), authentication and roles, roster pages, UI theme	Compliance and analytics pages: runs, violations and explanation, rules, dashboard, activity log
Quality and documentation	Database constraint tests	Compliance and MongoDB tests, diagrams, demo script

Table 10.1: *Individual contributions*

REFERENCES

[1] PostgreSQL Global Development Group, PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

[2] PostgreSQL 16 Documentation, Exclusion Constraints and the btree_gist module. https://www.postgresql.org/docs/16/btree-gist.html

[3] PostgreSQL 16 Documentation, Window Functions. https://www.postgresql.org/docs/16/tutorial-window.html

[4] PostgreSQL 16 Documentation, Full Text Search. https://www.postgresql.org/docs/16/textsearch.html

[5] MongoDB, Inc., MongoDB Manual: Aggregation Pipeline. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

[6] node-postgres documentation. https://node-postgres.com/

[7] Vercel, Next.js Documentation. https://nextjs.org/docs

[8] Directorate General of Civil Aviation, Civil Aviation Requirements Section 7 Series J Part III: Flight Duty Time Limitations and Rest Requirements. https://www.dgca.gov.in/

[9] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

GitHub Link: https://github.com/samyak-1021/crewguard

APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS

The following terms, acronyms and abbreviations are used in this report.

Term	Meaning
CAR	Civil Aviation Requirements, the regulations issued by the DGCA
CER	Rule code: valid type certification required
DDL / DML	Data Definition Language / Data Manipulation Language
DGCA	Directorate General of Civil Aviation, India's aviation regulator
Duty	The working period for one flight, from report time before departure to release after arrival
ER	Entity-Relationship
FDP	Flight Duty Period; rule code for the maximum length of one duty
FK / PK	Foreign key / primary key
FTS	Full-text search
GiST / GIN	Generalised Search Tree / Generalised Inverted Index, PostgreSQL index types
IATA	International Air Transport Association; its three-letter airport codes are used as keys
JWT	JSON Web Token, used for the signed session cookie
ObjectId	MongoDB's 12-byte document identifier, stored as 24 hexadecimal characters
Roster	The set of duties: which crew member works which flight, and when
RST	Rule code: minimum rest between duties
Type rating	A licence to operate a specific aircraft type, valid between two dates
UTC / IST	Coordinated Universal Time / Indian Standard Time (UTC + 5:30)
WDH	Rule code: maximum duty hours in any rolling seven days
3NF	Third normal form

Table A.1: *Definitions, acronyms and abbreviations*